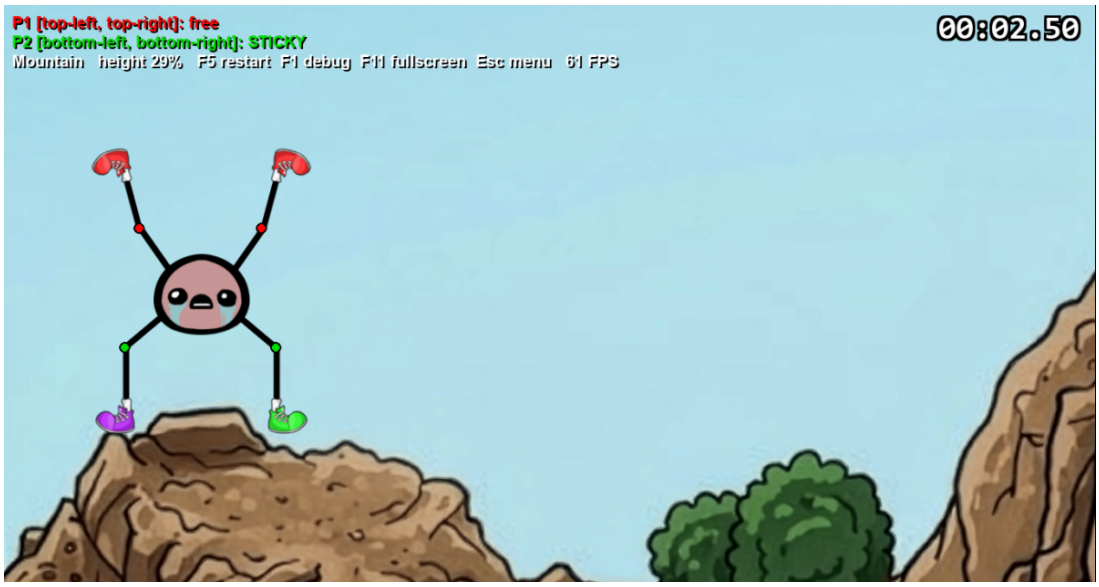
# All For One

## Code Guide

How every part of the game works - structure, data flow, physics, vision, and every file explained block by block with its full source




**From The Group MeEeEe - ME461**

13 October 2026

# Contents

**Part I - The big picture . . . . . 4**

1. What the game is . . . . . 4
2. Running it . . . . . 4
3. The files . . . . . 5
4. How the parts fit together . . . . . 6
5. One frame, start to finish . . . . . 7
6. Coordinates, units and conventions . . . . . 8

**Part II - The core systems . . . . . 10**

7. Terrain and collision: the signed distance field . . . . . 10
8. Physics of the character . . . . . 11
9. From the camera to the legs . . . . . 13
10. Drawing . . . . . 14
11. The screens . . . . . 15
12. Options and settings . . . . . 18
13. The Python environment (Windows and Linux) . . . . . 19

**Part III - Every file, block by block . . . . . 20**

main.py (51 lines) . . . . . 20
app.py (60 lines) . . . . . 21
config.py (174 lines) . . . . . 23
settings.py (148 lines) . . . . . 26
display.py (141 lines) . . . . . 29
minimap.py (50 lines) . . . . . 32
menu.py (166 lines) . . . . . 33
ui.py (136 lines) . . . . . 36
game.py (305 lines) . . . . . 39
scoreboard.py (70 lines) . . . . . 44
test_scene.py (77 lines) . . . . . 46
options_menu.py (169 lines) . . . . . 48
maps.py (84 lines) . . . . . 51
windmill.py (157 lines) . . . . . 53
terrain.py (116 lines) . . . . . 56
flag.py (47 lines) . . . . . 58
viewport.py (44 lines) . . . . . 59
character.py (329 lines) . . . . . 60
assets.py (110 lines) . . . . . 66
inputs.py (67 lines) . . . . . 68

vision.py (384 lines) . . . 70
player_state.py (111 lines) . . . 76
preview.py (85 lines) . . . 78
export_engine.py (17 lines) . . . 80
download_models.py (45 lines) . . . 80
setup.ps1 (99 lines) . . . 81
setup.sh (119 lines) . . . 83
requirements.txt (13 lines) . . . 85
**Part IV - Reference . . . 86**
14. Every setting in config.py . . . 86
15. How do I...? . . . 89
16. Glossary . . . 89
**Part V - How the AI models work . . . 91**
17. Pose detection: YOLO . . . 91
18. Face landmarks: MediaPipe Face Landmarker . . . 91
19. Limits to keep in mind . . . 92

# Part I - The big picture

## 1. What the game is

**All For One** is a co-op climbing game. Up to four people stand in front of one webcam and together control one cartoon character with four legs. Every player's arm (shoulder and elbow) drives one of the character's legs (hip and knee). There are no buttons: you push the character up the mountain by moving your arms, and you make its shoes sticky by opening your mouth. The goal is to touch the flag with the character's head as fast as possible.




*The game window: the game view on the left; on the right the camera picture and the whole map with the character (yellow dot). P2's mouth is open, so the bottom shoes are sticky - purple = glued to the rock, green = sticky but still in the air.*

Behind that simple idea there are four separate problems, and the code is organised around them:

- **Seeing the players** - find every person's body joints and mouth in the webcam picture (`vision.py`, using two AI models).
- **Turning bodies into commands** - convert arm positions into leg angles and mouth openings into sticky feet (`player_state.py`, `game.py`).
- **Physics** - make a character with rigid legs fall, stand, walk, jump and climb on an arbitrary drawn landscape (`terrain.py`, `character.py`).
- **Showing it all** - menus, the game view that follows the character, the camera preview, fullscreen (`display.py`, `menu.py`, `game.py`, `preview.py`).

## 2. Running it

Once per computer, run the setup (details in chapter 13). It creates a private Python environment with the right package versions and downloads the AI models:

| System | Set up once | Then start the game |
|---|---|---|
| Windows | double-click `setup.bat` | double-click `run.bat` |
| Linux | `./setup.sh` | `./run.sh` |

Options can be added after `run.bat` / `./run.sh` (or after `python main.py` inside the environment):

| Option | What it does |
|---|---|
| `(none)` | Normal game with webcam 0. |
| `--source 1` | Use another webcam (or a video file path). |
| `--keyboard` | No camera: legs move with Q/A W/S E/D R/F U/J I/K O/L P/; keys, hold 1-4 to open a player's mouth. Good for testing physics. |
| `--debug` | Start with collision circles shown (same as pressing F1). |

In the game: **F11** fullscreen, **F5** restart, **F1** collision circles, **Esc** back to the menu. On the title screen: **OPTIONS** (or O) to tune the game, **1P TEST** (or T) for the test screen.

## 3. The files

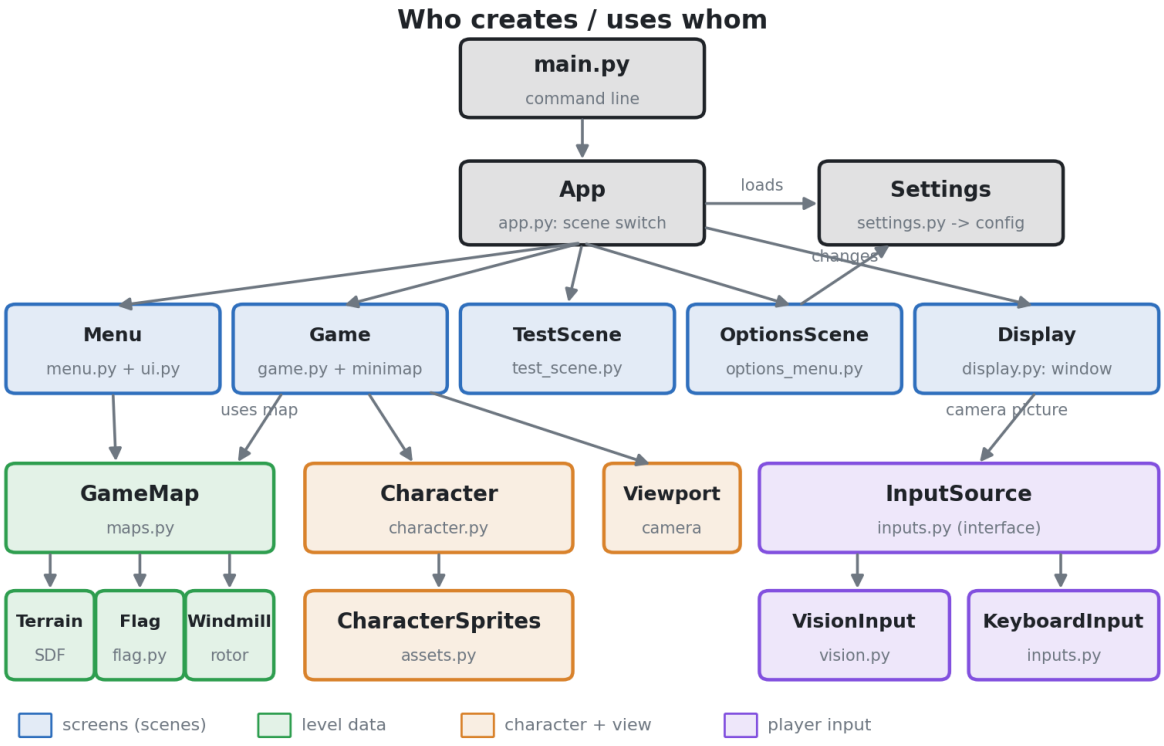
Every file has one job (the assignment asked for separate files per task and object-oriented code). The right column says which class or function to look for.

| File | Job | Main contents |
|---|---|---|
| `main.py` | Entry point: read the command line, choose camera or keyboard | `main()` |
| `app.py` | Create the window and switch between the screens | `App` |
| `config.py` | Every tunable number in one place | constants |
| `settings.py` | Which settings players may change, their safe ranges, settings.json | `Setting`, `Settings` |
| `options_menu.py` | The Options screen (sliders, switches, live readout) | `OptionsScene` |
| `display.py` | The one window: game panel + camera and map panels, scaling, fullscreen | `Display` |
| `minimap.py` | The whole-map overview under the camera | `MiniMap` |
| `menu.py` | Title page, player-count page, map page | `Menu` |
| `ui.py` | Buttons, sliders, switches and outlined text | `Button`, `MapButton`, `Slider`, `Toggle`, `draw_text` |
| `game.py` | The level: loop, physics steps, HUD, timer, finish, name entry | `Game`, `apply_players` |
| `scoreboard.py` | Finish times with team names, saved in scores.json | `ScoreBoard` |
| `test_scene.py` | 1-player test screen (no physics) | `TestScene` |
| `maps.py` | Load a map: scale it, find start point and flag | `GameMap`, `load_maps` |
| `terrain.py` | Map picture + collision (signed distance field) | `Terrain` |
| `windmill.py` | Windmill map: tower picture + spinning, solid rotor | `Windmill` |
| `flag.py` | The goal flag: placement, touch test, drawing | `Flag` |
| `viewport.py` | The game camera that follows the character | `Viewport`, `view_size` |
| `character.py` | Legs, kinematics, physics, drawing of the character | `Leg`, `Character` |
| `assets.py` | Character pictures: crop, scale, stretch, recolour, rotate | `PivotSprite`, `CharacterSprites` |
| `inputs.py` | Input interface + keyboard input | `InputSource`, `KeyboardInput` |
| `vision.py` | Webcam thread: pose, faces, players, mouth, head tilt | `VisionInput` and helpers |
| `player_state.py` | Player data + arm-angle maths | `PlayerState`, `limb_poses` |
| `preview.py` | Draws skeletons and lips on the camera picture | `PreviewRenderer` |
| `export_engine.py` | Builds the TensorRT engine | script |

| File | Job | Main contents |
|---|---|---|
| `download_models.py` | Downloads both AI models, reports GPU/CPU (run by the setup) | script |
| `setup.ps1 / setup.bat` | Windows setup: environment, packages, models | script |
| `setup.sh` | Linux setup: environment, packages, models, system checks | script |
| `run.bat / run.sh` | Start the game with the environment | script |
| `requirements*.txt` | Pinned package versions: game / optional GPU / optional docs tools | lists |
| `tools/docs/` | Rebuilds this PDF from the code (`make_docs.py`) | scripts |
| `pose_and_face_test.py` | Your original pose + face experiment (not used by the game) | script |

The `assets/` folder holds the pictures: `background*.png` (sky and far scenery), `foreground*.png` (the rock you collide with - its transparent pixels are air), the character parts `head.png`, `leg-part.png`, `leg-part-with-shoe.png`, `character-demo.png` (menu picture) and `flag.png`. The windmill map adds `map3-windmillbase.png` (the tower, only a picture) and `map3-windmillblade.png` (the rotor, solid); `map3-fullmapdemo.png` shows how the map looks assembled. The two AI model files `yolo26n-pose.pt` and `face_landmarker.task` download themselves on first run.

## 4. How the parts fit together



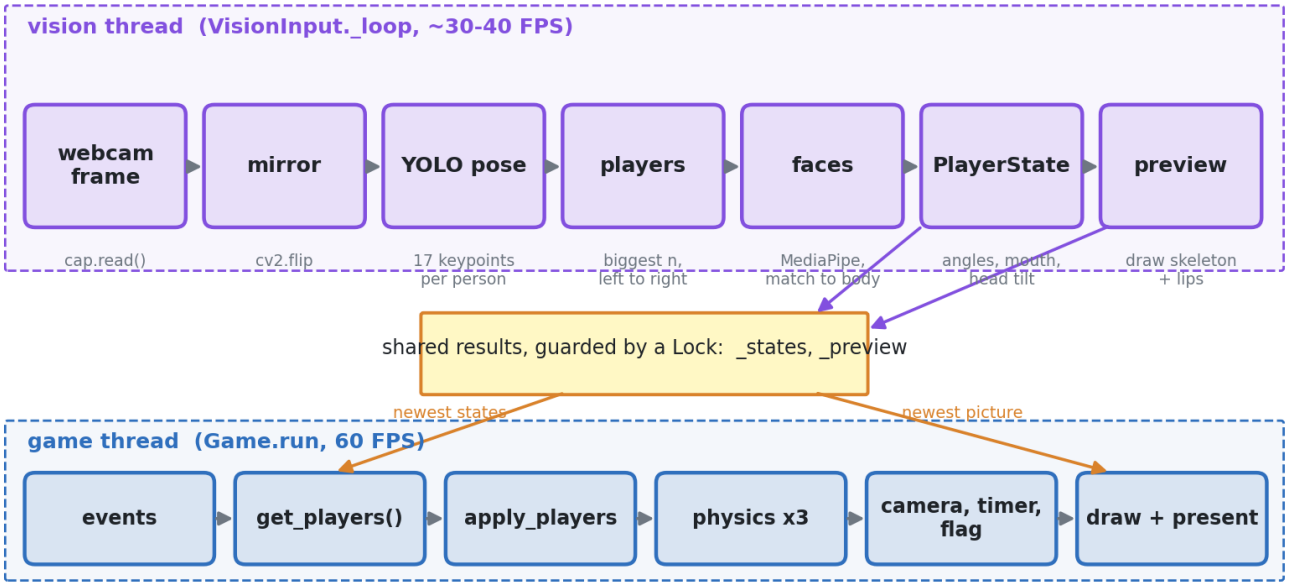

*Who creates and uses whom. Arrows point from the user to the thing it uses.*

Reading the diagram top-down: `main.py` creates an **input source** (camera or keyboard) and gives it to `App`. `App` creates the window (`Display`), loads the maps and creates the three **scenes** (Menu, Game, TestScene). A scene is a screen with its own loop. `App.run()` asks the menu what to do, runs that scene until it returns, and goes back to the menu.

The game scene uses a `GameMap` (which owns a `Terrain` for collisions, a `Flag` and, on the windmill map, a `Windmill`), a `MiniMap` (the overview under the camera) and a `ScoreBoard` (finish times in scores.json), builds a `Character` (which uses `CharacterSprites` for its pictures) and a `Viewport` (the game camera). It never talks to the webcam directly - it only calls `input.get_players()`. That is the key design idea: **the game does not know whether a camera or a keyboard is behind the input**. Both classes inherit from `InputSource` and offer the same methods (`get_players`, `get_preview`, `set_num_players`, `close`). This is called an *interface*, and it is why keyboard mode was trivial to add.

## 5. One frame, start to finish

Two things run at the same time (two *threads*). The **vision thread** reads the webcam and runs the AI models as fast as it can (about 30-40 times per second). The **game thread** runs the game at a steady 60 frames per second. They meet in one place: a small shared box with the newest results, protected by a *lock* so the game never reads a result while the vision thread is half-way through writing it.




*The two loops and the shared results between them.*

Why two threads? Running the AI models takes 20-110 ms per picture. If the game waited for them every frame, it would stutter at 9-30 FPS. With a separate thread the game always runs smoothly and simply uses the newest available result.

### Start-up, in order

1. `main()` parses the command line and creates `VisionInput` - this loads the pose model (picking TensorRT, GPU or CPU), the face model, opens the webcam and starts the vision thread.
2. `App.__init__` starts pygame, applies the player's saved settings from `settings.json` (chapter 12) and creates the `Display` (the window).
3. `load_maps()` loads every map in `config.MAPS`: scales the pictures, computes the collision field, places the flag and finds the start point (about 2 seconds in total).
4. The three scenes are created. `App.run()` shows the menu.
5. The player picks PLAY, a player count and a map; `Game.run(players, map)` starts the level.

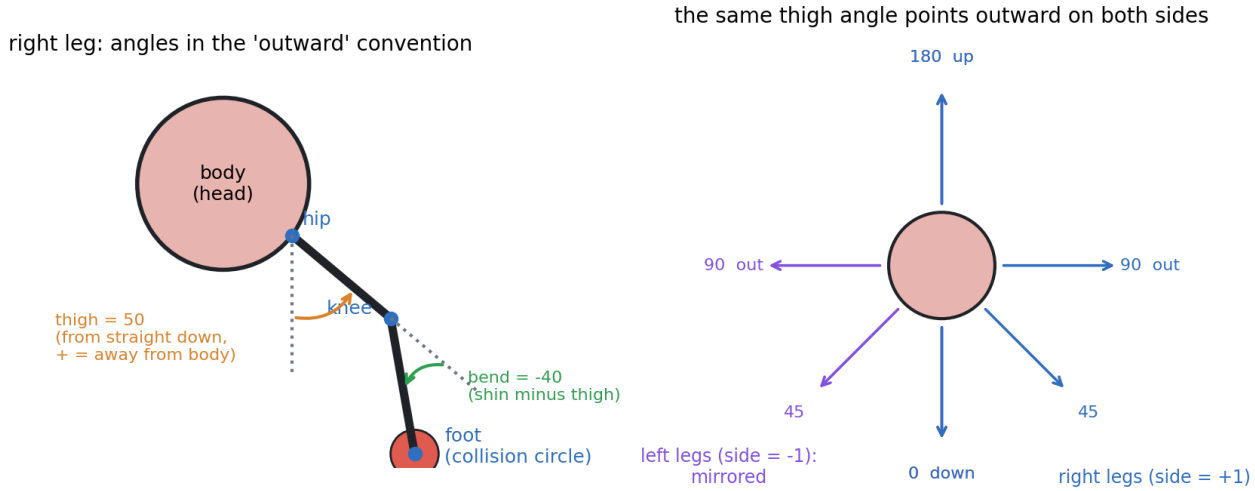
### Every game frame (Game.run)

1. `clock.tick(60)` waits so frames are 1/60 s apart and returns the real time since the last frame (`dt`).
2. Keyboard/window events: Esc, F5, F1, F11, closing the window.
3. `input.get_players()` returns one `PlayerState` per player (limb angles, mouth, head tilt).
4. `apply_players` copies each controlling limb's angles into its leg as a *target*, and sets each leg's `sticky` flag from the mouth.
5. Physics: `character.update()` three times with `dt/3` each (substeps).
6. The viewport glides towards the character; the timer runs; the flag touch is checked.
7. Drawing: map part, flag, character, knee dots, texts, timer, and `display.present()` shows it.

## 6. Coordinates, units and conventions

Most confusion in game code comes from coordinates. These are the conventions used everywhere here:

- **Map pixels** are the unit of the world. All maps are scaled so the mountain map is 2164 px tall (`MAP_HEIGHT`); positions, speeds (px/s) and gravity (px/s^2) are in map pixels.
- **y points down** (screen convention). Gravity is `+y`; "up" means a smaller y. The HUD height % is computed as `map height - y`.
- **Three sizes of picture**: the *map* (e.g. 4096 x 2164), the *canvas* (1365 x 721, the size the scenes draw at = what the viewport shows) and the *window* (whatever fits your screen). `display.present()` scales canvas to window; `display.to_canvas()` converts mouse clicks back.
- **Screen position = map position - viewport position.** That is why every `draw()` gets an `offset`.
- **Angles are degrees.** Leg and arm angles use the *outward* convention below.
- **Colours**: pygame uses (R, G, B), OpenCV uses (B, G, R). `config.PLAYER_COLORS_BGR` is the master list and the RGB list is made by reversing each tuple.




*Left: a leg is two angles. Right: left legs are mirrored so 'away from the body' is positive on both sides.*

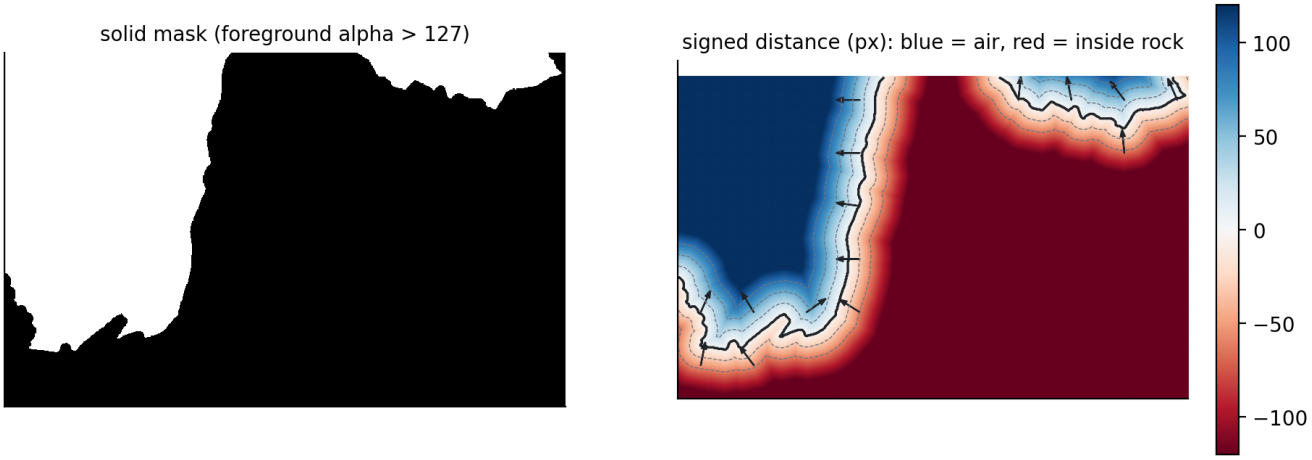
A leg is fully described by two numbers: **thigh** = the hip angle measured from straight down (0 = hanging, 90 = sideways, 180 = up) and **bend** = how much the shin turns relative to the thigh. The *same* two numbers are measured on the player's arm (shoulder angle and elbow angle), which is why an arm can drive a leg directly: arm hanging down -> leg hanging down, arm raised -> leg raised.

To draw or compute a leg on screen, the outward angle is multiplied by the leg's `side` (-1 left, +1 right) to get a normal screen rotation (`Leg.screen_angles`).

# Part II - The core systems

## 7. Terrain and collision: the signed distance field

The landscape is a drawing, not a set of boxes or polygons, so collision must work on pixels. The foreground picture's transparency decides what is solid: a pixel more than half opaque is rock. `Terrain` turns that yes/no mask into a **signed distance field (SDF)**: a table with one number per pixel telling how far that pixel is from the nearest rock surface - positive in the air, negative inside rock, zero on the surface.




*A part of the mountain. Right: the distance field; the black line is distance 0 (the surface), dashed lines are every 30 px, arrows are the normals (the way out).*

OpenCV computes this table once per map with `cv2.distanceTransform` (run on the air and on the rock, then subtracted). After that, three questions are cheap lookups:

- **Is this point inside rock, and how deep?** `terrain.distance(x, y)`. A foot circle of radius r is in the ground when `distance < r`, and it is `r - distance` deep.
- **Which way is out?** `terrain.normal(x, y)` - the direction in which the distance grows fastest (its gradient), found by sampling the distance 2 px left/right and up/down.
- **Where is the ground under the sky in this column?** `terrain.ground_y(x)`, used to place the flag.

```
distance(x, y) = bilinear blend of the 4 table cells around (x, y)
normal = normalize( d(x+2,y) - d(x-2,y) , d(x,y+2) - d(x,y-2) )
```

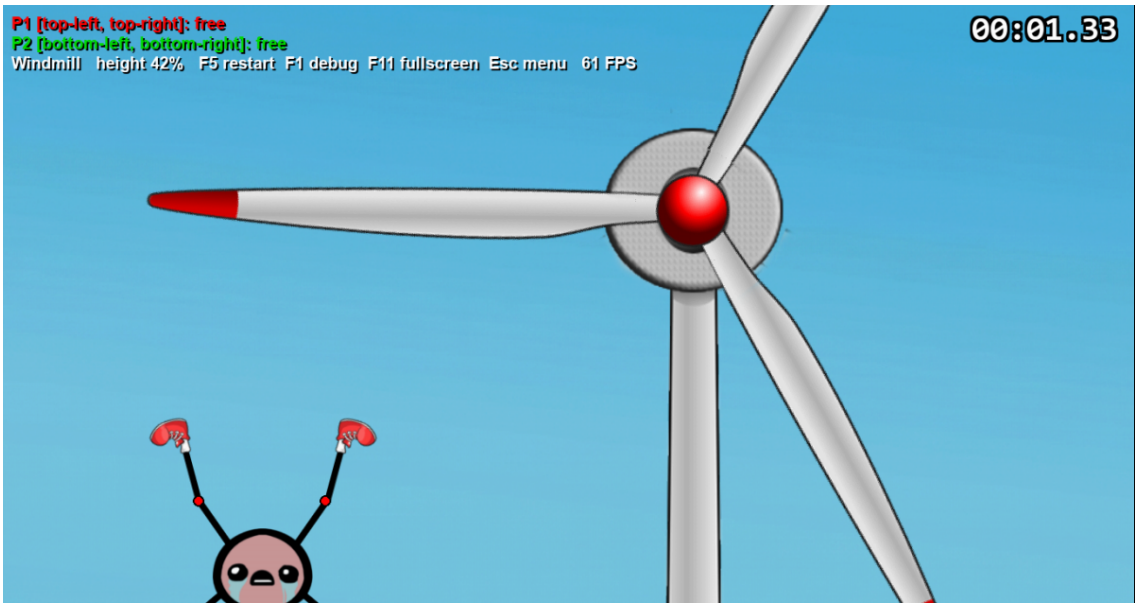
**Bilinear blending** means the distance changes smoothly between pixels instead of in 1 px steps, which keeps the physics from jittering. Points outside the map are handled specially: beyond the left and right edges counts as ever-deeper rock (invisible walls), and above the top edge the top row continues upward - sky stays sky, but a solid top border (like the cave's) becomes rock that gets deeper going up, so nothing can escape through it.

### Moving parts: the windmill rotor

The windmill map has a solid part that moves. Its rotor gets its own distance table, made once for the rotor standing still. To ask how far a point is from the rotor *now*, the point is turned back around the hub by the rotor's current angle and looked up in that still table - turning the question is much cheaper than rebuilding the table 180 times a second. `Terrain.distance` returns the nearest of the rock and every moving part, so the physics needs no special code to collide with the rotor. `Terrain.mover_at` tells which moving part a point touches.

```
local point = hub + rotate(point - hub, -angle)
rotor distance(point) = still table at the local point
```

The rotor turns clockwise at `rps` = 0.25 turns per second (set per map in `MAPS`). The tower is just part of the map picture and has no collision. Drawing the rotor means rotating a big picture every frame, so it is cut into four pieces (three blades and the hub); only pieces inside the view are rotated. The tower and the rotor are drawn after the character, so the character passes behind them.




*The windmill map: the rotor is solid and turning; the map panel shows it too.*

## 8. Physics of the character

The character is deliberately simple: **only the body (the head) has a position and a velocity**. It cannot rotate. The legs have no mass - each leg is just two angles that turn towards the angles the player asks for, at most 420 degrees per second. Where a foot is follows from the body position plus the two angles (*kinematics*, `Leg.joints`):

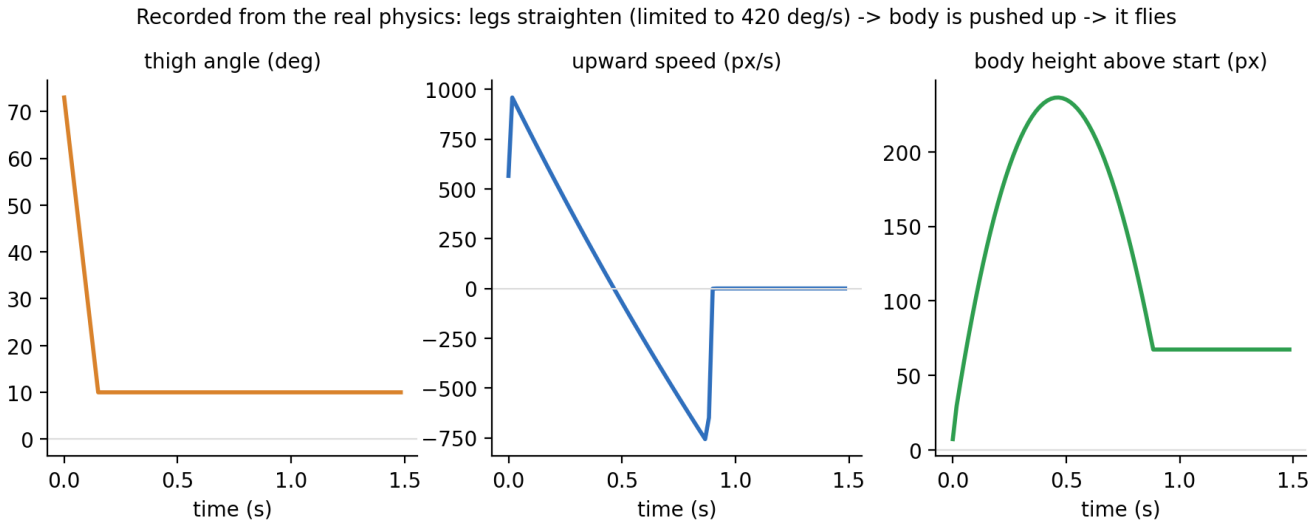
```
hip = body + hip_offset
knee = hip + rotate((0, thigh_length), thigh angle)
foot = knee + rotate(foot_offset, shin angle) # shin angle = thigh + bend
```

The method is **position-based dynamics** (PBD): instead of computing forces, each step first moves the body freely, then *corrects its position* until all contacts are satisfied, and finally derives the velocity from how far the body actually moved. One `_step` does:

1. **Sticky feet** - a sticky foot that is within 8 px of the rock gets *glued*: its position is remembered as an anchor. A foot that stops being sticky lets go.
2. **Predict** - add gravity to the velocity, apply a little air drag, move the body by velocity x dt.
3. **Solve** (4 passes) - for every foot: if glued, move the body so the foot goes back towards its anchor; otherwise, if the foot is inside rock, push the body out along the normal and apply friction. Then push the head out of the rock. Each correction can upset another, so the passes repeat (the Gauss-Seidel method).
4. **Rock has the last word** - two extra passes make sure no glued foot is more than 6 px deep.
5. **Velocity** - `velocity = (new position - old position) / dt`, then limits (head friction, max speed, stay inside the map). The 1500 px/s limit applies to the whole speed whenever something pushed or pulled the body; in free flight only the falling speed is limited, so a sideways fling keeps its sideways speed instead of drifting down slowly.

## Why this gives jumping and walking for free

**Jumping**: if a planted leg straightens quickly, its foot would go into the ground. The solver pushes the body up instead, by exactly the amount the foot would have sunk. Because velocity is *measured* from the movement, the body now has upward speed and keeps flying after the foot leaves the ground. The recording below comes from the real physics:




*A recorded jump on the mountain: the thigh angle drops (leg straightens), the body gets about 950 px/s upward, rises ~240 px and lands.*

**Walking**: when a free foot first touches walkable ground (flatter than 55 degrees) it remembers that point (`grip`). While it stays down, the solver does not let the foot slide along the surface (*static friction*). So if the player sweeps the leg backwards, the foot cannot move - the body moves forward instead. On steeper slopes there is no grip and feet slide, which is what makes walls need sticky feet.

**Climbing**: a glued foot works in both directions - it can push *and pull* the body. Hanging from a ledge, a player can bend the glued leg to pull the body up. If the glue is stretched more than 40 px it tears off.

**Riding the rotor**: a foot glued to the rotor remembers that (`anchor_on`). At the start of every physics step its anchor is turned with the rotor (`Windmill.carry`), so the body is pulled along. Because velocity is measured from movement, letting go flings the character with the speed it had. Free (non-sticky) feet only grip the rotor where it is flatter than `ROTOR_WALKABLE_SLOPE` (10 degrees instead of 55), so without sticky feet the character slips off the blades quickly. A blade that overlapped a free foot or the head deeply would push it out in one step - a huge kick - so leaving a blade may be at most `ROTOR_PUSH_SPEED` (400 px/s) faster than the blade moves there (`Windmill.velocity_at`).

## Keeping feet out of the rock while glued

With a foot glued, the body is pinned and cannot always move out of the way, so a leg could be forced into the rock. `Character.update` therefore checks every step: if a moving leg pushed its own foot deeper than allowed (3 px for free feet, 6 px for sticky feet), the step is undone and redone with that leg held still - the leg stops at the rock. A glued foot pushed in by *another* leg is held at 6 px by the extra solver passes.

## Substeps

Physics runs three times per frame with a third of the time each. Smaller steps mean feet move less per step, so they cannot jump through thin rock, and the solver stays stable.

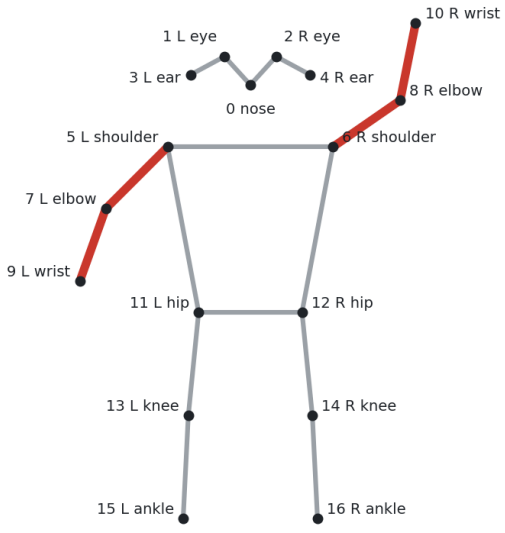
| Setting | Value | Effect when you raise it |
|---|---|---|
| GRAVITY | 2000 px/s^2 | Heavier, falls faster, lower jumps |
| LEG_MAX_SPEED | 420 deg/s | Legs react faster, kicks and jumps get stronger |
| MAX_SPEED | 1500 px/s | Allows faster flights |
| WALKABLE_SLOPE | 55 deg | Feet grip on steeper slopes without sticking |
| SOLVER_ITERATIONS | 4 | Stiffer, more exact contacts (slower) |
| STICKY_BREAK_DIST | 40 px | Glue holds longer before tearing off |
| PHYSICS_SUBSTEPS | 3 | More stable, slower |

## 9. From the camera to the legs

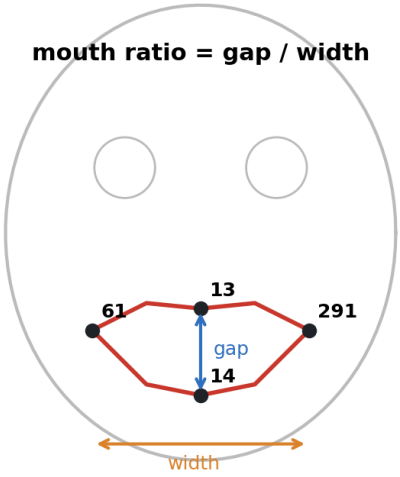
`VisionInput._loop` runs in the background thread. For each webcam frame it does:

1. **Mirror** the picture so it behaves like a mirror (your right hand appears on the right).
2. **Pose**: YOLO finds every person and 17 body keypoints each, with a confidence per keypoint.
3. **Number the players** (`assign_players`): take the n biggest people (closest to the camera) and number them left to right. With fewer people than players, the picture is split into n vertical strips and you get the number of the strip you stand in.
4. **Faces**: MediaPipe finds face landmarks on the whole frame. `match_faces` gives each face to the person whose head keypoints are closest in pixels (closest pairs first). A player without a face gets a second try on a crop around their head.
5. **Per player**: limb angles (`limb_poses`) smoothed by `AngleSmoother`, the mouth state (`MouthTracker`) and, with one player, the head tilt (`HeadTilt`).
6. **Preview**: draw the skeletons and lips on a copy of the frame (`PreviewRenderer`).
7. Store the results under the lock.

COCO 17 keypoints (YOLO pose output); controlling arms coloured, rest grey




4 of the 478 face-mesh points used for the mouth




*Left: the 17 keypoints of one body. Right: the four face points used for the mouth.*

### Arm angles

For an arm, the shoulder angle is the direction of shoulder -> elbow and the elbow angle is the direction of elbow -> wrist, both measured from 'down' and corrected by the torso direction (shoulders -> hips) so leaning does not swing the legs. Left limbs are mirrored (multiplied by -1) to

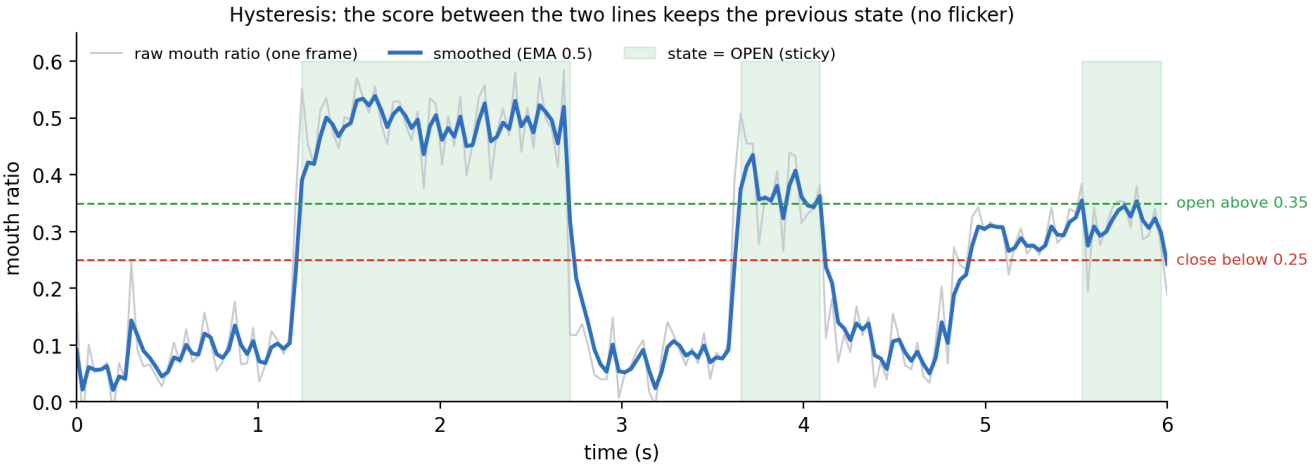
get the outward convention. Sides come from where the joints are on screen, not from the model's left/right labels, because on a mirrored picture those can be swapped.

```
heading(v) = atan2(v.x, v.y) # angle from straight down
thigh = side * wrap(heading(elbow - shoulder) - torso)
bend = wrap(side * wrap(heading(wrist - elbow) - torso) - thigh)
```

Keypoints jitter by a few pixels every frame. `AngleSmoother` applies an *exponential moving average* (EMA): each new value moves the smoothed value 60% of the way. It is *wrap-safe*: going from 179 to -179 degrees is treated as a 2 degree step, not a 358 degree swing.

### Mouth: open or closed

The mouth ratio is the gap between the lips divided by the mouth width. Dividing by the width makes it the same whether the face is near (big) or far (small). The ratio is smoothed, then a **hysteresis** decides the state: it opens only above 0.35 and closes only below 0.25. Between the two lines the state stays as it was, so a ratio hovering around one value cannot make the feet flicker.




*Hysteresis on a made-up recording: the 0.30 bump at t = 3.8 s is not enough to open; noise around the thresholds does not toggle the state.*

If the face is lost for a moment, the last state is kept for 0.5 s before it falls back to closed.

### Head tilt (1 player)

With one player, both legs belong to the same mouth, so the sideways tilt of the head picks which foot the open mouth makes sticky. The roll angle is the slope of the line between the ears (or the eyes if an ear is hidden). Tilting right (right ear lower) beyond 10 degrees selects the right foot, tilting left the left foot; in between, the last choice is kept.

## 10. Drawing

### Sprites that rotate around a joint

All character pictures share one 500 x 500 canvas, and the joint positions were measured on it (`config.LEG_PIVOT` etc.). `PivotSprite.draw` rotates an image around its joint: pygame rotates around the image centre, so the code rotates the vector from the joint to the centre by the same angle and places the rotated image's centre there - the joint ends up exactly where it should.

Which way a toe points: the shoe picture points right while the shin hangs down, so left legs use a mirrored copy to point outward. The two upper legs are usually raised, which turns the shoe upside down and its toe inward - so the legs listed in `config.MIRRORED_SHOES` (the upper two) use the

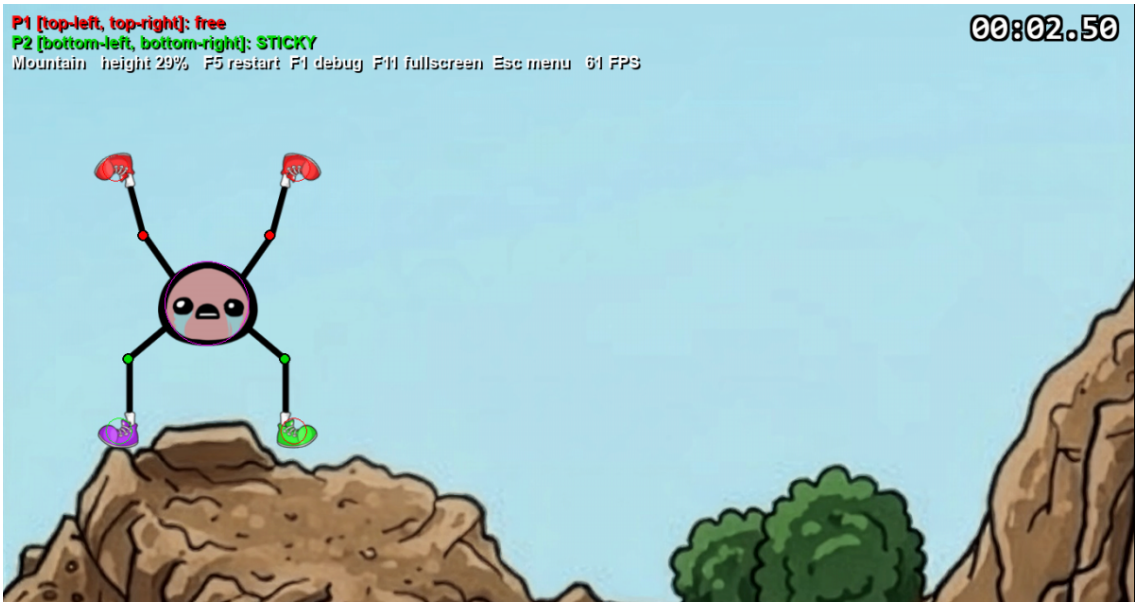
opposite copy, and their toes point outward too, like in `character-demo.png`. The foot's collision circle follows the same choice (`Leg.shoe_side`).

Pictures are prepared once: cropped to their content (so rotating is cheap), scaled, mirrored for left legs (toes point outward) and recoloured. The recolour filter takes clearly red pixels and moves their 'redness' into other channels, which keeps the shading: green = sticky, purple = glued. For the 1-player long legs, `stretch_leg` stretches only the black stick part and moves the shoe down unchanged.

### One window, three panels

Every scene draws onto the same canvas (1365 x 721). `Display.present()` fills the window, scales the canvas into the left panel, draws the newest camera picture at the top of the right column and the map overview under it, and calls `pygame.display.flip()`, which shows the finished frame all at once (no flicker). F11 recreates the window in fullscreen and recomputes the layout.

The map panel (`MiniMap`) shows the whole map with its flag, a white frame around the part the game view shows and a yellow dot for the character. The shrunk map picture is made once per panel size; only the frame, the dot and the windmill rotor are drawn each frame. On the other screens the panel shows a short note.



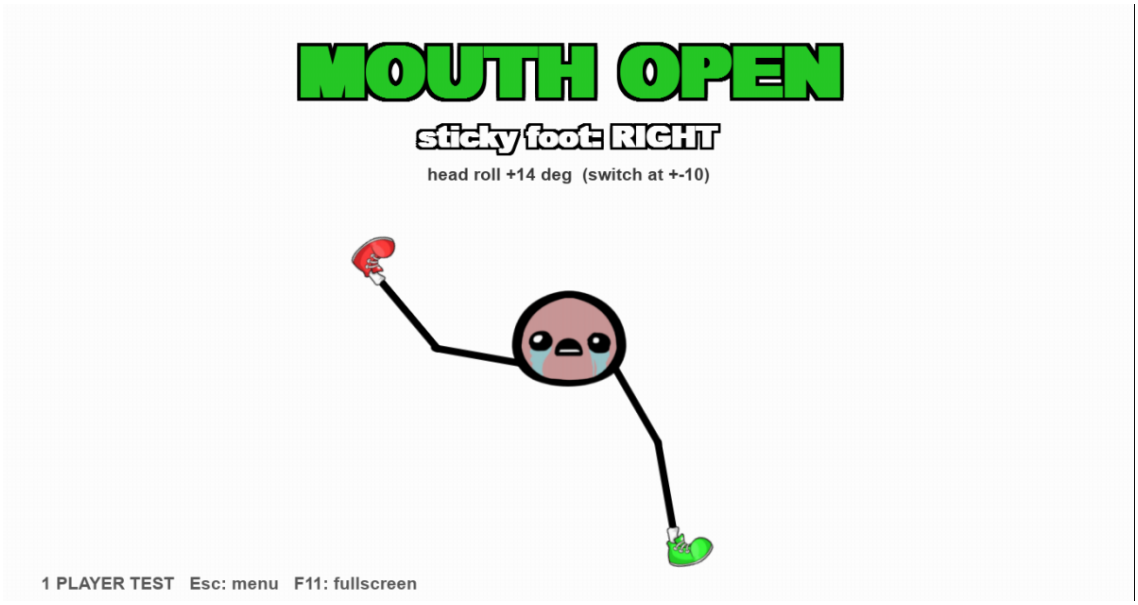

*F1 debug view: magenta = head circle, feet circles coloured green (glued), yellow (touching) or red (in the air).*

## 11. The screens

The menu has three pages in one loop (`page` = title, players, maps). PLAY leads to the player count, then to the map. 1P TEST opens the test screen. Every button position is a fraction of the canvas size.




*The map page. Each picture is the whole map shrunk, with its flag; the cards are laid out side by side with equal gaps.*




*The 1-player test screen: no physics, the body stays still and the legs follow the arms; mouth state in big letters.*




*Touching the flag with the head stops the timer and asks for the team name.*

## The score board

After the flag, the team types a name (up to 16 characters, any keyboard layout - the letters arrive as `TEXTINPUT` events). **Enter** saves, **Esc** skips saving. The board then shows the 8 fastest times on this map with this many players; the new run is gold, and if it is not in the top 8 it takes the last line with its real place. Enter goes to the menu, F5 plays again.

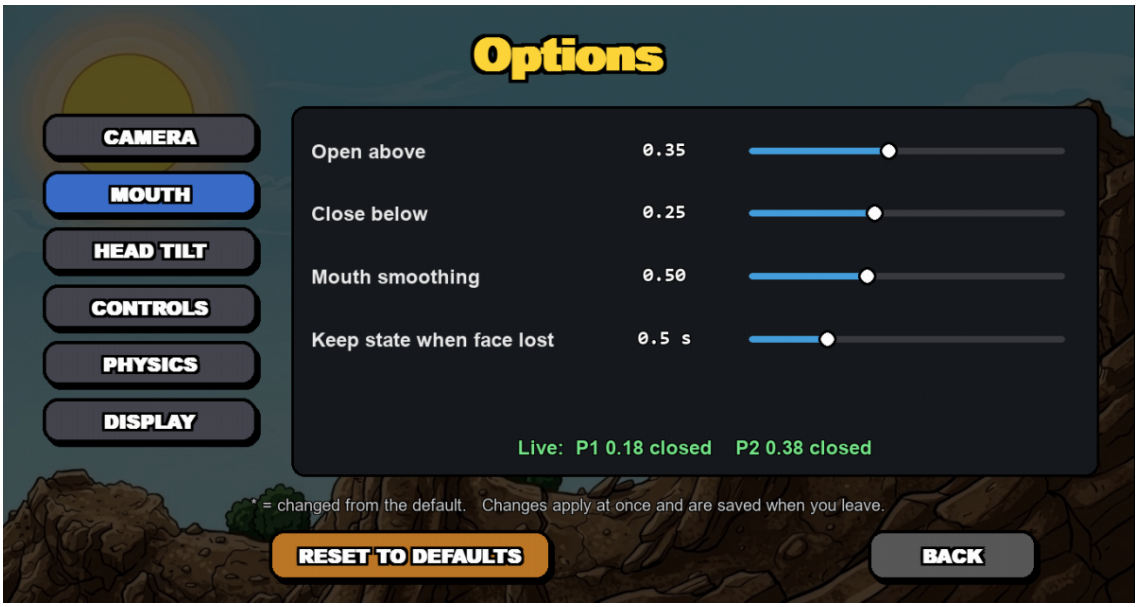
`ScoreBoard` keeps every saved run in `scores.json` (name, map, player count, time, date) and never deletes any. It writes a temporary file first and then swaps it in, so a crash cannot leave a half-written file. A broken file is renamed to `scores.json.broken` and the board starts empty. Like `settings.json`, the file is not in git.




*The score board after saving (demo names).*

# 12. Options and settings

The **OPTIONS** button on the title screen lets players tune the game without touching code: camera, mouth, head tilt, controls, physics and fullscreen. Changes apply at once and are remembered in `settings.json` next to the game.




*The Mouth tab. The green line shows each player's live mouth score, so the thresholds can be set while watching it. A gold * marks a changed value.*

## How a setting reaches the game

There is no separate settings system inside the game: every part already reads its numbers from `config` (for example `config.GRAVITY` in the physics step). A `Setting` in `settings.py` names one of those values, and `Settings.set()` simply writes the new value into `config` - so the next physics step, the next mouth check or the next arm reading already uses it.

1. Start-up: `App` creates `Settings`, which first remembers the values written in `config.py` as the defaults, then `load()` reads `settings.json` and applies it.
2. In the Options screen a slider or switch calls `settings.set(key, value)`.
3. Leaving the screen (BACK, Esc or closing the window) calls `save()`, which writes only the values that differ from the defaults. **RESET TO DEFAULTS** puts the `config.py` values back.

For this to work a value must be *read when used*, not copied once at start-up. Two places did copy: `limb_poses` (joint confidence) and `AngleSmoother` (arm smoothing); they now read `config` each time.

## Why it cannot break the game

- Only safe values are offered. Things like the camera number, model files, sprite geometry or solver internals are not in the list, because a wrong value there stops the game.
- Every number has a minimum, maximum and step (`Setting.clamp`); anything outside is pulled back in.
- The two mouth thresholds are kept in order: 'close' always stays at least 0.05 below 'open'.
- A missing, broken or hand-edited `settings.json` never crashes the game: wrong types and unknown names are ignored, numbers are clamped, unreadable files fall back to the defaults.
- `settings.json` is not in git: every computer keeps its own tuning.

| Tab | Settings |
|---|---|
| Camera | mirror on/off, joint confidence needed |

| Tab | Settings |
|---|---|
| Mouth | open threshold, close threshold, smoothing, keep state when the face is lost |
| Head tilt | switch angle, swap sides (1 player) |
| Controls | leg speed, arm smoothing |
| Physics | gravity, grip on slopes, sticky-foot tear-off distance |
| Display | fullscreen (applied at once, remembered) |

# 13. The Python environment (Windows and Linux)

The game depends on a few big libraries (PyTorch, Ultralytics, MediaPipe, OpenCV, pygame) that change often. To make the game install the same way on every computer, it runs in its own **virtual environment**: a private folder `.venv` with exactly the tested package versions, separate from any other Python projects on the computer.

## What the setup script does

1. Finds **Python 3.12** - exactly that version, because newer ones (3.13, 3.14) do not have mediapipe/PyTorch packages yet. If it is missing, Windows offers to install it with `winget`; Linux offers to get it with `uv`, which downloads a private Python 3.12 for your user (no `sudo`, the system Python is not changed).
2. Creates `.venv` with Python's built-in `venv` module (or updates it if it exists). A `.venv` made with another Python version is deleted and rebuilt with 3.12.
3. Installs **PyTorch first**: the CUDA build when an NVIDIA GPU is found (`nvidia-smi`), otherwise the small CPU build. It must come first, because `ultralytics` would otherwise pull a default build.
4. Installs the pinned game packages from `requirements.txt`.
5. Optionally (NVIDIA only, asks y/N) installs TensorRT and builds the engine (`export_engine.py`).
6. Runs `download_models.py`: downloads both AI models and reports whether the GPU will be used.

| File | Contents |
|---|---|
| `requirements.txt` | pygame, numpy, OpenCV, mediapipe, ultralytics (exact versions). Both OpenCV packages are pinned to the same version: they install the same `cv2` module and would overwrite each other otherwise. |
| `requirements-gpu.txt` | TensorRT (optional, NVIDIA) |
| `requirements-dev.txt` | tools to rebuild this PDF (not needed to play) |

## Linux specifics

- `setup.sh` checks things Windows does not need: that the `venv` module is installed (`sudo apt install python3.12-venv`), that OpenCV finds its system library (`sudo apt install libgl1 libglib2.0-0`), and that your user may open the webcam (`sudo usermod -aG video $USER`, then log in again).
- Fonts: the game asks for a list of fonts (`config.FONT_*`) and uses the first one installed - Arial and Consolas on Windows, DejaVu or Liberation on Linux. If no emoji font exists, the sheep in the menu is simply left out instead of showing an empty box.
- NVIDIA on Linux: the CUDA 13 build of PyTorch needs driver 580 or newer; with an older driver `download_models.py` says so and the game runs on the CPU. The TensorRT engine must be built on each computer anyway (`export_engine.py`), because it only works on the GPU it was made for.
- `.gitattributes` keeps `.sh` files with Linux line endings even when committed from Windows.

# Part III - Every file, block by block

Each file below starts with what it is for, then a table that walks through the file from top to bottom (the line numbers match the real file), then the complete source with its comments. Read the table next to the listing: the table says *why*, the comments in the code say *what*.

## main.py (51 lines)

The entry point. It only reads the command line, creates the right input source and starts the app.

| Lines | What happens there |
|---|---|
| 1-12 | Module docstring: how to start the program and what to install. |
| 13-18 | Imports. `KeyboardInput` is cheap to import; the camera input is imported later. |
| 21-26 | Warn if this is not Python 3.12 (the version the packages are pinned for). |
| 28-34 | Declare the four command-line options; `parse_args` reads what the user typed. |
| 36-44 | Pick the input source. The vision module is imported only here because loading torch/ultralytics/mediapipe takes seconds. |
| 45 | Create the app with that source and run it until the player quits. |
| 48-51 | Standard Python idiom: run `main()` only when the file is started directly. |

**Full source**

```python
"""All For One - entry point.

    python main.py                  # webcam 0
    python main.py --source 1       # another webcam
    python main.py --keyboard       # no camera, keyboard debug controls (see inputs.py)
    python main.py --debug          # draw collision circles

Install: pip install pygame ultralytics mediapipe opencv-python

This file only reads the command line, picks an input source (camera or
keyboard) and hands it to App, which runs everything else.
"""
import argparse
import sys

import config
from app import App
from inputs import KeyboardInput


def main():
    # The packages are pinned for Python 3.12 (newer Pythons lack mediapipe / PyTorch
    # builds). run.bat / run.sh use the .venv made by the setup, which is 3.12.
    if sys.version_info[:2] != (3, 12):
        print(f"Warning: this is Python {sys.version.split()[0]}, the game is made for 3.12.\n"
              "Start it with run.bat / ./run.sh (after setup.bat / ./setup.sh).")

    # Describe the command-line options; argparse also builds `--help` from this.
    ap = argparse.ArgumentParser()
    ap.add_argument("--source", default=config.CAMERA_SOURCE, help="webcam index or video path")
    ap.add_argument("--model", default=config.POSE_MODEL)
    ap.add_argument("--keyboard", action="store_true", help="play without a camera")
    ap.add_argument("--debug", action="store_true")
    args = ap.parse_args()

    # Both input sources have the same methods (see inputs.InputSource), so the
    # rest of the game never needs to know which one it got.
    if args.keyboard:
        source = KeyboardInput()
    else:
        # Imported here, not at the top: loading torch/ultralytics/mediapipe takes
        # seconds and is not needed in keyboard mode.
        from vision import VisionInput   # heavy imports only in camera mode
        source = VisionInput(args.source, args.model)
    App(source, debug=args.debug).run()


# Run main() only when this file is started directly (python main.py),
# not when another file imports it.
if __name__ == "__main__":
    main()
```

## app.py (60 lines)

Owns the window and the three scenes, and switches between them. Each scene's `run()` returns where to go next.

| Lines | What happens there |
|---|---|
| 1-6 | Docstring explaining scenes. |
| 7-17 | Imports of every scene and helper. |
| 20-24 | Constructor: start pygame, title the window. |
| 25-28 | Load `settings.json` into `config` before anything is built (chapter 12). |
| 30-38 | Create the display first (images can only be converted for fast drawing once a window exists), then load maps, then the scenes. |
| 40-60 | The switchboard loop: menu -> chosen scene (game, test or options) -> back to menu, until quit. The `finally` block always stops the camera thread and closes pygame, even after a crash. |

### Full source

```python
"""App: owns the single window and switches between menu, game and test scenes.

A "scene" is a screen with its own loop (menu, game, 1P test). Each scene's
run() keeps drawing frames until the player leaves it, then returns what
should happen next; App.run() is the switchboard between them.
"""
import pygame

import config
from display import Display
from game import Game
from menu import Menu
from maps import load_maps
from options_menu import OptionsScene
from settings import Settings
from test_scene import TestScene
from viewport import view_size


class App:
    def __init__(self, input_source, debug=False):
        pygame.init()                                   # start pygame: video, fonts, events
        pygame.display.set_caption(config.GAME_TITLE)   # window title bar text
        self.input = input_source
        # Saved player settings go into config first, so everything built below
        # (e.g. the window in fullscreen or not) already uses them.
        self.settings = Settings()
        self.settings.load()

        # The window must exist before images can be converted for fast drawing,
        # so Display comes first, then the maps (which load and convert images).
        self.display = Display(view_size(), input_source)
        self.maps = load_maps()
        # Every scene shares the same display (window) and input source.
        self.menu = Menu(self.display, self.maps, input_source)
        self.game = Game(self.display, input_source, debug)
        self.test = TestScene(self.display, input_source)
        self.options = OptionsScene(self.display, self.maps, input_source, self.settings)

    def run(self):
        # try/finally: even if the game crashes, the camera thread is stopped and
        # the window is closed properly.
        try:
            while True:
                choice = self.menu.run()        # blocks until the player picks something
                if choice is None:              # Esc / window closed on the title page
                    break
                if choice[0] == "test":
                    scene = self.test.run()
                elif choice[0] == "options":
                    scene = self.options.run()
                else:
                    _, players, map_index = choice
                    scene = self.game.run(players, self.maps[map_index])
                if scene == "quit":             # window closed during a scene
                    break
                # otherwise the scene returned "menu": loop back to the menu
        finally:
            self.input.close()
            pygame.quit()
```

## config.py (174 lines)

Every number that changes how the game looks or feels. Nothing here runs logic; the other files read these values. The Part IV reference table lists every setting.

| Lines | What happens there |
|---|---|
| 1-10 | Docstring: units and colour conventions. |
| 11-12 | Paths: the project folder and the assets folder, built from this file's location. |
| 15-17 | Helper: full path of an asset picture. |
| 20-38 | Maps: reference height, the list of maps (name, pictures, spawn, scale) and how high the character starts. |
| 40-53 | Game view size, window layout, frame rate, physics substeps, camera smoothing, sky colour. |
| 55-71 | Character size and the geometry measured on the 500 px sprite canvas (head, joints, feet). |
| 72-80 | The four legs: hip position, side, rest pose. |
| 81-93 | Control schemes for 1-4 players: (player, limb, leg) rows. |
| 94-97 | Player limit, leg stretch for 1 player, where the shoe starts in the shin picture. |
| 99-104 | Flag size and placement settings. |
| 106-121 | Physics constants. |
| 123-161 | Camera, model files, keypoint threshold, face matching and crop settings, mouth and head-tilt thresholds, smoothing, colours. |
| 163-169 | Font lists with Windows and Linux names; pygame uses the first one installed. |
| 171-174 | Title and credit text. |

### Full source

```python
"""All tunable numbers live here so gameplay can be tweaked without touching logic.

Every other file does `import config` and reads values like `config.GRAVITY`.
Units: "px" means map pixels (the mountain map's pixels), angles are degrees,
times are seconds, colours are (R, G, B) for pygame or (B, G, R) for OpenCV.
"""
import os

# Folder this file is in = the project folder. Building paths from it means the
# game finds its files no matter which folder you start Python from.
ROOT = os.path.dirname(os.path.abspath(__file__))
ASSET_DIR = os.path.join(ROOT, "assets")


def asset(name):
    """Full path of an image in the assets folder, e.g. asset("head.png")."""
    return os.path.join(ASSET_DIR, name)


# ------------------------------------------------------------------------ maps --
# Every map is scaled to MAP_HEIGHT px tall (times its "scale"), so character size,
# view and physics feel the same on all maps; scale 1.5 = map 1.5x as big compared to
# the character. spawn = (x, y) as fractions of the map: the character drops from
# there to the floor. The flag goes on the top-right high ground.
MAP_HEIGHT = 2164
MAPS = [
    {"name": "Mountain", "background": "background.png", "foreground": "foreground.png",
     "spawn": (0.06, 0.0)},
    {"name": "Cave", "background": "background-cave.png", "foreground": "foreground-cave.png",
     "spawn": (0.04, 0.84), "scale": 1.5},
    # windmill: tower picture (no collision) + rotor that spins and is solid.
    # hub = rotation centre in the images' pixels, rps = turns per second (clockwise)
    {"name": "Windmill", "background": "map3-background.png", "foreground": "map3-foreground.png",
     "spawn": (0.04, 0.0),
     "windmill": {"base": "map3-windmillbase.png", "blade": "map3-windmillblade.png",
                  "hub": (1555, 700), "rps": 0.25}},
]
SPAWN_LIFT = 1 / 8          # start this far (MAP_HEIGHT fraction) above the floor, or mid-gap if lower

# ------------------------------------------------------------------- map / view --
VIEW_FRACTION = 1 / 3       # game view height = 1/3 of the map height
VIEW_ASPECT = 4096 / 2164   # view width / height (the mountain map's shape)
WINDOW_SCALE = 1.0          # max scale of the window (it also shrinks to fit the screen)
# one window: game screen left, camera right (see display.py), sizes relative to the game view width
LAYOUT_MARGIN = 0.025       # gap between window edge / game / camera
LAYOUT_CAMERA_WIDTH = 0.40  # camera panel width
LAYOUT_SCREEN_FILL = 0.92   # window may use at most this fraction of the desktop
LAYOUT_BG = (40, 42, 48)    # dark grey around the two panels
START_FULLSCREEN = False    # F11 toggles fullscreen at any time
FPS = 60                    # frames drawn per second (the loop waits to keep this rate)
PHYSICS_SUBSTEPS = 3        # physics steps per rendered frame (stability)
CAMERA_LERP = 6.0           # how fast the game view follows the character (1/s)
SKY_COLOR = (135, 206, 235) # fills transparent gaps behind the map images

# ------------------------------------------------------------- character size --
# All character pngs share a 500x500 canvas with the same scale.
# character-demo.png canvas height = CHARACTER_FRACTION * map height.
CHARACTER_FRACTION = 1 / 8
SPRITE_CANVAS = 500         # size of the character png canvases

# Geometry measured on the 500x500 canvas (px)
HEAD_CENTER = (250, 250)
HEAD_RADIUS = 95            # collision circle of the head
LEG_PIVOT = (249.5, 190.5)  # joint at the top of leg-part.png / leg-part-with-shoe.png
SEGMENT_LENGTH = 117        # pivot -> pivot of leg-part.png
FOOT_OFFSET = (20, 160)     # shoe centre from LEG_PIVOT while the sprite hangs down (toe = +x)
FOOT_RADIUS = 30            # collision circle of the shoe

# name: (hip offset from head centre, side (-1 left / +1 right), rest thigh, rest bend)
# Angles in degrees, "outward" convention: 0 = pointing down, 90 = pointing
```

```python
# sideways away from the body, 180 = pointing up. bend = shin angle - thigh angle.
LEGS = {
    "upper_left":  ((-75, -55), -1, 145, 20),
    "upper_right": ((75, -55), +1, 145, 20),
    "lower_left":  ((-85, 50), -1, 50, -50),
    "lower_right": ((85, 50), +1, 50, -50),
}
# Legs whose shoe is drawn mirrored. Upper legs usually point up, which turns a normal
# shoe's toe inward; mirroring makes their toes point outward, like character-demo.png.
MIRRORED_SHOES = {"upper_left", "upper_right"}
# Who drives which leg, per number of players: (player index, body limb, character leg).
# Limbs are named by screen side of the mirrored camera image (= the player's own side).
# Players are numbered left -> right as they stand in front of the camera.
# The character only gets the legs listed in its scheme.
CONTROL_SCHEMES = {
    1: [(0, "left_arm", "lower_left"), (0, "right_arm", "lower_right")],
    2: [(0, "left_arm", "upper_left"), (0, "right_arm", "upper_right"),
        (1, "left_arm", "lower_left"), (1, "right_arm", "lower_right")],
    3: [(0, "left_arm", "upper_left"), (0, "right_arm", "upper_right"),
        (1, "left_arm", "lower_left"), (2, "right_arm", "lower_right")],
    4: [(0, "left_arm", "upper_left"), (1, "right_arm", "upper_right"),
        (2, "left_arm", "lower_left"), (3, "right_arm", "lower_right")],
}
MAX_PLAYERS = 4              # highest player count offered in the menu
# leg length multiplier per player count (only the black sticks stretch, shoes keep their size)
LEG_STRETCH = {1: 1.5}
SHOE_TOP = 300               # canvas y where the shoe starts on leg-part-with-shoe.png

# ------------------------------------------------------------------------ flag --
# flag.png stands on the highest ground in the right-most FLAG_SEARCH part of the map
# that has room above it for the whole flag.
FLAG_SEARCH = 0.1            # fraction of the map width searched for the peak
FLAG_FRACTION = 1 / 9        # flag.png canvas height relative to MAP_HEIGHT
FLAG_BASE = (160, 492)       # canvas px of flag.png that sits on the ground (pole foot)

# ---------------------------------------------------------------------- physics --
GRAVITY = 2000.0             # px/s^2 (map pixels)
MAX_SPEED = 1500.0           # px/s, speed limit while touching; falling speed limit in the air
AIR_DRAG = 0.3               # 1/s
LEG_MAX_SPEED = 420.0        # deg/s, how fast a leg joint can follow the arm
SOLVER_ITERATIONS = 4        # passes over all contacts per physics step (more = stiffer, slower)
WALKABLE_SLOPE = 55.0        # deg; feet only get friction on surfaces flatter than this
ROTOR_WALKABLE_SLOPE = 10.0  # deg; same for the windmill rotor: slippery (sticky feet still glue)
ROTOR_PUSH_SPEED = 400.0     # px/s; a blade can push a free foot/head away at most this much faster than it moves
CONTACT_EPS = 2.0            # px; a foot this close to the ground still counts as touching
STICKY_GRAB_DIST = 8.0       # px; a sticky foot grabs ground within this distance
STICKY_BREAK_DIST = 40.0     # px; a sticky foot tears off if pulled this far from its anchor
STICKY_STIFFNESS = 0.6       # share of a glued foot's error fixed per solver pass (1 = all)
LEG_BLOCK_TOLERANCE = 3.0    # px a free foot may sink into rock while another foot is glued
STICKY_SINK = 6.0            # px a sticky/glued foot may sink into rock while another foot is glued
HEAD_FRICTION = 6.0          # 1/s, damping while the head scrapes the ground

# ---------------------------------------------------------------------- vision --
CAMERA_SOURCE = "0"          # webcam index or video path
POSE_MODEL = "yolo26n-pose.pt"   # YOLO "nano" pose model, downloaded automatically
# Use yolo26n-pose.engine (TensorRT, made for THIS GPU) when it exists and a GPU is present.
# Rebuild it after changing GPU/driver/TensorRT:  python export_engine.py
USE_TENSORRT = True
FACE_MODEL = os.path.join(ROOT, "face_landmarker.task")   # MediaPipe face model file
FACE_URL = ("https://storage.googleapis.com/mediapipe-models/face_landmarker/"
            "face_landmarker/float16/1/face_landmarker.task")
MIRROR_CAMERA = True         # flip the webcam so it behaves like a mirror
KEYPOINT_THR = 0.5           # keypoints with lower confidence (0..1) count as "not seen"

# face <-> pose matching (pixel closeness)
FACE_MATCH_FACTOR = 1.0      # max face-to-head distance = factor * shoulder width
FACE_MATCH_MIN_PX = 40
FACE_CROP_FALLBACK = True    # if the full-frame face search misses a player, retry on a head crop
CROP_SCALE = 2.4             # head crop size = spread of nose/eyes/ears * this
CROP_DOWN = 0.25             # move the crop down (in spreads) so chin and mouth are inside
```

```python
MIN_CROP = 24               # px; smaller heads are skipped (too few pixels for a face mesh)
UPSCALE_TO = 256            # small crops are enlarged to this before the face model

# mouth -> sticky feet
MOUTH_OPEN_T, MOUTH_CLOSE_T = 0.35, 0.25   # hysteresis on lip gap / lip width
MOUTH_EMA = 0.5             # smoothing of the mouth score (1 = no smoothing)
MOUTH_TIMEOUT = 0.5         # s; forget the mouth state if the face is lost for this long

# 1 player: sideways head tilt (roll) picks which foot the open mouth sticks.
# head tilted to the right (right ear lower) -> right-arm leg, tilted left -> left-arm leg.
HEAD_TILT_PLAYERS = {1}     # player counts that use this
HEAD_TILT_ANGLE = 10.0      # deg the ear line must tilt to switch side (in between: keep last)
HEAD_TILT_INVERT = False    # True: swap (tilt right -> left-arm leg)

# arm angles -> legs
ANGLE_EMA = 0.6             # smoothing of arm angles (1 = no smoothing)

# player colours (BGR for OpenCV): red, green, blue, yellow
PLAYER_COLORS_BGR = [(0, 0, 255), (0, 220, 0), (255, 120, 0), (0, 220, 255)]
PLAYER_COLORS_RGB = [tuple(reversed(c)) for c in PLAYER_COLORS_BGR]
BODY_COLOR_BGR = (150, 150, 150)   # grey for non-controlling body parts in the preview

# ------------------------------------------------------------------- fonts --
# Comma-separated lists: pygame uses the first one installed. Windows has Arial /
# Consolas / Segoe UI Emoji; Linux usually has DejaVu, Liberation or FreeFont.
FONT_HEAVY = "arialblack,arial,dejavusans,liberationsans,freesans"     # titles, buttons
FONT_TEXT = "arial,dejavusans,liberationsans,freesans"                 # normal text
FONT_MONO = "consolas,couriernew,dejavusansmono,liberationmono,freemono"  # numbers, timer
FONT_EMOJI = "segoeuiemoji,notocoloremoji,notoemoji,symbola"           # the sheep in the menu

# ------------------------------------------------------------------- menu --
GAME_TITLE = "All For One"
GAME_CREDIT = "From The Group MeEeEe"
GAME_CREDIT_EMOJI = "[sheep emoji]"   # sheep
```

## settings.py (148 lines)

Everything about player settings: which config values the Options screen may change, the safe range of each, and loading/saving `settings.json` (chapter 12).

| Lines | What happens there |
|---|---|
| 1-13 | Docstring: how settings flow into `config`, and why they cannot break the game. |
| 14-21 | Imports, the file location and the minimum gap between the mouth thresholds. |
| 24-34 | `Setting`: one tunable value - config name, label, tab, type, range, step, unit, help. |
| 36-44 | Convert a value to the right type and force it into the safe range (None if impossible). |
| 47-84 | The tabs, and the list of all tunable settings with their ranges and help texts. |
| 85 | Look-up table: config name -> Setting. |
| 88-92 | Remember the config.py values as the defaults. |
| 94-95 | Current value (read from config). |
| 97-109 | Clamp, write into config, keep 'close' below 'open'. |
| 111-114 | Put the defaults back. |
| 116-120 | Only the values that differ from the defaults. |
| 121-136 | Read settings.json safely; anything wrong is skipped. |
| 138-148 | Write the changed values (or delete the file when nothing changed). |

### Full source

```python
"""Player-tunable settings: what can be changed in the Options screen, the safe range
of each value, and saving/loading them to settings.json.

How it works: the game reads all its numbers from `config`. A Setting names one of
those config values; Settings.set() writes the new value straight into `config`
(e.g. config.GRAVITY = 2400), so every part of the game sees it immediately.
settings.json only stores the values the player changed; at start-up they are
put back into `config` before anything else is built.

Only values that cannot break the game are offered, each limited to a safe
range. A missing, broken or hand-edited settings.json never crashes the game:
unknown names and wrong types are ignored and numbers are clamped into range.
"""
import json
import os
from dataclasses import dataclass

import config

SETTINGS_FILE = os.path.join(config.ROOT, "settings.json")
MOUTH_GAP = 0.05     # "close" threshold is kept at least this far below "open"


@dataclass
class Setting:
    key: str             # name of the value in config.py
    label: str           # text shown in the Options screen
    group: str           # tab it appears in
    kind: str            # "float", "int" or "bool"
    lo: float = 0.0      # smallest allowed value (numbers only)
    hi: float = 1.0      # largest allowed value
    step: float = 0.01   # slider step
    unit: str = ""       # shown after the value, e.g. "deg"
    help: str = ""       # one-line explanation shown at the bottom of the screen

    def clamp(self, value):
        """Convert to the right type and force into the safe range (None if impossible)."""
        if self.kind == "bool":
            return value if isinstance(value, bool) else None
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            return None
        value = min(max(float(value), self.lo), self.hi)
        value = round(round(value / self.step) * self.step, 6)       # snap to the slider step
        return int(round(value)) if self.kind == "int" else float(value)


GROUPS = ["Camera", "Mouth", "Head tilt", "Controls", "Physics", "Display"]

SETTINGS = [
    Setting("MIRROR_CAMERA", "Mirror camera", "Camera", "bool",
            help="Flip the camera picture like a mirror (your right hand shows on the right)."),
    Setting("KEYPOINT_THR", "Joint confidence needed", "Camera", "float", 0.3, 0.8, 0.05,
            help="Lower: arms are tracked in worse light but jitter more. "
                 "Higher: steadier, lost more often."),

    Setting("MOUTH_OPEN_T", "Open above", "Mouth", "float", 0.15, 0.6, 0.01,
            help="Mouth ratio (lip gap / mouth width) needed to turn sticky on. Lower = easier."),
    Setting("MOUTH_CLOSE_T", "Close below", "Mouth", "float", 0.05, 0.55, 0.01,
            help="Ratio below which sticky turns off again. Always kept a bit below 'Open above'."),
    Setting("MOUTH_EMA", "Mouth smoothing", "Mouth", "float", 0.2, 1.0, 0.05,
            help="1 = react instantly, lower = smoother but slower to open/close."),
    Setting("MOUTH_TIMEOUT", "Keep state when face lost", "Mouth", "float", 0.0, 2.0, 0.1, "s",
            help="How long the mouth state is kept when the face is not found for a moment."),

    Setting("HEAD_TILT_ANGLE", "Switch angle", "Head tilt", "float", 5, 25, 1, "deg",
            help="1 player: how far you tilt your head sideways to choose the sticky foot."),
    Setting("HEAD_TILT_INVERT", "Swap sides", "Head tilt", "bool",
            help="1 player: tilt right picks the LEFT foot instead of the right one."),

    Setting("LEG_MAX_SPEED", "Leg speed", "Controls", "float", 240, 720, 10, "deg/s",
```

```
            help="How fast the legs follow your arms. Higher = snappier, stronger kicks and jumps."),
    Setting("ANGLE_EMA", "Arm smoothing", "Controls", "float", 0.2, 1.0, 0.05,
            help="1 = legs copy every arm movement, lower = smoother legs but a little delay."),

    Setting("GRAVITY", "Gravity", "Physics", "float", 1200, 3000, 50, "px/s2",
            help="Higher = heavier character: falls faster, jumps lower."),
    Setting("WALKABLE_SLOPE", "Grip on slopes up to", "Physics", "float", 30, 70, 1, "deg",
            help="Feet grip (don't slide) on ground flatter than this, without being sticky."),
    Setting("STICKY_BREAK_DIST", "Sticky foot tears off at", "Physics", "float", 20, 80, 2, "px",
            help="How far a glued foot can be pulled from its spot before it lets go."),

    Setting("START_FULLSCREEN", "Fullscreen", "Display", "bool",
            help="Play in fullscreen (also F11 at any time). Remembered for the next start."),
]
BY_KEY = {s.key: s for s in SETTINGS}


class Settings:
    def __init__(self, path=SETTINGS_FILE):
        self.path = path
        # the values written in config.py = the defaults "Reset to defaults" goes back to
        self.defaults = {s.key: getattr(config, s.key) for s in SETTINGS}

    def get(self, key):
        return getattr(config, key)

    def set(self, key, value):
        """Change one setting (clamped to its safe range) and apply it immediately."""
        setting = BY_KEY.get(key)
        value = setting.clamp(value) if setting else None
        if value is None:
            return
        setattr(config, key, value)
        # keep the mouth thresholds in a valid order (close < open)
        if key == "MOUTH_OPEN_T" and config.MOUTH_CLOSE_T > value - MOUTH_GAP:
            config.MOUTH_CLOSE_T = BY_KEY["MOUTH_CLOSE_T"].clamp(value - MOUTH_GAP)
        if key == "MOUTH_CLOSE_T" and config.MOUTH_OPEN_T < value + MOUTH_GAP:
            config.MOUTH_OPEN_T = BY_KEY["MOUTH_OPEN_T"].clamp(value + MOUTH_GAP)
            config.MOUTH_CLOSE_T = min(config.MOUTH_CLOSE_T, config.MOUTH_OPEN_T - MOUTH_GAP)

    def reset(self):
        """Back to the values written in config.py."""
        for key, value in self.defaults.items():
            setattr(config, key, value)

    def changed(self):
        """Only the settings that differ from the defaults (what gets saved)."""
        return {k: self.get(k) for k in self.defaults if self.get(k) != self.defaults[k]}

    # ------------------------------------------------------------- file --
    def load(self):
        """Apply settings.json if it exists. Anything wrong in it is skipped."""
        try:
            with open(self.path, encoding="utf-8") as f:
                data = json.load(f)
        except FileNotFoundError:
            return
        except (OSError, ValueError) as e:
            print(f"settings.json ignored ({e}); using defaults")
            return
        if not isinstance(data, dict):
            return
        # open threshold first, so the close threshold is checked against it
        for key in sorted(data, key=lambda k: k != "MOUTH_OPEN_T"):
            if key in BY_KEY:
                self.set(key, data[key])

    def save(self):
        """Write the changed settings; with nothing changed the file is removed."""
        changed = self.changed()
        try:
            if changed:
                with open(self.path, "w", encoding="utf-8") as f:
                    json.dump(changed, f, indent=2)
            elif os.path.exists(self.path):
                os.remove(self.path)
        except OSError as e:
            print(f"could not save settings ({e})")
```

## display.py (141 lines)

The single window. Scenes draw on a fixed-size canvas; this class scales it into the window next to the camera picture and handles fullscreen.

| Lines | What happens there |
|---|---|
| 1-19 | Docstring with the layout sketch. |
| 20-23 | Imports. |
| 26-41 | Constructor: layout size in canvas units (game + camera + margins), create the window and the canvas. |
| 42-53 | Create the window: fullscreen at screen size, or windowed at the biggest scale that fits 92% of the desktop. |
| 55-73 | Compute where the game panel, the camera panel (top of the right column, 4:3) and the map panel (the rest of the column) go for a given window size. |
| 75-76 | Switch mode. |
| 78-83 | Keys every scene shares (F11). Returns True if it used the event. |
| 85-90 | Convert a mouse position from window pixels to canvas pixels. |
| 91-106 | Draw the frame: background, scaled canvas, frame, camera, map; then `flip()` shows it. |
| 108-128 | Turn the newest camera frame (numpy) into a pygame image only when it changed, fit it into the panel, or show 'no camera'. |
| 130-141 | Let the game draw the map overview (it passes a function), or show a note. |

### Full source

```python
"""The single app window: game screen on the left, camera preview on the right.

    +-------------------------------------------+
    |  +-------------------------+              |
    |  |                         |  +---------+ |
    |  |       game screen       |  | camera  | |
    |  |                         |  +---------+ |
    |  |                         |  +---------+ |
    |  |                         |  |   map   | |
    |  +-------------------------+  +---------+ |
    +-------------------------------------------+

Scenes draw onto `canvas` (map-view sized); present() scales it into the
window. Mouse positions must go through to_canvas() before hit-testing.
F11 (handled in handle_event) switches between window and fullscreen.

Why a canvas: scenes always draw at the same size (1365 x 721) no matter how
big the window is; only present() deals with the real window size.
"""
import pygame

import config
from ui import draw_text


class Display:
    def __init__(self, canvas_size, input_source):
        self.input = input_source
        self.canvas_size = canvas_size
        cw, ch = canvas_size
        # whole layout at canvas scale (1 unit = 1 canvas px)
        self.margin = config.LAYOUT_MARGIN * cw
        self.full_w = cw + config.LAYOUT_CAMERA_WIDTH * cw + 3 * self.margin   # game + camera + 3 gaps
        self.full_h = ch + 4 * self.margin
        self.fullscreen = False
        self.cam_surface = None        # last camera picture, already scaled for the panel
        self._cam_source = None        # the frame cam_surface was made from
        self._set_mode(config.START_FULLSCREEN)
        self.canvas = pygame.Surface(canvas_size).convert()

    # ------------------------------------------------------------------ window --
    def _set_mode(self, fullscreen):
        """Create (or re-create) the window, then work out where things go in it."""
        self.fullscreen = fullscreen
        if fullscreen:
            self.window = pygame.display.set_mode((0, 0), pygame.FULLSCREEN)   # (0, 0) = screen size
        else:
            # biggest scale that fits LAYOUT_SCREEN_FILL of the desktop (and WINDOW_SCALE)
            desk_w, desk_h = pygame.display.get_desktop_sizes()[0]
            s = min(config.WINDOW_SCALE, desk_w * config.LAYOUT_SCREEN_FILL / self.full_w,
                    desk_h * config.LAYOUT_SCREEN_FILL / self.full_h)
            self.window = pygame.display.set_mode((round(self.full_w * s), round(self.full_h * s)))
        self._layout(*self.window.get_size())

    def _layout(self, w, h):
        """Fit the layout into a w x h window, centred (letterboxed in fullscreen)."""
        cw, ch = self.canvas_size
        s = min(w / self.full_w, h / self.full_h)    # scale that fits both width and height
        self.scale = s
        left = (w - self.full_w * s) / 2             # empty space on each side, if any
        m = self.margin * s
        self.game_rect = pygame.Rect(round(left + m), 0, round(cw * s), round(ch * s))
        self.game_rect.centery = h // 2
        cam_w = round(config.LAYOUT_CAMERA_WIDTH * cw * s)
        self.label_font = pygame.font.SysFont(config.FONT_TEXT, max(14, cam_w // 22), bold=True)
        label = self.label_font.get_linesize() + 6           # room for a "CAMERA" / "MAP" title
        # right column, top-aligned with the game: camera (4:3 like a webcam picture),
        # then the map overview using the rest of the height
        x = self.game_rect.right + round(m)
        self.cam_rect = pygame.Rect(x, self.game_rect.top + label, cam_w, round(cam_w * 3 / 4))
```

```
        map_top = self.cam_rect.bottom + round(m) + label
        self.map_rect = pygame.Rect(x, map_top, cam_w, max(20, self.game_rect.bottom - map_top))
        self._cam_source = None   # rescale the camera image for the new size

    def toggle_fullscreen(self):
        self._set_mode(not self.fullscreen)

    def handle_event(self, event):
        """Window-level keys shared by every scene. True if the event was used."""
        if event.type == pygame.KEYDOWN and event.key == pygame.K_F11:
            self.toggle_fullscreen()
            return True
        return False

    def to_canvas(self, pos):
        """Window pixel -> canvas pixel (for mouse clicks)."""
        # undo what present() did: subtract the game panel's corner, divide by the scale
        return ((pos[0] - self.game_rect.x) / self.scale, (pos[1] - self.game_rect.y) / self.scale)

    # ------------------------------------------------------------------ draw --
    def present(self, minimap=None):
        """Put the finished canvas, the camera picture and the map overview in the
        window and show it. minimap(surface, rect) draws the map (the game passes
        one); without it the map panel shows a short note."""
        self.window.fill(config.LAYOUT_BG)
        if self.game_rect.size == self.canvas.get_size():
            self.window.blit(self.canvas, self.game_rect)          # same size: plain copy (fast)
        else:
            self.window.blit(pygame.transform.smoothscale(self.canvas, self.game_rect.size),
                             self.game_rect)
        pygame.draw.rect(self.window, (0, 0, 0), self.game_rect.inflate(6, 6), 3)   # black frame
        self._draw_camera()
        self._draw_map(minimap)
        # Everything above was drawn into a hidden buffer; flip() shows it all at
        # once, so the player never sees a half-drawn frame.
        pygame.display.flip()

    def _draw_camera(self):
        frame, fresh = self.input.get_preview()
        # convert numpy -> pygame image only when a NEW camera frame arrived
        # (the camera runs slower than the game; re-converting each frame is waste)
        if frame is not None and (fresh or self._cam_source is not frame):
            self._cam_source = frame
            h, w = frame.shape[:2]
            img = pygame.image.frombuffer(frame.tobytes(), (w, h), "RGB")
            fit = min(self.cam_rect.w / w, self.cam_rect.h / h)    # fit inside the panel, keep shape
            self.cam_surface = pygame.transform.smoothscale(img, (round(w * fit), round(h * fit)))
        if self.cam_surface is not None and self._cam_source is not None:
            rect = self.cam_surface.get_rect(center=self.cam_rect.center)
            self.window.blit(self.cam_surface, rect)
        else:
            rect = self.cam_rect
            pygame.draw.rect(self.window, (20, 20, 24), rect)
            draw_text(self.window, "no camera (keyboard mode)", self.label_font,
                      (200, 200, 200), rect.center, width=1)
        pygame.draw.rect(self.window, (0, 0, 0), rect.inflate(6, 6), 3)
        draw_text(self.window, "CAMERA", self.label_font, (230, 230, 230),
                  (rect.centerx, rect.top - self.label_font.get_linesize()), width=1)

    def _draw_map(self, minimap):
        """The whole-map overview under the camera (only filled in during a game)."""
        if minimap is not None:
            rect = minimap(self.window, self.map_rect)
        else:
            rect = self.map_rect
            pygame.draw.rect(self.window, (20, 20, 24), rect)
            draw_text(self.window, "the whole map is shown here during the game", self.label_font,
                      (150, 150, 150), rect.center, width=1)
        pygame.draw.rect(self.window, (0, 0, 0), rect.inflate(6, 6), 3)
        draw_text(self.window, "MAP", self.label_font, (230, 230, 230),
                  (rect.centerx, rect.top - self.label_font.get_linesize()), width=1)
```

# minimap.py (50 lines)

The whole map in the panel under the camera (chapter 10).

| Lines | What happens there |
|---|---|
| 1-3 | Docstring and imports. |
| 5-6 | Colours of the character dot and the view frame. |
| 9-13 | Remembers its map and the cached small picture. |
| 15-27 | The map with its flag shrunk to the panel size; redone only when the size changes. |
| 29-50 | Fit the map into the panel keeping its shape, draw the turning rotor, the frame of the visible area and the character dot (kept inside the picture). |

**Full source**

```
"""Map overview for the panel under the camera: the whole map, the flag, the
part of the map the game view shows, and where the character is."""
import pygame

MARKER = (255, 220, 40)        # character dot (yellow, visible on sky and in the dark cave)
VIEW_FRAME = (255, 255, 255)   # rectangle of the area shown in the game view


class MiniMap:
    def __init__(self, game_map):
        self.map = game_map
        self._image = None          # map picture scaled for the panel (made once per panel size)
        self._size = None

    def _picture(self, size):
        """The whole map with its flag, shrunk to `size`. Cached: scaling the big map
        picture is slow, so it is redone only when the panel size changes (F11)."""
        if size != self._size:
            t = self.map.terrain
            k = size[1] / t.height
            img = pygame.transform.smoothscale(t.surface, size)
            flag = self.map.flag
            fw, fh = flag.image.get_size()
            small = pygame.transform.smoothscale(flag.image, (max(1, round(fw * k)), max(1, round(fh * k))))
            img.blit(small, flag.pos * k)
            self._image, self._size = img, size
        return self._image

    def draw(self, surface, rect, char_pos, view_rect):
        """Draw into `rect` (window pixels); returns the rectangle actually used."""
        t = self.map.terrain
        k = min(rect.w / t.width, rect.h / t.height)        # fit the whole map, keep its shape
        img = self._picture((max(1, round(t.width * k)), max(1, round(t.height * k))))
        area = img.get_rect(center=rect.center)
        surface.blit(img, area)
        self.map.draw_movers_small(surface, area.topleft, k)   # windmill rotor, turning

        # the part of the map that the game view shows right now
        view = pygame.Rect(area.x + view_rect.x * k, area.y + view_rect.y * k,
                           max(2, view_rect.w * k), max(2, view_rect.h * k)).clip(area)
        pygame.draw.rect(surface, (0, 0, 0), view.inflate(2, 2), 1)
        pygame.draw.rect(surface, VIEW_FRAME, view, 2)

        # the character: a dot with a dark ring, kept inside the picture even when
        # the character is above the map's top edge
        x = min(max(area.x + char_pos[0] * k, area.left + 6), area.right - 6)
        y = min(max(area.y + char_pos[1] * k, area.top + 6), area.bottom - 6)
        pygame.draw.circle(surface, (0, 0, 0), (x, y), 7)
        pygame.draw.circle(surface, MARKER, (x, y), 5)
        return area
```

# menu.py (166 lines)

The menu screen with its three pages. Returns what the player chose.

| Lines | What happens there |
|---|---|
| 1-7 | Docstring: how the page loop works. |
| 8-13 | Imports. |
| 14-19 | Help text on the player-count buttons. |
| 22-43 | Constructor: backdrop picture, demo character, fonts. |
| 45-72 | Create every button: PLAY, 1P TEST, the four player-count buttons, BACK and one picture button per map, laid out side by side with equal gaps. |
| 74-114 | The menu loop: handle quit/F11, convert mouse positions to canvas pixels, react to keys and clicks depending on the page, then draw. |
| 115-123 | Render the sheep with an emoji font, or None if none is installed (Linux). |
| 125-134 | Draw the credit line, with the sheep if it could be rendered. |
| 136-166 | Draw the current page. |

### Full source

```python
"""Main menu: title + Play / Test, then the player-count picker, then the map picker.

The menu is one loop with a `page` variable ("title" -> "players" -> "maps").
Every frame it handles clicks/keys for the current page and redraws that page.
All positions are fractions of the canvas size (e.g. h * 0.80 = 80% down),
so the layout stays right if the canvas size ever changes.
"""
import pygame

import config
from ui import Button, MapButton, draw_text

# two lines of help text shown on each player-count button
PLAYER_INFO = {
    1: ["two long legs", "left arm + right arm"],
    2: ["P1 arms: top legs", "P2 arms: bottom legs"],
    3: ["P1 arms: top legs", "P2 left arm, P3 right arm"],
    4: ["one arm = one leg", "P1 P2 top, P3 P4 bottom"],
}


class Menu:
    """run() returns ("play", players, map index), ("test", 1), ("options",) or None to quit."""

    def __init__(self, display, maps, input_source):
        self.display = display
        self.canvas = display.canvas
        self.input = input_source
        w, h = self.canvas.get_size()
        # background picture: the whole first map shrunk to the canvas
        self.backdrop = pygame.transform.smoothscale(maps[0].terrain.surface, (w, h))
        demo = pygame.image.load(config.asset("character-demo.png")).convert_alpha()
        size = int(h * 0.42)
        self.demo = pygame.transform.smoothscale(demo, (size, size))

        # SysFont("a,b,...") uses the first installed font of the list (see config FONT_*)
        self.title_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.13), bold=True)
        self.big_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.06), bold=True)
        self.mid_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.04), bold=True)
        self.small_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.024), bold=True)
        self.credit_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.034), bold=True)
        self.sheep = self._render_emoji(config.GAME_CREDIT_EMOJI, int(h * 0.04))
        self.clock = pygame.time.Clock()

        cx = w // 2
        self.play = Button("PLAY", (cx, int(h * 0.80)), (int(w * 0.22), int(h * 0.12)),
                           self.big_font, (220, 60, 60))
        self.options = Button("OPTIONS", (int(w * 0.20), int(h * 0.80)), (int(w * 0.15), int(h * 0.09)),
                              self.mid_font, (190, 120, 40))
        self.test = Button("1P TEST", (int(w * 0.80), int(h * 0.80)), (int(w * 0.15), int(h * 0.09)),
                           self.mid_font, (60, 110, 200))
        bw, bh = int(w * 0.21), int(h * 0.22)
        self.counts = []
        for i, n in enumerate(range(1, config.MAX_PLAYERS + 1)):
            x = int(w * (0.5 + (i - 1.5) * 0.235))      # 4 buttons spread around the centre
            label = f"{n} PLAYER" + ("S" if n > 1 else "")
            self.counts.append((n, Button(label, (x, int(h * 0.55)), (bw, bh), self.big_font,
                                          config.PLAYER_COLORS_RGB[i], PLAYER_INFO[n],
                                          self.small_font)))
        self.back = Button("BACK", (cx, int(h * 0.85)), (int(w * 0.14), int(h * 0.09)),
                           self.big_font, (90, 90, 90))
        self.maps = []
        for game_map in maps:
            self.maps.append(MapButton(game_map.name.upper(), game_map.thumbnail,
                                       (0, int(h * 0.58)), self.mid_font))
        # side by side, centred, with equal gaps (the cards have different widths)
        gap = 40
        x = (w - sum(b.rect.w for b in self.maps) - gap * (len(self.maps) - 1)) // 2
        for b in self.maps:
            b.rect.left = x
```

```
            x += b.rect.w + gap
        self.players = 1

    def run(self):
        page = "title"
        while True:
            self.clock.tick(config.FPS)
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    return None
                if self.display.handle_event(event):
                    continue
                # mouse events have a window position: convert it to canvas pixels,
                # because the buttons were placed on the canvas
                pos = self.display.to_canvas(event.pos) if hasattr(event, "pos") else (-1, -1)
                key = event.key if event.type == pygame.KEYDOWN else None
                if key == pygame.K_ESCAPE:
                    if page == "title":
                        return None
                    page = "players" if page == "maps" else "title"
                if page == "title":
                    if self.play.clicked(event, pos) or key == pygame.K_RETURN:
                        page = "players"
                    elif self.test.clicked(event, pos) or key == pygame.K_t:
                        return "test", 1
                    elif self.options.clicked(event, pos) or key == pygame.K_o:
                        return ("options",)
                elif page == "players":
                    if self.back.clicked(event, pos):
                        page = "title"
                    for n, button in self.counts:
                        if button.clicked(event, pos) or key == pygame.K_0 + n:
                            self.players, page = n, "maps"
                            break
                else:
                    if self.back.clicked(event, pos):
                        page = "players"
                    for i, button in enumerate(self.maps):
                        if button.clicked(event, pos) or key == pygame.K_1 + i:
                            return "play", self.players, i
            self.draw(page)
            self.display.present()

    @staticmethod
    def _render_emoji(char, size):
        """The emoji as a picture, or None if no emoji font is installed (common on
        Linux) - then the credit line is shown without it instead of an empty box."""
        if pygame.font.match_font(config.FONT_EMOJI) is None:
            return None
        try:
            return pygame.font.SysFont(config.FONT_EMOJI, size).render(char, True, (255, 255, 255))
        except (pygame.error, ValueError):   # e.g. a bitmap emoji font that can't be resized
            return None

    def draw_credit(self, cx, cy):
        """'From The Group MeEeEe' + a colour sheep emoji (if an emoji font exists)."""
        text = self.credit_font.render(config.GAME_CREDIT, True, (255, 255, 255))
        sheep = self.sheep
        gap = 8 if sheep else 0
        left = cx - (text.get_width() + gap + (sheep.get_width() if sheep else 0)) // 2
        draw_text(self.canvas, config.GAME_CREDIT, self.credit_font, (255, 255, 255),
                  (left + text.get_width() // 2, cy), width=2)
        if sheep:
            self.canvas.blit(sheep, sheep.get_rect(midleft=(left + text.get_width() + gap, cy)))

    def draw(self, page):
        c = self.canvas
        w, h = c.get_size()
        mouse = self.display.to_canvas(pygame.mouse.get_pos())
        c.blit(self.backdrop, (0, 0))
```

```
        shade = pygame.Surface((w, h), pygame.SRCALPHA)
        shade.fill((0, 0, 0, 90))
        c.blit(shade, (0, 0))
        draw_text(c, config.GAME_TITLE, self.title_font, (255, 215, 60),
                  (w // 2, int(h * 0.14)), width=5)
        self.draw_credit(w // 2, int(h * 0.245))
        if page == "title":
            c.blit(self.demo, self.demo.get_rect(center=(w // 2, int(h * 0.51))))
            self.play.draw(c, mouse)
            self.test.draw(c, mouse)
            self.options.draw(c, mouse)
            draw_text(c, "F11 fullscreen", self.small_font, (230, 230, 230),
                      (int(w * 0.92), int(h * 0.96)), width=2)
        elif page == "players":
            draw_text(c, "How many players?", self.big_font, (255, 255, 255),
                      (w // 2, int(h * 0.32)))
            for _, button in self.counts:
                button.draw(c, mouse)
            self.back.draw(c, mouse)
        else:
            players = f"{self.players} player" + ("s" if self.players > 1 else "")
            draw_text(c, f"Choose a map  ({players})", self.big_font, (255, 255, 255),
                      (w // 2, int(h * 0.32)))
            for button in self.maps:
                button.draw(c, mouse)
            self.back.draw(c, mouse)
```

## ui.py (136 lines)

Reusable drawing helpers for the screens.

| Lines | What happens there |
|---|---|
| 1-2 | Docstring. |
| 5-18 | Text with a black outline: the text is pasted 8 times in black around its position, then once in colour on top. |
| 21-31 | A rounded clickable rectangle. |
| 33-36 | True for a left click inside the button. |
| 38-54 | Hover effect, shadow, body, border, label and optional subtitle lines. |
| 57-77 | A button that shows a map picture with its name; reuses Button's click test. |
| 80-117 | A slider: mouse x -> value snapped to the step; click or drag; draws track, fill and knob. |
| 120-136 | An on/off switch: click to flip; draws a pill with a knob left (off) or right (on). |

### Full source

```python
"""Small UI helpers: clickable buttons, sliders, switches and outlined text."""
import pygame


def draw_text(surface, text, font, color, center, outline=(0, 0, 0), width=3):
    """Text with a dark outline so it reads on top of the busy map art."""
    img = font.render(text, True, color)        # True = smooth (anti-aliased) letters
    rect = img.get_rect(center=center)
    if outline is not None:
        # the outline = the same text in black, pasted 8 times around the real
        # position (left, right, up, down and the diagonals), then the text on top
        shadow = font.render(text, True, outline)
        for dx in range(-width, width + 1, width):
            for dy in range(-width, width + 1, width):
                if dx or dy:
                    surface.blit(shadow, rect.move(dx, dy))
    surface.blit(img, rect)
    return rect


class Button:
    """A rounded, clickable rectangle with a label (and optional lines of small text)."""

    def __init__(self, text, center, size, font, color=(70, 160, 70), subtitle=None, small_font=None):
        self.text = text
        self.rect = pygame.Rect(0, 0, *size)
        self.rect.center = center
        self.font = font
        self.color = pygame.Color(color)
        self.subtitle = subtitle
        self.small_font = small_font

    def clicked(self, event, pos):
        """pos = event.pos already mapped to canvas coordinates."""
        return (event.type == pygame.MOUSEBUTTONDOWN and event.button == 1
                and self.rect.collidepoint(pos))

    def draw(self, surface, mouse=None):
        hover = mouse is not None and self.rect.collidepoint(mouse)   # mouse over the button?
        # hovering: 25% lighter colour and slightly bigger, so it feels "pressable"
        fill = self.color.lerp((255, 255, 255), 0.25) if hover else self.color
        rect = self.rect.inflate(8, 8) if hover else self.rect
        pygame.draw.rect(surface, (0, 0, 0), rect.move(4, 5), border_radius=18)   # drop shadow
        pygame.draw.rect(surface, fill, rect, border_radius=18)                    # body
        pygame.draw.rect(surface, (0, 0, 0), rect, 4, border_radius=18)           # 4 px black border
        if self.subtitle and self.small_font:   # subtitle = list of lines
            draw_text(surface, self.text, self.font, (255, 255, 255),
                      (rect.centerx, rect.centery - rect.h * 0.18))
            line_h = self.small_font.get_linesize()
            for i, line in enumerate(self.subtitle):
                draw_text(surface, line, self.small_font, (255, 255, 255),
                          (rect.centerx, rect.centery + rect.h * 0.15 + i * line_h), width=2)
        else:
            draw_text(surface, self.text, self.font, (255, 255, 255), rect.center)


class MapButton(Button):
    """A button showing a map picture with its name underneath."""

    def __init__(self, name, image, center, font, color=(60, 60, 70)):
        pad, label_h = 14, font.get_linesize() + 6
        size = (image.get_width() + 2 * pad, image.get_height() + label_h + 2 * pad)
        super().__init__(name, center, size, font, color)    # reuse Button's setup and clicked()
        self.image, self.pad = image, pad

    def draw(self, surface, mouse=None):
        hover = mouse is not None and self.rect.collidepoint(mouse)
        fill = self.color.lerp((255, 255, 255), 0.25) if hover else self.color
        rect = self.rect.inflate(10, 10) if hover else self.rect
        pygame.draw.rect(surface, (0, 0, 0), rect.move(4, 5), border_radius=18)
```

```
        pygame.draw.rect(surface, fill, rect, border_radius=18)
        pygame.draw.rect(surface, (0, 0, 0), rect, 4, border_radius=18)
        img_rect = self.image.get_rect(midtop=(rect.centerx, rect.top + self.pad))
        surface.blit(self.image, img_rect)
        pygame.draw.rect(surface, (0, 0, 0), img_rect, 2)
        draw_text(surface, self.text, self.font, (255, 255, 255),
                  (rect.centerx, (img_rect.bottom + rect.bottom) // 2), width=2)


class Slider:
    """Horizontal slider for a number between lo and hi. Click or drag to change it."""

    def __init__(self, rect, lo, hi, step, color=(70, 160, 220)):
        self.rect = pygame.Rect(rect)        # the track
        self.lo, self.hi, self.step = lo, hi, step
        self.color = pygame.Color(color)
        self.dragging = False

    def value_at(self, x):
        """Mouse x position -> value on the track (snapped to the step)."""
        t = min(max((x - self.rect.x) / self.rect.w, 0.0), 1.0)
        v = self.lo + t * (self.hi - self.lo)
        return round(round(v / self.step) * self.step, 6)

    def handle(self, event, pos):
        """Returns the new value while the slider is clicked/dragged, otherwise None.
        pos = mouse position already converted to canvas coordinates."""
        grab = self.rect.inflate(16, 24)     # a bit bigger than the thin track: easier to hit
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1 and grab.collidepoint(pos):
            self.dragging = True
            return self.value_at(pos[0])
        if event.type == pygame.MOUSEMOTION and self.dragging:
            return self.value_at(pos[0])
        if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
            self.dragging = False
        return None

    def draw(self, surface, value):
        t = (value - self.lo) / (self.hi - self.lo) if self.hi > self.lo else 0
        knob_x = self.rect.x + t * self.rect.w
        track = self.rect.inflate(0, -self.rect.h + 8)               # 8 px tall bar, centred
        pygame.draw.rect(surface, (60, 60, 66), track, border_radius=4)
        filled = track.copy()
        filled.w = max(0, int(knob_x - track.x))
        pygame.draw.rect(surface, self.color, filled, border_radius=4)  # part left of the knob
        pygame.draw.circle(surface, (0, 0, 0), (knob_x, self.rect.centery), 11)
        pygame.draw.circle(surface, (255, 255, 255), (knob_x, self.rect.centery), 8)


class Toggle:
    """On/off switch. Click anywhere on it to flip."""

    def __init__(self, rect, color=(70, 180, 90)):
        self.rect = pygame.Rect(rect)
        self.color = pygame.Color(color)

    def clicked(self, event, pos):
        return (event.type == pygame.MOUSEBUTTONDOWN and event.button == 1
                and self.rect.inflate(10, 10).collidepoint(pos))

    def draw(self, surface, on):
        r = self.rect.h // 2
        pygame.draw.rect(surface, self.color if on else (90, 90, 96), self.rect, border_radius=r)
        pygame.draw.rect(surface, (0, 0, 0), self.rect, 2, border_radius=r)
        knob = (self.rect.right - r if on else self.rect.x + r, self.rect.centery)
        pygame.draw.circle(surface, (255, 255, 255), knob, r - 4)
```

## game.py (305 lines)

The level itself: the game loop, players -> legs, physics substeps, camera, timer, flag, HUD and the finish screen.

| Lines | What happens there |
|---|---|
| 1-10 | Docstring listing the six steps of every frame. |
| 11-19 | Imports. |
| 21-24 | Rows of the score board; readable leg names for the HUD. |
| 27-41 | For each (player, limb, leg) row of the scheme: copy the limb's angles to the leg as its target and set the leg's sticky flag from the mouth (and head tilt). |
| 44-75 | Constructor: fonts, clock, sprite scale (character = 1/8 of the map height), score board and name-entry state. |
| 77-78 | Start position of the current map. |
| 80-86 | Create a character with only the controlled legs; sprites are cached per leg length. |
| 88-98 | Back to the start: rotor, character (at a spot where no foot is in the rock), view, timer, name entry. |
| 99-145 | Set up the map, view, input and character, then the frame loop: events (typing the name first), players, physics substeps (the rotor turns too), camera, timer, flag check, drawing. |
| 147-153 | Flag touched: stop the timer, compare with the saved best, start the name entry. |
| 155-174 | Typing: characters, Backspace, Enter saves to the score board, Esc skips. |
| 176-179 | Method wrapper around the module function. |
| 180-193 | Draw in painter's order (map, flag, character and knee dots, windmill in front, HUD) and present. |
| 195-197 | Called by the display to draw the map panel. |
| 199-204 | Coloured dot on each knee showing which player drives that leg. |
| 206-221 | Player lines and the info line. |
| 222-225 | Seconds -> mm:ss.cc. |
| 227-246 | Dark overlay with COMPLETED!, time and best time, then name entry or board. |
| 248-258 | Question, text box with a blinking cursor, help line. |
| 260-278 | Top 8 for this map and player count; this run in gold, on the last line if lower. |
| 280-286 | Shorten a long name with '...' so it never runs into the time column. |
| 288-292 | One table cell aligned left or right. |
| 294-299 | Timer in the top-right corner (gold when finished). |
| 301-305 | Small text with a shadow. |

### Full source

```python
"""Game scene: ties terrain, character, camera view and the players' input together.

Every frame of Game.run():
    1. handle keys/window events
    2. read the players (arm angles, mouths) from the input source
    3. turn them into leg targets and sticky flags (apply_players)
    4. run the physics a few small steps (substeps)
    5. move the camera view, update the timer, check the flag
    6. draw everything and show it (display.present)
"""
import pygame

import config
from assets import CharacterSprites
from character import Character
from minimap import MiniMap
from scoreboard import NAME_MAX, ScoreBoard
from ui import draw_text
from viewport import Viewport, view_size

BOARD_ROWS = 8      # rows of the score board on the completion screen

LEG_LABELS = {"upper_left": "top-left", "upper_right": "top-right",
              "lower_left": "bottom-left", "lower_right": "bottom-right"}


def apply_players(character, scheme, players):
    """Players' limb angles -> leg targets, mouths (and head tilt) -> sticky feet."""
    # scheme rows look like (0, "left_arm", "upper_left"): player 0's left arm
    # drives the character's upper-left leg (see config.CONTROL_SCHEMES)
    for player, limb, leg_name in scheme:
        if player >= len(players):      # vision thread not switched over yet
            continue
        st = players[player]
        leg = character.legs[leg_name]
        pose = st.limbs.get(limb)
        if pose is not None:            # arm not seen this frame: the leg keeps its last target
            leg.set_target(pose.thigh, pose.bend)
        # 1 player: head tilt picks a side, only that side's foot gets sticky
        side_ok = st.sticky_side is None or limb.startswith(st.sticky_side)
        leg.sticky = st.mouth_open and side_ok


class Game:
    """run(num_players, game_map) plays until Esc ('menu') or window close ('quit').
    Touching the flag with the head completes the level and stops the timer; the
    team then types a name and the time goes on the score board (scores.json)."""

    def __init__(self, display, input_source, debug=False):
        self.display = display
        self.canvas = display.canvas            # draw here; display.present() shows it
        self.map = self.terrain = self.flag = self.view = None   # set when a game starts
        self.input = input_source
        self.debug = debug

        # sprite scale: the 500 px character canvas becomes 1/8 of the map height
        self.sprite_scale = config.MAP_HEIGHT * config.CHARACTER_FRACTION / config.SPRITE_CANVAS
        self._sprites = {}          # leg stretch -> CharacterSprites
        self._minimaps = {}         # map name -> MiniMap (the overview under the camera)
        self.minimap = None
        self.character = None
        self.font = pygame.font.SysFont(config.FONT_TEXT, 20, bold=True)
        self.timer_font = pygame.font.SysFont(config.FONT_MONO, 40, bold=True)
        self.clock = pygame.time.Clock()        # measures frame time and limits the FPS
        self.scheme = config.CONTROL_SCHEMES[2]
        self.elapsed = 0.0                      # timer, seconds
        self.finished = False
        self.scores = ScoreBoard()  # all finished runs, saved in scores.json
        self.entering = False       # completion screen: typing the team name
        self.name = ""              # the name being typed (kept for the next run)
```

```
        self.saved = None           # the score-board row of this run, once saved
        self.new_best = False
        self.big_font = pygame.font.SysFont(config.FONT_HEAVY, 72, bold=True)
        self.mid_font = pygame.font.SysFont(config.FONT_HEAVY, 34, bold=True)
        self.row_font = pygame.font.SysFont(config.FONT_HEAVY, 28, bold=True)   # score board rows

    def spawn_point(self):
        return pygame.Vector2(self.map.spawn)

    def build_character(self, num_players):
        """Only the legs that someone controls, stretched for small teams."""
        stretch = config.LEG_STRETCH.get(num_players, 1.0)
        if stretch not in self._sprites:        # build each sprite set once, then reuse it
            self._sprites[stretch] = CharacterSprites(self.sprite_scale, stretch)
        legs = [leg for _, _, leg in self.scheme]
        return Character(self._sprites[stretch], self.spawn_point(), legs)

    def restart(self):
        self.map.reset()
        self.character.reset(self.spawn_point())                # rest pose first...
        self.character.reset(self.character.free_spot(self.spawn_point(), self.terrain))   # ...then a clear spot
        self.view.snap(self.character.pos)
        self.elapsed = 0.0
        self.finished = False
        self.entering = False
        self.saved = None

    # ------------------------------------------------------------------ loop --
    def run(self, num_players, game_map):
        self.map, self.terrain, self.flag = game_map, game_map.terrain, game_map.flag
        if game_map.name not in self._minimaps:          # one overview per map, kept for next time
            self._minimaps[game_map.name] = MiniMap(game_map)
        self.minimap = self._minimaps[game_map.name]
        self.view = Viewport(self.terrain.width, self.terrain.height, *view_size())
        self.input.set_num_players(num_players)
        self.scheme = config.CONTROL_SCHEMES[num_players]
        self.character = self.build_character(num_players)
        self.restart()
        self.clock.tick()                       # reset the clock so the first dt is small
        while True:
            # tick() waits so the loop runs at most FPS times per second and returns
            # the ms since the last frame. dt is capped at 1/20 s so a hiccup (e.g.
            # dragging the window) can't make the physics take one giant step.
            dt = min(self.clock.tick(config.FPS) / 1000.0, 1 / 20)
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    return "quit"
                if self.display.handle_event(event):    # F11 etc.
                    continue
                if self.entering and self.name_key(event, num_players):
                    continue                            # a key used for typing the name
                if event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_ESCAPE or (
                            self.finished and event.key == pygame.K_RETURN):
                        return "menu"
                    if event.key == pygame.K_F5:
                        self.restart()
                    elif event.key == pygame.K_F1:
                        self.debug = not self.debug

            players = self.input.get_players()
            self.apply_players(players)
            # several small physics steps per frame: smaller steps = fewer feet
            # passing through thin rock and a more stable solver
            sub = dt / config.PHYSICS_SUBSTEPS
            for _ in range(config.PHYSICS_SUBSTEPS):
                self.map.update(sub)                    # windmill rotor turns
                self.character.update(sub, self.terrain)
            self.view.follow(self.character.pos, dt)
            if not self.finished:
```

```
                self.elapsed += dt
                if self.flag.touches(self.character.pos, self.character.head_radius):
                    self.complete(num_players)

            self.draw(players)

    def complete(self, num_players):
        """Flag reached: stop the timer and ask for the team name."""
        self.finished = True
        best = self.scores.best(self.map.name, num_players)
        self.new_best = best is None or self.elapsed < best
        self.entering, self.saved = True, None
        pygame.key.start_text_input()           # deliver typed letters as TEXTINPUT events

    def name_key(self, event, num_players):
        """Typing the team name. True if the event was used for it.
        Enter saves, Esc skips saving; both then show the score board."""
        if event.type == pygame.TEXTINPUT:      # a typed character (works for any keyboard layout)
            text = "".join(ch for ch in event.text if ch.isprintable())
            self.name = (self.name + text)[:NAME_MAX]
            return True
        if event.type != pygame.KEYDOWN:
            return False
        if event.key in (pygame.K_RETURN, pygame.K_KP_ENTER):
            if self.name.strip():               # no empty names
                self.saved = self.scores.add(self.name, self.map.name, num_players, self.elapsed)
                self.entering = False
        elif event.key == pygame.K_BACKSPACE:
            self.name = self.name[:-1]
        elif event.key == pygame.K_ESCAPE:
            self.entering = False               # don't save, just show the board
        else:
            return event.key not in (pygame.K_F1, pygame.K_F5)   # letters etc. only type
        return True

    def apply_players(self, players):
        apply_players(self.character, self.scheme, players)

    # ------------------------------------------------------------------ drawing --
    def draw(self, players):
        # Painter's order: things drawn later cover earlier ones.
        rect = self.view.rect
        offset = pygame.Vector2(rect.topleft)
        self.canvas.blit(self.terrain.surface, (0, 0), rect)   # copy just the visible part of the map
        self.flag.draw(self.canvas, offset)
        self.character.draw(self.canvas, offset, self.debug)
        self.draw_leg_owners(offset)
        self.map.draw_movers(self.canvas, offset)               # windmill tower + rotor, in front of all that
        self.draw_hud(players)
        self.draw_timer()
        if self.finished:
            self.draw_complete()
        self.display.present(self.draw_minimap)

    def draw_minimap(self, surface, rect):
        """Called by the display: the whole map with the character and the visible area."""
        return self.minimap.draw(surface, rect, self.character.pos, self.view.rect)

    def draw_leg_owners(self, offset):
        """Small dot on each knee in the colour of the player driving that leg."""
        for player, _, leg_name in self.scheme:
            _, knee, _ = self.character.legs[leg_name].joints(self.character.pos - offset)
            pygame.draw.circle(self.canvas, (0, 0, 0), knee, 7)     # black ring
            pygame.draw.circle(self.canvas, config.PLAYER_COLORS_RGB[player], knee, 5)

    def draw_hud(self, players):
        """Status text in the top-left corner."""
        y = 10
        n = len({p for p, _, _ in self.scheme})                    # number of players
        for i in range(n):
```

```
            st = players[i] if i < len(players) else None
            legs = ", ".join(LEG_LABELS[leg] for p, _, leg in self.scheme if p == i)
            state = ("STICKY" if st.mouth_open else "free") if st and st.visible else "not seen"
            self._text(f"P{i + 1} [{legs}]: {state}", (10, y), config.PLAYER_COLORS_RGB[i])
            y += 24
        height = self.terrain.height - self.character.pos.y        # y grows downward
        self._text(f"{self.map.name}   height {height / self.terrain.height * 100:.0f}%   "
                   f"F5 restart  F1 debug  F11 fullscreen  Esc menu   {self.clock.get_fps():.0f} FPS",
                   (10, y), (255, 255, 255))

    @staticmethod
    def format_time(t):
        """83.456 s -> "01:23.46"."""
        minutes, seconds = divmod(t, 60)
        return f"{int(minutes):02d}:{seconds:05.2f}"

    def draw_complete(self):
        """The "COMPLETED!" overlay: time, then the name entry or the score board."""
        w, h = self.canvas.get_size()
        shade = pygame.Surface((w, h), pygame.SRCALPHA)   # see-through dark layer
        shade.fill((0, 0, 0, 150))                        # alpha 150 of 255
        self.canvas.blit(shade, (0, 0))
        draw_text(self.canvas, "COMPLETED!", self.big_font, (255, 215, 60), (w // 2, h * 0.12), width=5)
        n = len({p for p, _, _ in self.scheme})
        if self.new_best:
            note = "NEW BEST!"
        else:
            note = f"best  {self.format_time(self.scores.best(self.map.name, n))}"
        draw_text(self.canvas, f"time  {self.format_time(self.elapsed)}     {note}", self.mid_font,
                  (255, 255, 255), (w // 2, h * 0.25))
        if self.entering:
            self.draw_name_entry(w, h)
        else:
            self.draw_board(w, h, n)
            draw_text(self.canvas, "Enter: menu    F5: play again", self.font,
                      (255, 255, 255), (w // 2, h * 0.94), width=2)

    def draw_name_entry(self, w, h):
        draw_text(self.canvas, "Team name for the score board:", self.mid_font,
                  (255, 255, 255), (w // 2, h * 0.43))
        box = pygame.Rect(0, 0, w * 0.42, h * 0.11)
        box.center = (w // 2, h * 0.56)
        pygame.draw.rect(self.canvas, (25, 25, 32), box, border_radius=12)
        pygame.draw.rect(self.canvas, (255, 215, 60), box, 3, border_radius=12)
        cursor = "|" if pygame.time.get_ticks() // 500 % 2 else " "    # blinks once a second
        draw_text(self.canvas, self.name + cursor, self.mid_font, (255, 255, 255), box.center)
        draw_text(self.canvas, "Enter: save    Esc: don't save", self.font,
                  (255, 255, 255), (w // 2, h * 0.70), width=2)

    def draw_board(self, w, h, n):
        """Fastest runs on this map with this many players; this run highlighted."""
        rows = self.scores.ranking(self.map.name, n)
        title = f"BEST TIMES   {self.map.name}, {n} player{'s' if n > 1 else ''}"
        draw_text(self.canvas, title, self.mid_font, (120, 255, 120), (w // 2, h * 0.37))
        if not rows:
            draw_text(self.canvas, "no times saved yet", self.font, (220, 220, 220), (w // 2, h * 0.50))
            return
        shown = list(enumerate(rows[:BOARD_ROWS], 1))      # (place, row)
        if self.saved is not None and self.saved not in rows[:BOARD_ROWS]:
            # this run is further down: show it in the last line with its real place
            shown[-1] = (rows.index(self.saved) + 1, self.saved)
        for i, (place, row) in enumerate(shown):
            y = h * 0.46 + i * h * 0.058
            color = (255, 215, 60) if row is self.saved else (255, 255, 255)
            self._cell(f"{place}.", w * 0.25, y, color, "right")
            self._cell(self._fit(row["name"], w * 0.37), w * 0.27, y, color, "left")
            self._cell(self.format_time(row["time"]), w * 0.73, y, color, "right")
            self._cell(row.get("date", "")[:10], w * 0.76, y, (180, 180, 180), "left")

    def _fit(self, text, width):
```

```
281         """Shorten text with '...' until it is at most `width` px wide in the row font."""
282         if self.row_font.size(text)[0] <= width:
283             return text
284         while text and self.row_font.size(text + "...")[0] > width:
285             text = text[:-1]
286         return text + "..."
287 
288     def _cell(self, text, x, y, color, align):
289         """One table cell: outlined text whose left or right edge is at x."""
290         width = self.row_font.size(text)[0]
291         cx = x + width / 2 if align == "left" else x - width / 2
292         draw_text(self.canvas, text, self.row_font, color, (cx, y), width=2)
293 
294     def draw_timer(self):
295         text = self.format_time(self.elapsed)
296         color = (255, 215, 60) if self.finished else (255, 255, 255)    # gold when finished
297         img = self.timer_font.render(text, True, (255, 255, 255))      # only to measure the width
298         center = (self.canvas.get_width() - img.get_width() // 2 - 20, 32)   # top-right corner
299         draw_text(self.canvas, text, self.timer_font, color, center)
300 
301     def _text(self, msg, pos, color):
302         """Small text with a 2 px black shadow so it reads on any background."""
303         shadow = self.font.render(msg, True, (0, 0, 0))
304         self.canvas.blit(shadow, (pos[0] + 2, pos[1] + 2))
305         self.canvas.blit(self.font.render(msg, True, color), pos)
```

## scoreboard.py (70 lines)

Finish times with team names, kept in scores.json (chapter 11).

| Lines | What happens there |
|---|---|
| 1-9 | Docstring with the file format. |
| 10-17 | Imports, file path and longest name. |
| 20-23 | Load the file at start. |
| 25-41 | Read the file; keep only well-formed rows; rename a broken file instead of crashing. |
| 43-52 | Write a temporary file, then swap it in. |
| 54-60 | Store a run with the date and save. |
| 62-65 | Runs on one map with one player count, fastest first. |
| 67-70 | The fastest time or None. |

### Full source

```python
"""Score board: finish times with the players' team name, kept in scores.json.

Every finished run that gets a name is stored (nothing is ever thrown away);
rankings are made per map and player count, fastest first. scores.json looks like
    {"scores": [{"name": "MeEeEe", "map": "Cave", "players": 2,
                 "time": 83.46, "date": "2026-10-06 14:03"}, ...]}
A missing or broken file never crashes the game: it is treated as empty, and a
broken file is kept as scores.json.broken instead of being overwritten.
"""
import datetime
import json
import os

import config

SCORES_FILE = os.path.join(config.ROOT, "scores.json")
NAME_MAX = 16               # longest team name (characters)


class ScoreBoard:
    def __init__(self, path=SCORES_FILE):
        self.path = path
        self.scores = self._load()

    def _load(self):
        if not os.path.exists(self.path):
            return []
        try:
            with open(self.path, encoding="utf-8") as f:
                rows = json.load(f)["scores"]
            # keep only rows that have everything with the right type
            return [r for r in rows if isinstance(r, dict)
                    and isinstance(r.get("name"), str) and isinstance(r.get("map"), str)
                    and isinstance(r.get("players"), int) and isinstance(r.get("time"), (int, float))]
        except (OSError, ValueError, KeyError, TypeError) as e:
            print(f"scores.json could not be read ({e}); starting an empty score board")
            try:
                os.replace(self.path, self.path + ".broken")    # keep it for a human to look at
            except OSError:
                pass
            return []

    def _save(self):
        # write to a temporary file first, then swap it in: a crash half-way
        # through writing can never leave a cut-off scores.json behind
        tmp = self.path + ".tmp"
        try:
            with open(tmp, "w", encoding="utf-8") as f:
                json.dump({"scores": self.scores}, f, indent=2, ensure_ascii=False)
            os.replace(tmp, self.path)
        except OSError as e:
            print(f"scores.json could not be saved ({e})")

    def add(self, name, map_name, players, time):
        """Store a finished run and save the file. Returns the new row."""
        row = {"name": name.strip()[:NAME_MAX] or "?", "map": map_name, "players": players,
               "time": round(time, 2), "date": datetime.datetime.now().strftime("%Y-%m-%d %H:%M")}
        self.scores.append(row)
        self._save()
        return row

    def ranking(self, map_name, players):
        """All runs on this map with this many players, fastest first."""
        rows = [r for r in self.scores if r["map"] == map_name and r["players"] == players]
        return sorted(rows, key=lambda r: r["time"])

    def best(self, map_name, players):
        """Fastest time so far, or None."""
        rows = self.ranking(map_name, players)
        return rows[0]["time"] if rows else None
```

## test_scene.py (77 lines)

1-player test screen: same input and leg logic as the game, but no physics, so the player can check the controls.

| Lines | What happens there |
|---|---|
| 1-2 | Docstring. |
| 3-9 | Imports. |
| 11-13 | Settings: one player, bigger character, white background. |
| 16-31 | Constructor: centre position, scheme, its own (bigger) character and fonts. |
| 33-53 | Loop: events, players, `apply_players`, move only the leg angles, draw. |
| 55-77 | White background, character, the mouth state in huge letters, sticky side and head roll, help line. |

### Full source

```python
"""1-player test: the character hangs fixed in front of a white background so a
player can check arm -> leg control, mouth (sticky) and head tilt without physics."""
import pygame

import config
from assets import CharacterSprites
from character import Character
from game import apply_players
from ui import draw_text

PLAYERS = 1
TEST_CHARACTER_SCALE = 0.62   # sprite scale (canvas px per png px), bigger than in game
BG = (255, 255, 255)


class TestScene:
    """run() until Esc ('menu') or window close ('quit')."""

    def __init__(self, display, input_source):
        self.display = display
        self.canvas = display.canvas
        self.input = input_source
        w, h = self.canvas.get_size()
        self.center = pygame.Vector2(w / 2, h * 0.56)
        self.scheme = config.CONTROL_SCHEMES[PLAYERS]
        sprites = CharacterSprites(TEST_CHARACTER_SCALE, config.LEG_STRETCH.get(PLAYERS, 1.0))
        self.character = Character(sprites, self.center, [leg for _, _, leg in self.scheme])
        self.huge_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.12), bold=True)
        self.mid_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.05), bold=True)
        self.small_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.03), bold=True)
        self.clock = pygame.time.Clock()

    def run(self):
        self.input.set_num_players(PLAYERS)
        self.character.reset(self.center)
        self.clock.tick()
        while True:
            dt = min(self.clock.tick(config.FPS) / 1000.0, 1 / 20)
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    return "quit"
                if self.display.handle_event(event):
                    continue
                if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
                    return "menu"
            players = self.input.get_players()
            apply_players(self.character, self.scheme, players)
            # no Character.update() here = no physics: only the leg angles are
            # moved towards the arm angles, the body never falls or moves
            for leg in self.character.legs.values():   # legs move, body stays put
                leg.update_angles(dt)
            self.draw(players[0] if players else None)
            self.display.present()

    def draw(self, st):
        c = self.canvas
        w, h = c.get_size()
        c.fill(BG)
        self.character.draw(c, pygame.Vector2(0, 0))    # offset 0: the position is already a canvas position

        if st is None or not st.visible:
            mouth, color = "NOT SEEN", (150, 150, 150)
        elif st.mouth_open:
            mouth, color = "MOUTH OPEN", (40, 200, 40)
        else:
            mouth, color = "MOUTH CLOSED", (220, 60, 60)
        draw_text(c, mouth, self.huge_font, color, (w // 2, int(h * 0.11)), width=4)

        if st is not None and st.sticky_side is not None:
            side = st.sticky_side.upper()
            draw_text(c, f"sticky foot: {side}", self.mid_font, (255, 255, 255),
                      (w // 2, int(h * 0.22)), width=3)
            draw_text(c, f"head roll {st.head_tilt:+.0f} deg  (switch at "
                         f"+-{config.HEAD_TILT_ANGLE:.0f})", self.small_font, (60, 60, 60),
                      (w // 2, int(h * 0.285)), outline=None)
        draw_text(c, "1 PLAYER TEST   Esc: menu   F11: fullscreen", self.small_font, (90, 90, 90),
                  (int(w * 0.2), int(h * 0.97)), outline=None)
```

## options_menu.py (169 lines)

The Options screen: tabs on the left, the settings of the selected tab as sliders and switches on the right, a help line and a live readout (chapter 12).

| Lines | What happens there |
|---|---|
| 1-12 | Docstring and imports. |
| 14 | Height of one setting row. |
| 17-25 | Value -> text: ON/OFF, or the number with the decimals of its step and its unit. |
| 28-47 | Constructor: dark backdrop and fonts. |
| 49-53 | One tab button per group. |
| 54-65 | One row per setting: label position, value position and its widget. |
| 66-71 | RESET TO DEFAULTS and BACK buttons. |
| 73-74 | The settings of the selected tab. |
| 76-111 | Loop: track 1 player on the Head tilt tab (tilt is only measured then), else up to 4; handle quit/F11/Esc/BACK (all save), Reset, tabs, switches and sliders; draw. |
| 113-118 | Make the window match the Fullscreen setting at once. |
| 119-153 | Backdrop, title, tabs, panel with rows (changed values in gold with *), live line, help line, buttons. |
| 155-169 | Live mouth scores (Mouth tab) or head roll (Head tilt tab) from the camera. |

### Full source

```python
"""Options screen: tune the settings in settings.SETTINGS with sliders and switches.

Changes apply immediately (they are written into `config`) and are saved to
settings.json when the player leaves the screen. While tuning, the Mouth and
Head tilt tabs show the live camera values so the thresholds can be set by
watching them.
"""
import pygame

import config
from settings import GROUPS, SETTINGS
from ui import Button, Slider, Toggle, draw_text

ROW_H = 0.105          # height of one setting row (fraction of the canvas height)


def format_value(setting, value):
    if setting.kind == "bool":
        return "ON" if value else "OFF"
    if setting.kind == "int" or setting.step >= 1:
        text = f"{value:.0f}"
    else:
        decimals = len(f"{setting.step:g}".split(".")[1])     # 0.05 -> 2 decimals
        text = f"{value:.{decimals}f}"
    return f"{text} {setting.unit}".strip()


class OptionsScene:
    """run() until BACK/Esc ('menu') or window close ('quit'); saves on the way out."""

    def __init__(self, display, maps, input_source, settings):
        self.display = display
        self.canvas = display.canvas
        self.input = input_source
        self.settings = settings
        w, h = self.canvas.get_size()
        self.backdrop = pygame.transform.smoothscale(maps[0].terrain.surface, (w, h))
        shade = pygame.Surface((w, h), pygame.SRCALPHA)
        shade.fill((0, 0, 0, 165))
        self.backdrop.blit(shade, (0, 0))

        self.title_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.075), bold=True)
        self.tab_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.032), bold=True)
        self.label_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.034), bold=True)
        self.value_font = pygame.font.SysFont(config.FONT_MONO, int(h * 0.034), bold=True)
        self.help_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.028))
        self.clock = pygame.time.Clock()

        # left column: one tab per group
        self.tabs = []
        for i, group in enumerate(GROUPS):
            self.tabs.append((group, Button(group.upper(), (int(w * 0.13), int(h * (0.22 + i * 0.095))),
                                            (int(w * 0.19), int(h * 0.075)), self.tab_font, (70, 70, 80))))
        # right panel: one row per setting of the selected group
        self.panel = pygame.Rect(int(w * 0.255), int(h * 0.17), int(w * 0.715), int(h * 0.62))
        self.rows = {}                                  # key -> (label pos, value pos, widget)
        for group in GROUPS:
            items = [s for s in SETTINGS if s.group == group]
            for i, s in enumerate(items):
                y = int(self.panel.y + h * (0.075 + i * ROW_H))
                if s.kind == "bool":
                    widget = Toggle((int(w * 0.80), y - 16, 74, 32))
                else:
                    widget = Slider((int(w * 0.66), y - 12, int(w * 0.28), 24), s.lo, s.hi, s.step)
                self.rows[s.key] = ((self.panel.x + 24, y), (int(w * 0.585), y), widget)
        self.reset = Button("RESET TO DEFAULTS", (int(w * 0.36), int(h * 0.92)),
                            (int(w * 0.25), int(h * 0.085)), self.tab_font, (190, 120, 40))
        self.back = Button("BACK", (int(w * 0.84), int(h * 0.92)), (int(w * 0.15), int(h * 0.085)),
                           self.tab_font, (90, 90, 90))
        self.group = GROUPS[0]
```

```
        self.hover_help = ""

    def visible(self):
        return [s for s in SETTINGS if s.group == self.group]

    def run(self):
        self.group = GROUPS[0]
        while True:
            self.clock.tick(config.FPS)
            # head tilt is only measured in 1-player mode; elsewhere watch up to 4 people
            self.input.set_num_players(1 if self.group == "Head tilt" else config.MAX_PLAYERS)
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    self.settings.save()
                    return "quit"
                if self.display.handle_event(event):
                    continue
                pos = self.display.to_canvas(event.pos) if hasattr(event, "pos") else (-1, -1)
                escape = event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE
                if escape or self.back.clicked(event, pos):
                    self.settings.save()
                    return "menu"
                if self.reset.clicked(event, pos):
                    self.settings.reset()
                    self._apply_fullscreen()
                for group, button in self.tabs:
                    if button.clicked(event, pos):
                        self.group = group
                for s in self.visible():
                    widget = self.rows[s.key][2]
                    if s.kind == "bool":
                        if widget.clicked(event, pos):
                            self.settings.set(s.key, not self.settings.get(s.key))
                            if s.key == "START_FULLSCREEN":
                                self._apply_fullscreen()
                    else:
                        value = widget.handle(event, pos)
                        if value is not None:
                            self.settings.set(s.key, value)
            self.draw()
            self.display.present()

    def _apply_fullscreen(self):
        """Make the window match the Fullscreen setting right away."""
        if self.display.fullscreen != config.START_FULLSCREEN:
            self.display.toggle_fullscreen()

    # ------------------------------------------------------------------ draw --
    def draw(self):
        c = self.canvas
        w, h = c.get_size()
        mouse = self.display.to_canvas(pygame.mouse.get_pos())
        c.blit(self.backdrop, (0, 0))
        draw_text(c, "Options", self.title_font, (255, 215, 60), (w // 2, int(h * 0.08)), width=4)

        for group, button in self.tabs:
            button.color = pygame.Color((60, 110, 200) if group == self.group else (70, 70, 80))
            button.draw(c, mouse)

        pygame.draw.rect(c, (25, 27, 32), self.panel, border_radius=16)
        pygame.draw.rect(c, (0, 0, 0), self.panel, 3, border_radius=16)
        help_text = ""
        for s in self.visible():
            (lx, ly), (vx, vy), widget = self.rows[s.key]
            value = self.settings.get(s.key)
            changed = value != self.settings.defaults[s.key]
            label = self.label_font.render(s.label + ("  *" if changed else ""), True,
                                           (255, 225, 120) if changed else (235, 235, 235))
            c.blit(label, label.get_rect(midleft=(lx, ly)))
            draw_text(c, format_value(s, value), self.value_font, (255, 255, 255), (vx, vy), width=1)
```

```
            widget.draw(c, value)
            row = pygame.Rect(self.panel.x, ly - h * ROW_H / 2, self.panel.w, h * ROW_H)
            if row.collidepoint(mouse) or getattr(widget, "dragging", False):
                help_text = s.help
        live = self._live_text()
        if live:
            draw_text(c, live, self.label_font, (120, 230, 140),
                      (self.panel.centerx, self.panel.bottom - int(h * 0.05)), width=1)
        default_help = "* = changed from the default.   Changes apply at once and are saved when you leave."
        draw_text(c, help_text or default_help, self.help_font, (210, 210, 210),
                  (w // 2, int(h * 0.835)), width=1)
        self.reset.draw(c, mouse)
        self.back.draw(c, mouse)

    def _live_text(self):
        """Live camera values for the tabs where they help tuning."""
        players = self.input.get_players()
        if self.group == "Mouth":
            parts = []
            for i, st in enumerate(players):
                if st.visible and st.face_found:
                    parts.append(f"P{i + 1} {st.mouth_score:.2f} {'OPEN' if st.mouth_open else 'closed'}")
            return "Live:  " + ("    ".join(parts) if parts else "no face seen")
        if self.group == "Head tilt":
            st = players[0] if players else None
            if st is None or st.sticky_side is None:
                return "Live head tilt is measured in 1-player mode (try it in 1P TEST)."
            return f"Live:  roll {st.head_tilt:+.0f} deg  ->  sticky foot {st.sticky_side.upper()}"
        return ""
```

## maps.py (84 lines)

Turns an entry of `config.MAPS` into a playable map.

| Lines | What happens there |
|---|---|
| 1-7 | Docstring and imports. |
| 9 | Height of the menu picture. |
| 12-26 | Load both pictures and scale them to the common height (times the map's own scale). |
| 27 | Build collision. |
| 28-34 | Windmill maps: paint the tower into the map picture and add the rotor as a moving solid part; then place the flag and find the start point. |
| 36-48 | Make the menu thumbnail with the flag (and rotor) pasted on it. |
| 49-63 | Moving parts: reset, turn, draw big and small. Maps without them do nothing. |
| 65-80 | From the configured point: go up out of rock if needed, then find floor and ceiling and start above the floor (or mid-gap in a low tunnel). |
| 83-84 | Build every map in the list. |

### Full source

```python
"""Playable maps: terrain + flag + start point, built from the entries in config.MAPS."""
import pygame

import config
from flag import Flag
from terrain import Terrain
from windmill import Windmill

THUMB_HEIGHT = 230   # menu picture height (canvas px)


class GameMap:
    def __init__(self, spec):
        # spec = one dictionary from config.MAPS (name, image files, spawn, scale)
        self.name = spec["name"]
        bg = pygame.image.load(config.asset(spec["background"])).convert_alpha()
        fg = pygame.image.load(config.asset(spec["foreground"])).convert_alpha()
        # Resize so the map is MAP_HEIGHT tall (times its own "scale"). The
        # character, flag and view are sized from MAP_HEIGHT, so they look the
        # same on every map; "scale" makes a map bigger compared to them.
        height = round(config.MAP_HEIGHT * spec.get("scale", 1.0))   # MAP_HEIGHT x map scale
        scale = height / fg.get_height()
        if abs(scale - 1) > 1e-3:                                     # skip if already right
            size = (round(fg.get_width() * scale), height)            # keep the shape
            bg = pygame.transform.smoothscale(bg, size)
            fg = pygame.transform.smoothscale(fg, size)
        self.terrain = Terrain(bg, fg)
        self.windmill = None
        if "windmill" in spec:
            self.windmill = Windmill(spec["windmill"], scale)
            self.terrain.surface.blit(self.windmill.base, self.windmill.base_pos)   # tower: just a picture
            self.terrain.movers.append(self.windmill)               # rotor: solid
        self.flag = Flag(self.terrain)
        self.spawn = self._find_spawn(*spec["spawn"])

        # Small picture for the map-selection menu: the whole map shrunk to
        # THUMB_HEIGHT tall, with a shrunk flag pasted at the same relative spot.
        t = self.terrain.surface
        thumb = (round(t.get_width() * THUMB_HEIGHT / t.get_height()), THUMB_HEIGHT)
        self.thumbnail = pygame.transform.smoothscale(t, thumb)
        self.thumbnail.blit(pygame.transform.smoothscale(
            self.flag.image, [max(1, round(v * THUMB_HEIGHT / t.get_height()))
                              for v in self.flag.image.get_size()]),
            self.flag.pos * THUMB_HEIGHT / t.get_height())
        if self.windmill:
            self.windmill.draw_small(self.thumbnail, (0, 0), THUMB_HEIGHT / t.get_height())

    # --- moving parts (windmill rotor); maps without them do nothing here ---
    def reset(self):
        if self.windmill:
            self.windmill.reset()

    def update(self, dt):
        if self.windmill:
            self.windmill.update(dt)

    def draw_movers(self, target, offset):
        if self.windmill:
            self.windmill.draw(target, offset)

    def draw_movers_small(self, target, corner, k):
        if self.windmill:
            self.windmill.draw_small(target, corner, k)

    def _find_spawn(self, fx, fy):
        """Drop from (fx, fy) (map fractions) to the floor below; start a little above it,
        but not higher than the middle of the gap (so it fits in tunnels)."""
        t = self.terrain
        x, y = fx * t.width, fy * t.height
        while y > 0 and t.distance(x, y) <= 0:            # start point in rock: go up into air
```

```
            y -= 2
        floor = y
        while floor < t.height - 1 and t.distance(x, floor) > 0:    # walk down to the floor
            floor += 2
        ceiling = y
        while ceiling > 0 and t.distance(x, ceiling) > 0:           # walk up to the ceiling (or top)
            ceiling -= 2
        lift = config.MAP_HEIGHT * config.SPAWN_LIFT
        # normally start `lift` above the floor; in a low tunnel start at its middle
        return pygame.Vector2(x, max(floor - lift, (floor + ceiling) / 2))


def load_maps():
    return [GameMap(spec) for spec in config.MAPS]
```

## windmill.py (157 lines)

The windmill: a tower picture and a solid, spinning rotor (chapter 7).

| Lines | What happens there |
|---|---|
| 1-14 | Docstring: how the rotor collision works and how feet ride along. |
| 15-23 | Imports. |
| 24-25 | Exact-distance margin around the swept circle; radius of the hub piece. |
| 28-44 | Speed, slipperiness, hub position (scaled to the map), angle; crop and scale the tower picture. |
| 46-52 | Crop the rotor picture to its content and scale it. |
| 54-70 | How far the rotor reaches; distance table of the still rotor covering the whole swept circle plus a 150 px margin, so every point that could touch it is exact. |
| 72-97 | Cut the rotor into three blades and the hub for fast drawing. |
| 98-100 | Back to angle 0. |
| 102-106 | Turn by 360 x rps x dt; remember this turn and the cosine/sine for lookups. |
| 108-111 | Speed of the rotor surface at a point (for limiting pushes). |
| 113-117 | Turn a point stuck on the rotor by the last turn. |
| 118-134 | Beyond the margin: distance to the swept circle (at least 150 px, nothing can touch). Near: turn the point back and look it up in the table. |
| 135-145 | Draw the tower, then rotate and draw the rotor pieces inside the view (after the character, so the windmill is in front of it). |
| 147-157 | The rotor for the map panel and the menu picture. |

### Full source

```python
"""Windmill: a decorative tower (no collision) and a spinning rotor that is solid.

The rotor's collision works like the terrain's (a signed distance field, see
terrain.py), but the table is made once for the rotor standing still. To ask
"how far is point p from the rotor right now?", p is turned back by the
rotor's current angle around the hub and looked up in that still table.

Feet glued to the rotor (or gripping it by friction) ride along: carry() turns
their glue point with the rotor every physics step, so the rotor can lift the
character and fling it when the foot lets go.

Angles: degrees, clockwise on screen (pygame.Vector2.rotate turns that way
because y points down).
"""
import math

import numpy as np
import pygame

import config
from terrain import Terrain

V = pygame.Vector2
MARGIN = 150        # px of exact distances outside the circle the blade tips sweep
HUB_RADIUS = 70     # px around the hub drawn as one piece (the ball and the blade roots)


class Windmill:
    def __init__(self, spec, scale):
        """spec = the "windmill" dictionary of a map in config.MAPS (image names,
        hub position in image px, turns per second); scale = image px -> map px."""
        self.rps = spec["rps"]
        self.walkable_slope = config.ROTOR_WALKABLE_SLOPE   # non-sticky feet slip on steeper blade parts
        self.hub = V(spec["hub"]) * scale                       # rotation centre, map px
        self.angle = 0.0                                        # current rotation (deg)
        self.turn = 0.0                                         # rotation of the last update (deg)
        self._cos, self._sin = 1.0, 0.0                         # of -angle, for distance()

        # tower: crop the big canvas to the tower, scale it, remember where it goes on the map
        base = pygame.image.load(config.asset(spec["base"])).convert_alpha()
        crop = base.get_bounding_rect(min_alpha=1).inflate(4, 4).clip(base.get_rect())
        self.base = pygame.transform.smoothscale(
            base.subsurface(crop), (round(crop.w * scale), round(crop.h * scale)))
        self.base_pos = V(crop.topleft) * scale

        # rotor: crop the big canvas to the rotor, then scale to map size
        img = pygame.image.load(config.asset(spec["blade"])).convert_alpha()
        crop = img.get_bounding_rect(min_alpha=1).inflate(4, 4).clip(img.get_rect())
        size = (round(crop.w * scale), round(crop.h * scale))
        rotor = pygame.transform.smoothscale(img.subsurface(crop), size)
        self.corner = V(crop.topleft) * scale                   # rotor image corner on the map (angle 0)
        self.rotor = rotor

        # collision table of the still rotor: a square around the hub that covers the
        # whole circle the blade tips sweep plus MARGIN, so every point that could
        # touch the rotor at any angle gets an exact distance (MARGIN is bigger than
        # any collision circle: head, feet, contact and grab distances)
        alpha = pygame.surfarray.array_alpha(rotor).T > 127     # (h, w), True = solid
        ys, xs = np.nonzero(alpha)                              # how far the rotor reaches from the hub
        self.reach = float(np.hypot(xs + self.corner.x - self.hub.x,
                                    ys + self.corner.y - self.hub.y).max())
        half = math.ceil(self.reach + MARGIN) + 2
        self.origin = V(round(self.hub.x) - half, round(self.hub.y) - half)   # map position of sdf[0, 0]
        solid = np.zeros((2 * half + 1, 2 * half + 1), bool)
        ox, oy = round(self.corner.x - self.origin.x), round(self.corner.y - self.origin.y)
        solid[oy:oy + alpha.shape[0], ox:ox + alpha.shape[1]] = alpha
        self.sdf = Terrain._signed_distance(solid)

        self.pieces = self._split(rotor)
        self._small = None                                      # (k, rotor image) for the overview map
```

```
71
72      def _split(self, rotor):
73          """Cut the rotor into one piece per blade (120 deg slices around the hub) and
74          the hub ball.
75          Turning three thin pieces each frame is much faster than turning the one
76          big square picture, and pieces outside the view are skipped."""
77          w, h = rotor.get_size()
78          xs, ys = np.meshgrid(np.arange(w), np.arange(h))        # pixel coordinates
79          hub = self.hub - self.corner                           # hub inside the rotor image
80          ang = np.degrees(np.arctan2(ys - hub.y, xs - hub.x))    # direction of each pixel from the hub
81          near = np.hypot(xs - hub.x, ys - hub.y) <= HUB_RADIUS   # the hub ball: one piece, no seams
82          pieces = []
83          for k in range(4):
84              if k == 3:
85                  inside = near
86              else:
87                  centre = -90 + 120 * k                         # up, lower right, lower left
88                  inside = (np.abs((ang - centre + 180) % 360 - 180) <= 60) & ~near
89              part = rotor.copy()
90              pygame.surfarray.pixels_alpha(part)[...] *= inside.T   # clear the other two slices
91              rect = part.get_bounding_rect(min_alpha=1)
92              piece = part.subsurface(rect).copy()
93              offset = V(rect.center) - hub                      # piece centre relative to the hub
94              pieces.append((piece, offset, math.hypot(*rect.size) / 2))
95          return pieces
96
97      # ------------------------------------------------------------- motion --
98      def reset(self):
99          self.angle, self.turn = 0.0, 0.0
100         self._cos, self._sin = 1.0, 0.0
101
102     def update(self, dt):
103         self.turn = 360.0 * self.rps * dt
104         self.angle = (self.angle + self.turn) % 360.0
105         a = math.radians(-self.angle)
106         self._cos, self._sin = math.cos(a), math.sin(a)
107
108     def velocity_at(self, p):
109         """How fast the rotor surface at point p moves (px/s)."""
110         r = V(p) - self.hub
111         return V(-r.y, r.x) * math.radians(360.0 * self.rps)   # clockwise on screen
112
113     def carry(self, p):
114         """Where a point stuck on the rotor is after the last update."""
115         return self.hub + (V(p) - self.hub).rotate(self.turn)
116
117     # ---------------------------------------------------------- collision --
118     def distance(self, x, y):
119         """Signed distance from (x, y) to the rotor at its current angle."""
120         dx, dy = x - self.hub.x, y - self.hub.y
121         r = math.hypot(dx, dy)
122         if r > self.reach + MARGIN:
123             return r - self.reach         # far away: at least MARGIN, nothing can touch here
124         # turn the point back by the rotor angle -> position in the still table
125         lx = self.hub.x + dx * self._cos - dy * self._sin - self.origin.x
126         ly = self.hub.y + dx * self._sin + dy * self._cos - self.origin.y
127         x0, y0 = int(lx), int(ly)         # bilinear sample, as in Terrain.distance
128         fx, fy = lx - x0, ly - y0
129         s = self.sdf
130         top = s[y0, x0] * (1 - fx) + s[y0, x0 + 1] * fx
131         bot = s[y0 + 1, x0] * (1 - fx) + s[y0 + 1, x0 + 1] * fx
132         return float(top * (1 - fy) + bot * fy)
133
134     # ------------------------------------------------------------- drawing --
135     def draw(self, target, offset):
136         """Draw the tower, then the rotor at its current angle; offset = top-left of the
137         view. Called after the character, so the windmill is in front of it."""
138         view = target.get_rect()
139         target.blit(self.base, self.base_pos - offset)
140         for piece, centre, radius in self.pieces:
```

```
141             pos = self.hub + centre.rotate(self.angle) - offset   # piece centre on screen
142             if not view.colliderect((pos.x - radius, pos.y - radius, 2 * radius, 2 * radius)):
143                 continue                                          # off screen: skip the slow part
144             img = pygame.transform.rotate(piece, -self.angle)     # pygame turns counter-clockwise
145             target.blit(img, img.get_rect(center=(round(pos.x), round(pos.y))))
146 
147     def draw_small(self, target, corner, k):
148         """Rotor for the overview map: whole map shrunk by k, map (0, 0) at `corner`."""
149         if self._small is None or self._small[0] != k:
150             size = (max(1, round(self.rotor.get_width() * k)), max(1, round(self.rotor.get_height() * k)))
151             small = pygame.transform.smoothscale(self.rotor, size)
152             self._small = (k, small, (self.hub - self.corner) * k)
153         _, small, hub = self._small
154         img = pygame.transform.rotate(small, -self.angle)
155         centre = V(small.get_size()) / 2 - hub                    # image centre relative to the hub
156         pos = V(corner) + self.hub * k + centre.rotate(self.angle)
157         target.blit(img, img.get_rect(center=(round(pos.x), round(pos.y))))
```

## terrain.py (116 lines)

Map picture and collision via a signed distance field (see chapter 7).

| Lines | What happens there |
|---|---|
| 1-10 | Docstring explaining the signed distance field. |
| 11-17 | Imports. |
| 20-33 | Constructor: compose the map picture (sky, background, foreground). |
| 35-43 | Build the solid mask from the foreground alpha, add invisible side walls, compute the distance field. |
| 44-46 | Moving solid parts (the windmill rotor) are added here by the map. |
| 47-58 | Two distance transforms (air -> rock, rock -> air) subtracted. |
| 59-64 | Distance to the nearest solid thing: rock or a moving part. |
| 66-74 | Which moving part a point touches, if any (for feet that ride the rotor). |
| 76-95 | Smooth lookup of the rock distance with special rules outside the map. |
| 97-105 | Direction out of the rock from the distance gradient. |
| 107-116 | First rock below the open sky in a column (used for the flag). |

**Full source**

```python
"""Map image + collision. The opaque pixels of foreground.png are solid ground.

Collision uses a "signed distance field" (SDF): a big table with one number per
map pixel, saying how far that pixel is from the nearest rock edge:
    > 0  in the air   (e.g. +20 = 20 px away from the nearest rock)
    < 0  inside rock  (e.g. -5  = 5 px deep inside)
    = 0  exactly on the surface
With it, "is this foot inside the ground, how deep, and which way is out?"
is just a table lookup, no matter how wiggly the rock outline is.
"""
import math

import cv2
import numpy as np
import pygame

import config


class Terrain:
    """Holds the rendered map and a signed distance field for collisions.

    distance(p) > 0 in the air, < 0 inside rock; normal(p) points out of the rock.
    """

    def __init__(self, background, foreground):
        self.width, self.height = foreground.get_size()
        # One flat picture of the whole map (sky colour, then background, then
        # foreground on top). The game just copies a rectangle of it each frame.
        self.surface = pygame.Surface((self.width, self.height)).convert()
        self.surface.fill(config.SKY_COLOR)
        self.surface.blit(background, (0, 0))
        self.surface.blit(foreground, (0, 0))

        # Alpha channel of the foreground = how opaque each pixel is (0..255).
        # array_alpha gives it as [x][y]; .T flips it to [y][x] (row, column),
        # which is how numpy/OpenCV index images. More than half opaque = rock.
        solid = pygame.surfarray.array_alpha(foreground).T > 127   # (h, w)
        solid[:, :2] = True      # invisible walls on the map's left/right edges
        solid[:, -2:] = True
        self.sdf = self._signed_distance(solid)
        # moving solid parts (e.g. a windmill rotor). Each has distance(x, y);
        # distance() below reports the nearest of the rock and all of them.
        self.movers = []

    @staticmethod
    def _signed_distance(solid):
        # cv2.distanceTransform: for every non-zero pixel, distance to the nearest
        # zero pixel. Run it twice: once on the air (distance to rock) and once on
        # the rock (distance to air); air distance minus rock distance gives one
        # signed number per pixel.
        air = (~solid).astype(np.uint8)
        rock = solid.astype(np.uint8)
        out = cv2.distanceTransform(air, cv2.DIST_L2, 5)   # air px -> distance to rock
        inside = cv2.distanceTransform(rock, cv2.DIST_L2, 5)
        return (out - inside).astype(np.float32)

    # ------------------------------------------------------------ queries --
    def distance(self, x, y):
        """Signed distance to the nearest solid thing: rock or a moving part."""
        d = self.rock_distance(x, y)
        for m in self.movers:
            d = min(d, m.distance(x, y))
        return d

    def mover_at(self, x, y):
        """The moving part a point touches (closer to it than to the rock), or None.
        Used to make a foot that grabs a rotor ride along with it."""
        best, d = None, self.rock_distance(x, y)
        for m in self.movers:
```

```
            dm = m.distance(x, y)
            if dm < d:
                best, d = m, dm
        return best

    def rock_distance(self, x, y):
        """Bilinear sample of the SDF. Above the map the top row continues upward:
        where it is sky, open sky; where it is rock (side walls, a solid top
        border like the cave's), rock that gets deeper going up, so anything
        poking through is pushed back down into the map instead of escaping."""
        outside = max(0.0, -x, x - (self.width - 1.001))   # past the side walls = deeper in rock
        above = max(0.0, -y)                               # how far above the map's top edge
        # clamp the point into the table so we never index outside it
        x = min(max(x, 0.0), self.width - 1.001)
        y = min(max(y, 0.0), self.height - 1.001)
        # Bilinear interpolation: the point usually lies between 4 table cells;
        # blend them by how close the point is to each (fx, fy = 0..1 fractions).
        # This makes the distance change smoothly instead of in 1 px steps.
        x0, y0 = int(x), int(y)
        fx, fy = x - x0, y - y0
        s = self.sdf
        top = s[y0, x0] * (1 - fx) + s[y0, x0 + 1] * fx
        bot = s[y0 + 1, x0] * (1 - fx) + s[y0 + 1, x0 + 1] * fx
        d = float(top * (1 - fy) + bot * fy) - outside
        return d - above if d < 0 else d    # rock continues (deeper) above the map; sky stays sky

    def normal(self, x, y, h=2.0):
        """Unit vector pointing out of the rock = direction the distance grows fastest.
        Measured by sampling the distance h px left/right and up/down (the gradient)."""
        dx = self.distance(x + h, y) - self.distance(x - h, y)
        dy = self.distance(x, y + h) - self.distance(x, y - h)
        n = math.hypot(dx, dy)                  # length of (dx, dy)
        if n < 1e-6:                            # flat spot: no direction, guess "up"
            return pygame.Vector2(0, -1)
        return pygame.Vector2(dx / n, dy / n)   # divide by length -> length 1

    def ground_y(self, x):
        """Top of the ground under the open sky in column x: the first rock below the
        first air from the top (skips a solid border along the map's top edge).
        0 if the column is rock from the top down (no sky)."""
        col = self.sdf[:, int(min(max(x, 0), self.width - 1))]   # one column of the table
        air = np.nonzero(col > 0)[0]                               # row numbers that are air
        if not len(air) or air[0] > self.height * 0.02:           # no sky near the top
            return 0
        rock = np.nonzero(col[air[0]:] < 0)[0]                     # rock rows below that air
        return int(air[0] + rock[0]) if len(rock) else self.height
```

# flag.py (47 lines)

The goal flag.

| Lines | What happens there |
|---|---|
| 1-4 | Docstring and imports. |
| 7-17 | Load and size the flag; build its pixel mask. |
| 19-32 | Find the right-most ground with room for the flag and take the highest point near it; place the pole foot there. |
| 34-43 | Pixel-perfect test: draw the head as a circle mask and check overlap with the flag mask. |
| 45-47 | Draw at its screen position. |

**Full source**

```python
"""Goal flag: stands on the top-right peak; touching it with the head wins."""
import pygame

import config


class Flag:
    def __init__(self, terrain):
        img = pygame.image.load(config.asset("flag.png")).convert_alpha()
        # Size: FLAG_FRACTION of the reference map height, so the flag is the same
        # size on every map no matter how big that map is.
        scale = config.MAP_HEIGHT * config.FLAG_FRACTION / img.get_height()   # same size on every map
        self.image = pygame.transform.smoothscale(
            img, (round(img.get_width() * scale), round(img.get_height() * scale)))
        # A mask is a yes/no grid of which pixels are visible; used for exact
        # pixel-perfect touch detection (not just the rectangle around the flag).
        self.mask = pygame.mask.from_surface(self.image)

        # stand on the highest ground near the right edge, fully inside the map:
        # find the right-most column with room above it for the whole flag, then
        # take the highest such ground within FLAG_SEARCH of the map width from there
        base_x, base_y = config.FLAG_BASE[0] * scale, config.FLAG_BASE[1] * scale  # pole foot in the image
        x_max = int(terrain.width - (self.image.get_width() - base_x))   # cloth must not leave the map
        # columns (right to left) whose ground is low enough that the flag fits above it
        fits = [x for x in range(x_max, int(base_x), -1) if terrain.ground_y(x) >= base_y]
        if not fits:
            fits = [x_max]
        window = [x for x in fits if x >= fits[0] - terrain.width * config.FLAG_SEARCH]
        peak_x = min(window, key=terrain.ground_y)     # smallest y = highest ground on screen
        # place the image so its pole foot (base_x, base_y) lands on the ground at peak_x
        self.pos = pygame.Vector2(peak_x - base_x, terrain.ground_y(peak_x) - base_y)
        self.rect = self.image.get_rect(topleft=self.pos)

    def touches(self, center, radius):
        """True if a circle (the head) overlaps the flag's visible pixels."""
        r = int(radius)
        # draw the head as a filled circle on a small transparent image ...
        circle = pygame.Surface((2 * r + 1, 2 * r + 1), pygame.SRCALPHA)
        pygame.draw.circle(circle, (255, 255, 255), (r, r), r)
        # ... and ask pygame if its mask overlaps the flag mask at that position
        # (offset = where the circle image's corner sits relative to the flag's corner)
        offset = (int(center[0] - r - self.rect.x), int(center[1] - r - self.rect.y))
        return self.mask.overlap(pygame.mask.from_surface(circle), offset) is not None

    def draw(self, target, offset):
        # offset = top-left of the visible view; map position - offset = screen position
        target.blit(self.image, self.pos - offset)
```

# viewport.py (44 lines)

The game camera (the visible part of the map).

| Lines | What happens there |
|---|---|
| 1-9 | Docstring. |
| 12-15 | View size: 1/3 of the reference height, mountain map's shape. |
| 18-22 | Holds the map size, view size and top-left position. |
| 24-28 | Keep the view inside the map. |
| 30-32 | Jump to a target. |
| 34-41 | Glide towards a target (fast when far, slow when close). |
| 42-44 | Visible area as a rectangle. |

**Full source**

```python
"""Game camera: a window onto the map that smoothly follows the character.

The map is much bigger than the screen; the viewport is the rectangle of the
map that is currently shown. Its top-left corner `pos` is what everything is
drawn relative to (screen position = map position - pos).
"""
import pygame

import config


def view_size():
    """Size of the game view in map pixels (the same for every map, see config.MAP_HEIGHT)."""
    h = round(config.MAP_HEIGHT * config.VIEW_FRACTION)   # 2164 / 3 = 721
    return round(h * config.VIEW_ASPECT), h               # 721 * (4096 / 2164) = 1365


class Viewport:
    def __init__(self, map_w, map_h, w, h):
        self.map_w, self.map_h = map_w, map_h
        self.w, self.h = w, h
        self.pos = pygame.Vector2()   # top-left corner in map pixels

    def _clamped(self, p):
        """Keep the view inside the map (centred if the map is smaller than the view)."""
        def clamp(v, size, view):
            return (size - view) / 2 if size <= view else min(max(v, 0), size - view)
        return pygame.Vector2(clamp(p.x, self.map_w, self.w), clamp(p.y, self.map_h, self.h))

    def snap(self, target):
        """Jump straight to `target` (centred), e.g. at a restart."""
        self.pos = self._clamped(target - pygame.Vector2(self.w, self.h) / 2)

    def follow(self, target, dt):
        """Glide towards `target`: each frame cover a fraction (CAMERA_LERP * dt) of
        the remaining distance, so the camera moves fast when far and slows down
        as it arrives (smooth, no jitter)."""
        goal = self._clamped(target - pygame.Vector2(self.w, self.h) / 2)
        self.pos += (goal - self.pos) * min(1.0, config.CAMERA_LERP * dt)

    @property
    def rect(self):
        """The visible part of the map as a pygame.Rect (whole pixels)."""
        return pygame.Rect(int(self.pos.x), int(self.pos.y), self.w, self.h)
```

# character.py (329 lines)

The heart of the game: legs, kinematics, the physics step and drawing (see chapter 8).

| Lines | What happens there |
|---|---|
| 1-13 | Docstring: how the physics works. |
| 14-20 | Imports and the short vector name V. |
| 23-31 | Angle helpers: wrap to -180..180 and turn towards a target the short way. |
| 34-58 | One leg: geometry (scaled and stretched), rest pose, current and target angles, sticky state. |
| 59-60 | Store the angles the player asks for. |
| 62-67 | Turn towards the target at most LEG_MAX_SPEED. |
| 69-75 | Outward angles -> screen rotations. |
| 76-87 | Hip, knee and foot positions (kinematics). |
| 90-100 | The character: head radius and the controlled legs. |
| 102-112 | Back to the start pose. |
| 114-128 | Find a start point where the head and all feet are clear of the rock (long 1-player legs would otherwise start inside the ground and be thrown up). |
| 129-154 | One physics step with the 'undo and hold the leg' check. |
| 156-217 | The position-based physics step: move glue/grip points that sit on the rotor, glue, predict, solve, cap, velocity. |

| Lines | What happens there |
|---|---|
| 219-232 | Save / restore everything a step changes. |
| 234-236 | How deep a foot is in the rock. |
| 238-246 | Feet deeper than allowed and deeper than before. |
| 248-279 | Glued: pull towards the anchor. Free: push out of rock, then static friction on walkable ground (much less on the rotor). |
| 281-286 | Limit a glued foot's depth. |
| 288-297 | Push the head circle out of the rock. |
| 299-312 | Contact flags, tear off over-stretched glue, forget grips of lifted feet. |
| 313-329 | Legs then head; shoe colour by state; optional debug circles. |

### Full source

```python
"""The four-legged character: leg kinematics, physics and drawing.

Physics is position based: every step the body falls, then contacts push the
body out of the rock. Because legs are rigid, a foot that is pressed into the
ground by its leg pushes the body away -> extending a grounded leg fast jumps.
Grounded feet on walkable slopes don't slide (static friction), so sweeping a
planted leg moves the body like walking. Sticky feet are anchored both ways,
so they can pull the body too (climb walls, hang from ledges).

Only the body (head) has a position and velocity. Legs have no mass: each is
just two angles (thigh, bend) that follow the player's arm. Where the feet are
follows from the body position plus those angles ("kinematics").
"""
import math

import pygame

import config

V = pygame.Vector2   # short name: V(x, y) is a 2D vector with +, -, *, rotate, length...


def wrap_deg(a):
    """Bring any angle into -180..180 (e.g. 270 -> -90), so 'the shortest way round' works."""
    return (a + 180.0) % 360.0 - 180.0


def approach_angle(current, target, max_step):
    """Move `current` towards `target` by at most `max_step` degrees, the short way round."""
    diff = wrap_deg(target - current)
    return current + max(-max_step, min(max_step, diff))


class Leg:
    def __init__(self, name, hip, side, thigh, bend, scale, stretch=1.0):
        self.name = name
        self.side = side                    # -1 left, +1 right
        self.hip = V(hip) * scale           # offset from body centre
        self.length = config.SEGMENT_LENGTH * stretch * scale   # hip -> knee distance
        # the shin stick stretches, the shoe below it keeps its size (see assets.stretch_leg)
        extra = (config.SHOE_TOP - config.LEG_PIVOT[1]) * (stretch - 1)
        # which way the toe points when the shin hangs down: outward normally (= side),
        # inward for legs in config.MIRRORED_SHOES (they point up, which flips it back out)
        self.shoe_side = -side if name in config.MIRRORED_SHOES else side
        # knee -> shoe centre while the shin hangs straight down; x follows the toe direction
        self.foot_offset = V(self.shoe_side * config.FOOT_OFFSET[0], config.FOOT_OFFSET[1] + extra) * scale
        self.foot_radius = config.FOOT_RADIUS * scale
        self.rest = (thigh, bend)           # pose used at (re)start
        self.thigh, self.bend = thigh, bend  # current angles (outward convention)
        self.target_thigh, self.target_bend = thigh, bend   # angles the player asks for

        self.sticky = False         # player wants this foot sticky (mouth open)
        self.anchor = None          # world point a sticky foot is glued to
        self.grip = None            # world point a grounded foot grips by friction
        self.anchor_on = self.grip_on = None   # moving part (rotor) the anchor/grip rides on
        self.contact = False        # foot touching the ground this step

    # ------------------------------------------------------------ angles --
    def set_target(self, thigh, bend):
        self.target_thigh, self.target_bend = thigh, bend

    def update_angles(self, dt):
        """Turn towards the target, at most LEG_MAX_SPEED deg/s. The limit smooths
        out jerky arm tracking and caps how hard a leg can kick."""
        step = config.LEG_MAX_SPEED * dt
        self.thigh = wrap_deg(approach_angle(self.thigh, self.target_thigh, step))
        self.bend = wrap_deg(approach_angle(self.bend, self.target_bend, step))

    def screen_angles(self):
        """Rotation (deg, pygame convention) of thigh and shin sprites."""
```

```
        # "outward" angles are mirrored for left legs: multiplying by side (-1 / +1)
        # turns them into one screen rotation. shin direction = thigh + bend.
        return self.side * self.thigh, self.side * (self.thigh + self.bend)

    # ------------------------------------------------------- kinematics --
    def joints(self, body):
        """World positions of hip, knee and the foot (shoe) centre."""
        a_thigh, a_shin = self.screen_angles()
        hip = body + self.hip
        # V(0, length) is the thigh hanging straight down; rotating it gives the
        # thigh's real direction. Same for the shin, starting at the knee.
        knee = hip + V(0, self.length).rotate(-a_thigh)
        foot = knee + self.foot_offset.rotate(-a_shin)
        return hip, knee, foot

    def foot(self, body):
        return self.joints(body)[2]


class Character:
    def __init__(self, sprites, spawn, leg_names=tuple(config.LEGS)):
        self.sprites = sprites
        s = sprites.scale
        self.head_radius = config.HEAD_RADIUS * s
        # only the legs someone controls (1 player = 2 legs, otherwise 4)
        self.legs = {}
        for name in leg_names:
            hip, side, thigh, bend = config.LEGS[name]
            self.legs[name] = Leg(name, hip, side, thigh, bend, s, sprites.leg_stretch)
        self.reset(spawn)

    def reset(self, spawn):
        """Back to the start: standing still at `spawn`, legs in their rest pose."""
        self.pos = V(spawn)          # body (head) centre in map pixels
        self.vel = V()               # px per second
        self.head_contact = False
        for leg in self.legs.values():
            leg.thigh, leg.bend = leg.rest
            leg.set_target(*leg.rest)
            leg.anchor = leg.grip = None
            leg.anchor_on = leg.grip_on = None
            leg.contact = False

    def free_spot(self, start, terrain):
        """A start point near `start` where the head and every foot (in the rest pose)
        are out of the rock: lift it up, or if a ceiling is in the way, try a little
        to the side. Without this, long legs can start inside the ground and the
        solver throws the character into the air."""
        for dx in (0, 40, -40, 80, -80, 120, -120, 160, -160):
            for dy in range(0, -400, -4):
                p = V(start[0] + dx, start[1] + dy)
                if terrain.distance(*p) < self.head_radius + 2:
                    break                          # head would hit a ceiling: try another column
                if all(terrain.distance(*leg.foot(p)) >= leg.foot_radius + 2 for leg in self.legs.values()):
                    return p
        return V(start)

    # ------------------------------------------------------------ physics --
    def update(self, dt, terrain):
        """One physics step. While a foot is glued the body is pinned and can't
        always get out of the way, so a leg could be forced into the rock. Then
        the step is redone with that leg held still: it stops at the rock.
        Feet may only sink in a little: free feet LEG_BLOCK_TOLERANCE, sticky
        feet STICKY_SINK (looks like gripping). Glued feet pushed in by another
        leg are kept within STICKY_SINK by the solver (_cap_sink)."""
        before = self._snapshot()                  # to undo this step if needed
        depth0 = {leg.name: self._foot_depth(leg, terrain) for leg in self.legs.values()}
        old = {leg.name: (leg.thigh, leg.bend) for leg in self.legs.values()}
        for leg in self.legs.values():
            leg.update_angles(dt)
```

```
        new = {leg.name: (leg.thigh, leg.bend) for leg in self.legs.values()}
        self._step(dt, terrain)

        # Did a moving leg drive its own foot into the rock while a foot is glued?
        sunk = self._sunk_feet(depth0, terrain)
        if not sunk or not any(leg.anchor is not None for leg in self.legs.values()):
            return
        hold = [leg for leg in sunk if new[leg.name] != old[leg.name]]
        if hold:
            # undo the step and redo it with those legs not moving: they stop at the rock
            self._restore(before)
            for leg in hold:
                leg.thigh, leg.bend = old[leg.name]
            self._step(dt, terrain)

    def _step(self, dt, terrain):
        self._rotor_hits = []       # (rotor, normal, point) of free feet / head pushed by a rotor
        # 0) glue and grip points on a moving part (windmill rotor) move with it
        for leg in self.legs.values():
            if leg.anchor is not None and leg.anchor_on is not None:
                leg.anchor = leg.anchor_on.carry(leg.anchor)
            if leg.grip is not None and leg.grip_on is not None:
                leg.grip = leg.grip_on.carry(leg.grip)

        # 1) sticky feet: glue to the rock when close enough; let go when not sticky
        for leg in self.legs.values():
            if not leg.sticky:
                leg.anchor = None
            elif leg.anchor is None:
                foot = leg.foot(self.pos)
                if terrain.distance(*foot) < leg.foot_radius + config.STICKY_GRAB_DIST:
                    leg.anchor = foot
                    leg.anchor_on = terrain.mover_at(*foot)   # None = still rock

        # 2) predict: gravity speeds the body up, drag slows it a little, then move
        start = V(self.pos)
        self.vel.y += config.GRAVITY * dt                # +y is down on screen
        self.vel *= max(0.0, 1.0 - config.AIR_DRAG * dt)
        self.pos += self.vel * dt
        predicted = V(self.pos)

        # 3) solve: fix contacts by moving the body. Each fix can disturb another,
        #    so go round all of them a few times (Gauss-Seidel iterations).
        for _ in range(config.SOLVER_ITERATIONS):
            for leg in self.legs.values():
                self._solve_foot(leg, terrain)
            self._solve_head(terrain)
        for _ in range(2):          # last word goes to the rock: no glued foot deeper than allowed
            for leg in self.legs.values():
                if leg.anchor is not None:
                    self._cap_sink(leg, terrain)

        # 4) velocity = how far the body really moved this step / time.
        #    This is what turns a fast leg push into a jump: the push moved the
        #    body, so it now has speed and keeps flying after the foot leaves.
        self._update_contacts(terrain)
        self.vel = (self.pos - start) / dt
        # A rotor that overlapped a leg deeply pushes it out in one step, which would
        # look like a huge kick. Leaving a blade may only be ROTOR_PUSH_SPEED faster
        # than the blade itself moves there (glued feet are not limited).
        for rotor, n, p in self._rotor_hits:
            away = self.vel.dot(n) - rotor.velocity_at(p).dot(n)
            if away > config.ROTOR_PUSH_SPEED:
                self.vel -= n * (away - config.ROTOR_PUSH_SPEED)
        if self.head_contact and not any(l.contact for l in self.legs.values()):
            self.vel *= max(0.0, 1.0 - config.HEAD_FRICTION * dt)   # head sliding on rock slows down
        if self.pos != predicted:                       # something pushed or pulled the body
            if self.vel.length() > config.MAX_SPEED:    # pushes and kicks: limit the speed
                self.vel.scale_to_length(config.MAX_SPEED)
        else:
```

```
            # in free flight only the fall is limited. Limiting the whole speed here
            # would stop the fall from speeding up after a fast sideways fling, and
            # the character would drift down slowly as if gliding.
            self.vel.y = min(self.vel.y, config.MAX_SPEED)
        # stay inside the map horizontally and not below its bottom
        self.pos.x = min(max(self.pos.x, 0), terrain.width)
        self.pos.y = min(self.pos.y, terrain.height)

    def _snapshot(self):
        """Copy of everything _step changes, so a step can be undone."""
        legs = {leg.name: (V(leg.anchor) if leg.anchor is not None else None,
                           V(leg.grip) if leg.grip is not None else None,
                           leg.contact, leg.anchor_on, leg.grip_on) for leg in self.legs.values()}
        return V(self.pos), V(self.vel), self.head_contact, legs

    def _restore(self, snap):
        pos, vel, self.head_contact, legs = snap
        self.pos, self.vel = V(pos), V(vel)
        for leg in self.legs.values():
            anchor, grip, leg.contact, leg.anchor_on, leg.grip_on = legs[leg.name]
            leg.anchor = V(anchor) if anchor is not None else None
            leg.grip = V(grip) if grip is not None else None

    def _foot_depth(self, leg, terrain):
        """How far the foot circle is inside the rock (<= 0: outside)."""
        return leg.foot_radius - terrain.distance(*leg.foot(self.pos))

    def _sunk_feet(self, depth0, terrain):
        """Feet deeper in the rock than allowed and deeper than before this step."""
        out = []
        for leg in self.legs.values():
            limit = config.STICKY_SINK if leg.sticky else config.LEG_BLOCK_TOLERANCE
            d = self._foot_depth(leg, terrain)
            if d > max(limit, depth0[leg.name]) + 0.05:   # no creeping in, step by step
                out.append(leg)
        return out

    def _solve_foot(self, leg, terrain):
        foot = leg.foot(self.pos)
        if leg.anchor is not None:                        # sticky: pull or push
            # move the body part of the way so the foot gets back to its glue point
            self.pos += (leg.anchor - foot) * config.STICKY_STIFFNESS
            self._cap_sink(leg, terrain)                  # ...but never deeper than STICKY_SINK
            return
        d = terrain.distance(*foot)                       # distance of the shoe centre to rock
        if d >= leg.foot_radius:                          # shoe circle fully in the air: nothing to do
            return
        n = terrain.normal(*foot)
        mover = terrain.mover_at(*foot) if terrain.movers else None   # touching the rotor?
        if mover is not None:
            self._rotor_hits.append((mover, n, foot))
        self.pos += n * (leg.foot_radius - d)             # push out of the rock
        # Friction only on walkable ground: -n.y is how much the surface faces up
        # (1 = flat floor, 0 = vertical wall). Steeper than WALKABLE_SLOPE: slide.
        # A moving part (windmill rotor) has its own, much smaller limit, so a
        # non-sticky foot slips off it easily (glued feet never get here).
        slope = mover.walkable_slope if mover is not None else config.WALKABLE_SLOPE
        if -n.y < math.cos(math.radians(slope)):
            leg.grip = None                               # too steep, slide
            return
        foot = leg.foot(self.pos)
        if leg.grip is None:                              # first touch: remember where
            leg.grip = V(foot)
            leg.grip_on = mover                           # standing on the rotor: grip rides along
        # tangent = along the surface. Move the body so the foot does not slide
        # along the surface away from its grip point. If the leg sweeps backwards,
        # the foot stays put and the BODY moves forward instead = walking.
        tangent = V(-n.y, n.x)
        self.pos += tangent * (leg.grip - foot).dot(tangent)   # static friction

```

```
281     def _cap_sink(self, leg, terrain):
282         """Push the body so this glued foot is at most STICKY_SINK inside the rock."""
283         foot = leg.foot(self.pos)
284         excess = leg.foot_radius - config.STICKY_SINK - terrain.distance(*foot)
285         if excess > 0:
286             self.pos += terrain.normal(*foot) * excess
287 
288     def _solve_head(self, terrain):
289         """The head is a circle too: push it out of the rock like a foot (no friction)."""
290         d = terrain.distance(*self.pos)
291         self.head_contact = d < self.head_radius + config.CONTACT_EPS
292         if d < self.head_radius:
293             n = terrain.normal(*self.pos)
294             mover = terrain.mover_at(*self.pos) if terrain.movers else None
295             if mover is not None:
296                 self._rotor_hits.append((mover, n, V(self.pos)))
297             self.pos += n * (self.head_radius - d)
298 
299     def _update_contacts(self, terrain):
300         """After solving: which feet touch the ground, and tear off over-stretched glue."""
301         for leg in self.legs.values():
302             foot = leg.foot(self.pos)
303             if leg.anchor is not None:
304                 leg.contact = True
305                 if foot.distance_to(leg.anchor) > config.STICKY_BREAK_DIST:
306                     leg.anchor = None                   # torn off
307                 continue
308             leg.contact = terrain.distance(*foot) < leg.foot_radius + config.CONTACT_EPS
309             if not leg.contact:
310                 leg.grip = None                         # lifted: forget the friction point
311 
312     # ------------------------------------------------------------- drawing --
313     def draw(self, target, offset, debug=False):
314         body = self.pos - offset                        # map position -> screen position
315         sp = self.sprites
316         for leg in self.legs.values():                  # legs first, so the head covers the hips
317             hip, knee, foot = leg.joints(body)
318             a_thigh, a_shin = leg.screen_angles()
319             sp.thigh[leg.side].draw(target, hip, a_thigh)
320             # shoe colour: purple = glued to rock, green = sticky in the air, red = normal
321             shin = (sp.shin_stuck if leg.anchor is not None
322                     else sp.shin_sticky if leg.sticky else sp.shin)
323             shin[leg.shoe_side].draw(target, knee, a_shin)
324             if debug:   # F1: collision circles (green glued, yellow touching, red in air)
325                 color = (0, 255, 0) if leg.anchor else (255, 255, 0) if leg.contact else (255, 0, 0)
326                 pygame.draw.circle(target, color, foot, leg.foot_radius, 1)
327         sp.head.draw(target, body, 0)
328         if debug:
329             pygame.draw.circle(target, (255, 0, 255), body, self.head_radius, 1)
```

## assets.py (110 lines)

Prepares the character pictures (see chapter 10).

| Lines | What happens there |
|---|---|
| 1-9 | Docstring and imports. |
| 12-17 | An image plus the joint it rotates around. |
| 19-29 | Rotate around the joint (see chapter 10). |
| 31-35 | Mirror image with mirrored joint. |
| 38-47 | Crop the empty canvas away, scale, move the joint along. |
| 50-64 | Make a leg picture longer without stretching the shoe. |
| 67-82 | Turn the red shoe green or purple while keeping its shading. |
| 85-110 | Build all pictures once: head, thigh, shin in red/green/purple, each for left and right. |

**Full source**

```python
"""Character images: scaled, cropped, mirrored and tinted copies of the part pngs.

Everything here is done once when a character is built, so the game loop only
has to rotate and paste ready-made images.
"""
import numpy as np
import pygame

import config


class PivotSprite:
    """An image that is drawn rotated around a pivot point (e.g. a joint)."""

    def __init__(self, image, pivot):
        self.image = image
        self.pivot = pygame.Vector2(pivot)   # pivot position inside the image (px)

    def draw(self, target, pos, angle):
        """Draw with the pivot at `pos`. angle (deg) turns the 'hanging down'
        sprite counter-clockwise on screen, same as pygame.transform.rotate."""
        # rotozoom rotates (smoothly) but makes a bigger image with the picture
        # centred in it, so the pivot moves. Fix: take the vector from the pivot to
        # the image centre, rotate that vector by the same angle, and put the
        # rotated image's centre at pos + that vector -> the pivot ends at pos.
        img = pygame.transform.rotozoom(self.image, angle, 1.0)
        centre = pygame.Vector2(self.image.get_size()) / 2 - self.pivot
        # Vector2.rotate turns the other way on screen (y points down), hence -angle
        target.blit(img, img.get_rect(center=pos + centre.rotate(-angle)))

    def flipped(self):
        """Mirror image (left <-> right); the pivot is mirrored too."""
        w = self.image.get_width()
        return PivotSprite(pygame.transform.flip(self.image, True, False),
                           (w - self.pivot.x, self.pivot.y))


def _crop_scale(surface, pivot, scale):
    """Crop a canvas-sized png to its content and scale it; keep the pivot aligned."""
    # bounding rect = smallest rectangle around the visible pixels (+2 px margin);
    # cropping the empty canvas away makes rotating each frame much cheaper
    rect = surface.get_bounding_rect(min_alpha=1).inflate(4, 4).clip(surface.get_rect())
    part = surface.subsurface(rect).copy()
    size = (max(1, round(rect.w * scale)), max(1, round(rect.h * scale)))
    part = pygame.transform.smoothscale(part, size)
    # the pivot was measured on the full canvas: shift it by the crop, then scale it
    return PivotSprite(part, ((pivot[0] - rect.x) * scale, (pivot[1] - rect.y) * scale))


def stretch_leg(surface, k, split):
    """Lengthen a leg png by k: rows between the pivot and `split` are stretched,
    rows below `split` (the shoe) are only moved down. The pivot stays put."""
    if k == 1:
        return surface
    w, h = surface.get_size()
    top = int(config.LEG_PIVOT[1])
    split = min(split, h)
    mid_h = round((split - top) * k)                        # new height of the stick part
    out = pygame.Surface((w, top + mid_h + h - split), pygame.SRCALPHA)   # transparent canvas
    out.blit(surface, (0, 0), (0, 0, w, top))               # 1) above the pivot: unchanged
    mid = surface.subsurface((0, top, w, split - top))
    out.blit(pygame.transform.smoothscale(mid, (w, mid_h)), (0, top))   # 2) the stick: stretched
    out.blit(surface, (0, top + mid_h), (0, split, w, h - split))       # 3) the shoe: moved down
    return out


def recolor_red(surface, weights):
    """Filter that recolours the red shoe while keeping its shading.

    On reddish pixels the 'redness' (R above the grey level) is redistributed
```

```
    over the channels with `weights` (r, g, b), e.g. (0, 1, 0) = green.
    """
    out = surface.copy()
    rgb = pygame.surfarray.pixels3d(out)     # numpy view of the pixels: writes change `out`
    r, g, b = (rgb[..., i].astype(np.float32) for i in range(3))
    mask = (r > g + 40) & (r > b + 40)       # "clearly red" pixels only (not white laces/black lines)
    grey = (g + b) / 2                       # brightness without the red part
    for i, w in enumerate(weights):
        # new channel = grey + redness * weight -> bright red stays bright, dark stays dark
        rgb[..., i][mask] = np.clip(grey + (r - grey) * w, 0, 255)[mask].astype(np.uint8)
    del rgb                                  # release the pixel lock so the surface can be drawn
    return out


class CharacterSprites:
    """Head, thigh and shin(+shoe) sprites at game scale.

    Leg sprites hang straight down from their pivot; shoes point right (+x),
    so left legs use mirrored copies to keep the toes pointing outward.
    """

    def __init__(self, scale, leg_stretch=1.0):
        def load(name):
            return pygame.image.load(config.asset(name)).convert_alpha()

        self.scale = scale                   # game px per png px
        self.leg_stretch = leg_stretch       # 1.5 for the 1-player long legs
        self.head = _crop_scale(load("head.png"), config.HEAD_CENTER, scale)
        thigh = _crop_scale(stretch_leg(load("leg-part.png"), leg_stretch, config.SPRITE_CANVAS),
                            config.LEG_PIVOT, scale)
        shin = _crop_scale(stretch_leg(load("leg-part-with-shoe.png"), leg_stretch, config.SHOE_TOP),
                           config.LEG_PIVOT, scale)
        green = PivotSprite(recolor_red(shin.image, (0.0, 1.0, 0.0)), shin.pivot)
        purple = PivotSprite(recolor_red(shin.image, (0.7, 0.0, 1.0)), shin.pivot)

        # [side] -> sprite, side = -1 (left) or +1 (right)
        self.thigh = {+1: thigh, -1: thigh.flipped()}
        self.shin = {+1: shin, -1: shin.flipped()}              # normal: red shoe
        self.shin_sticky = {+1: green, -1: green.flipped()}     # sticky, in the air
        self.shin_stuck = {+1: purple, -1: purple.flipped()}    # sticky and glued to rock
```

## inputs.py (67 lines)

The input interface and the keyboard input.

| Lines | What happens there |
|---|---|
| 1-7 | Docstring and imports. |
| 10-27 | The interface: every input offers these methods. |
| 30-45 | Keyboard input: key layout and mouth keys. |
| 47-49 | Start every leg at its rest pose. |
| 51-67 | Turn held keys into angle changes, then hand each leg's pose to the player/limb that drives it. |

### Full source

```python
"""Input sources. The game only talks to InputSource, so camera and keyboard are swappable."""
import time

import pygame

import config
from player_state import LimbPose, PlayerState


class InputSource:
    """The interface every input must offer. Subclasses override these methods;
    the game calls them without knowing whether a camera or a keyboard is behind."""
    num_players = 1

    def set_num_players(self, n):
        self.num_players = n

    def get_players(self):
        """List of PlayerState, one per player slot."""
        raise NotImplementedError     # a subclass must provide this

    def get_preview(self):
        """(RGB numpy frame or None, is_new) for the camera panel."""
        return None, False

    def close(self):
        pass


class KeyboardInput(InputSource):
    """Debug controls without a camera. Keys move a character leg directly; the
    pose is handed to whichever player/limb drives that leg in the current scheme.

    top-left    Q/A thigh, W/S knee      top-right    E/D thigh, R/F knee
    bottom-left U/J thigh, I/K knee      bottom-right O/L thigh, P/; knee
    hold 1-4 = that player's mouth open (sticky)
    """
    SPEED = 240.0   # deg/s
    KEYS = {  # leg -> (thigh +, thigh -, bend +, bend -)
        "upper_left": (pygame.K_q, pygame.K_a, pygame.K_w, pygame.K_s),
        "upper_right": (pygame.K_e, pygame.K_d, pygame.K_r, pygame.K_f),
        "lower_left": (pygame.K_u, pygame.K_j, pygame.K_i, pygame.K_k),
        "lower_right": (pygame.K_o, pygame.K_l, pygame.K_p, pygame.K_SEMICOLON),
    }
    MOUTH_KEYS = [pygame.K_1, pygame.K_2, pygame.K_3, pygame.K_4]

    def __init__(self):
        self.poses = {leg: LimbPose(*config.LEGS[leg][2:]) for leg in self.KEYS}   # start at rest pose
        self._t = time.time()

    def get_players(self):
        now = time.time()
        step = self.SPEED * min(now - self._t, 0.1)     # degrees to turn since last call
        self._t = now
        keys = pygame.key.get_pressed()                 # which keys are held right now
        for leg, (tp, tm, bp, bm) in self.KEYS.items():
            pose = self.poses[leg]
            # True/False count as 1/0: (plus held) - (minus held) = +1, -1 or 0
            pose.thigh += step * (keys[tp] - keys[tm])
            pose.bend += step * (keys[bp] - keys[bm])

        states = [PlayerState(visible=True, face_found=True, mouth_open=bool(keys[self.MOUTH_KEYS[i]]))
                  for i in range(self.num_players)]
        # hand each leg's pose to the player/limb that controls that leg
        for player, limb, leg in config.CONTROL_SCHEMES[self.num_players]:
            states[player].limbs[limb] = self.poses[leg]
        return states
```

## vision.py (384 lines)

Everything about the webcam: the background thread, both AI models and the per-player trackers (see chapter 9 and Part V).

| Lines | What happens there |
|---|---|
| 1-12 | Docstring with the per-frame pipeline. |
| 13-30 | Imports. |
| 31-40 | One detected body: keypoints, confidences, box, face, player number. |
| 41-45 | Box size (closeness to camera). |
| 46-51 | Left/right position. |
| 53-56 | Average of visible head keypoints. |
| 58-62 | Body size for distance limits. |
| 64-80 | Square crop around the head for the face fallback. |
| 83-101 | Load YOLO on the fastest backend (TensorRT > GPU > CPU). |
| 103-112 | Run YOLO, convert results to Person objects. |
| 115-135 | Load MediaPipe (download the model if missing): VIDEO-mode for frames, IMAGE-mode for crops. |
| 136-139 | BGR numpy -> MediaPipe RGB image. |
| 141-149 | Faces on the whole frame, as pixel arrays. |
| 151-165 | Face in a head crop, mapped back to frame pixels. |
| 167-170 | Release the models. |
| 173-192 | Assign faces to bodies by pixel distance, closest first. |
| 195-211 | Number the biggest people left to right. |
| 214-219 | Lip gap / mouth width. |
| 222-241 | Smoothing, hysteresis and timeout for one player's mouth. |
| 244-274 | Ear-line roll angle -> sticky side, with a dead zone. |
| 277-299 | The camera input: models, camera, lock, thread start. |
| 301-302 | Ask the thread to switch player count. |
| 304-314 | Fresh trackers for a new player count. |
| 315-333 | The background loop: read, mirror, process, FPS, preview, store under the lock. |
| 335-368 | One frame -> PlayerStates. |
| 369-371 | Newest states (game thread). |
| 373-378 | Newest preview picture (game thread). |
| 380-384 | Stop the thread, release camera and models. |

### Full source

```python
"""Webcam -> players. YOLO pose for bodies, MediaPipe Face Landmarker for mouths.

Faces are found on the whole frame and given to the body whose head keypoints
are closest in pixels. Players are numbered left -> right on the mirrored image.
Everything runs in a background thread so the game keeps its frame rate.

Per camera frame (VisionInput._loop):
    read frame -> mirror -> YOLO pose (bodies) -> pick/number players
    -> MediaPipe faces -> match faces to bodies -> limb angles, mouth, head tilt
    -> PlayerState per player + annotated preview picture for the game window
"""
import importlib.util
import os
import threading
import time
import urllib.request
from dataclasses import dataclass

import cv2
import mediapipe as mp
import numpy as np
from mediapipe.tasks.python import BaseOptions, vision as mp_vision

import config
from inputs import InputSource
from player_state import L_SH, R_SH, LIMBS, AngleSmoother, PlayerState, limb_poses
from preview import PreviewRenderer


@dataclass
class Person:
    """One body found by the pose model in the current frame."""
    xy: np.ndarray          # (17, 2) keypoints, pixels
    conf: np.ndarray        # (17,)
    box: np.ndarray         # x1, y1, x2, y2
    face: np.ndarray | None = None   # (478, 2) face landmarks, pixels
    player: int | None = None

    # @property: used like a value (p.area), computed when read
    @property
    def area(self):
        """Size of the body's box; bigger = closer to the camera."""
        return float((self.box[2] - self.box[0]) * (self.box[3] - self.box[1]))

    @property
    def center_x(self):
        """Where the person stands left/right: middle of the shoulders (or of the box)."""
        ok = self.conf[[L_SH, R_SH]] > config.KEYPOINT_THR
        if ok.all():
            return float(self.xy[[L_SH, R_SH], 0].mean())
        return float((self.box[0] + self.box[2]) / 2)

    def head_center(self):
        """Average of the visible nose/eye/ear keypoints (0-4), or None."""
        ok = self.conf[:5] > config.KEYPOINT_THR
        return self.xy[:5][ok].mean(axis=0) if ok.any() else None

    def shoulder_width(self):
        """Body size in pixels, used to scale distance limits (far people are small)."""
        if (self.conf[[L_SH, R_SH]] > config.KEYPOINT_THR).all():
            return float(np.linalg.norm(self.xy[L_SH] - self.xy[R_SH]))
        return float(self.box[2] - self.box[0]) / 2

    def head_box(self, shape):
        """Square crop around nose/eyes/ears (fallback face search)."""
        pts = self.xy[:5][self.conf[:5] > config.KEYPOINT_THR]
        if len(pts) < 2:
            return None
        cx, cy = pts.mean(axis=0)
        # span = how spread out the head keypoints are (np.ptp = max - min)
```

```
        span = max(np.ptp(pts[:, 0]), np.ptp(pts[:, 1]), config.MIN_CROP / config.CROP_SCALE)
        cy += config.CROP_DOWN * span            # eyes/ears sit above the mouth: shift down
        half = span * config.CROP_SCALE / 2
        h, w = shape[:2]
        # clip the square to the picture borders
        x1, y1 = int(max(cx - half, 0)), int(max(cy - half, 0))
        x2, y2 = int(min(cx + half, w)), int(min(cy + half, h))
        if x2 - x1 < config.MIN_CROP or y2 - y1 < config.MIN_CROP:
            return None
        return x1, y1, x2, y2


class PoseDetector:
    """YOLO pose on the fastest backend available:
    TensorRT engine (GPU) > PyTorch on GPU > PyTorch on CPU."""

    def __init__(self, model_path=config.POSE_MODEL):
        import torch                    # slow imports, only needed in camera mode
        from ultralytics import YOLO

        gpu = torch.cuda.is_available()                         # NVIDIA GPU usable?
        engine = os.path.splitext(model_path)[0] + ".engine"    # yolo26n-pose.engine
        # the engine file alone is not enough: the tensorrt package must be installed too
        trt = importlib.util.find_spec("tensorrt") is not None
        if gpu and trt and config.USE_TENSORRT and model_path.endswith(".pt") and os.path.exists(engine):
            model_path, self.backend = engine, "TensorRT"
        else:
            self.backend = "GPU" if gpu else "CPU"
        self.device = 0 if gpu else "cpu"
        self.model = YOLO(model_path, task="pose")
        print(f"Pose model: {model_path} on {self.backend}")

    def detect(self, frame):
        """All people in the frame as Person objects (keypoints + box)."""
        r = self.model.predict(frame, verbose=False, device=self.device)[0]   # [0] = first (only) image
        if r.keypoints is None or len(r.keypoints) == 0 or r.keypoints.conf is None:
            return []
        # results live on the GPU as torch tensors: .cpu().numpy() makes plain arrays
        xy = r.keypoints.xy.cpu().numpy()
        cf = r.keypoints.conf.cpu().numpy()
        boxes = r.boxes.xyxy.cpu().numpy()
        return [Person(p, c, b) for p, c, b in zip(xy, cf, boxes)]


class FaceDetector:
    """MediaPipe Face Landmarker: 478 points per face (eyes, lips, outline...)."""

    def __init__(self, max_faces):
        if not os.path.exists(config.FACE_MODEL):
            print("Downloading face_landmarker.task ...")
            urllib.request.urlretrieve(config.FACE_URL, config.FACE_MODEL)

        def make(mode, n):
            return mp_vision.FaceLandmarker.create_from_options(mp_vision.FaceLandmarkerOptions(
                base_options=BaseOptions(model_asset_path=config.FACE_MODEL),
                running_mode=mode, num_faces=n))

        # VIDEO mode: for one continuous stream (it reuses the last frame's result
        # to track faces faster). IMAGE mode: every picture is independent, needed
        # for the head crops, which come from different people each time.
        self.full = make(mp_vision.RunningMode.VIDEO, max_faces)
        self.crop = make(mp_vision.RunningMode.IMAGE, 1) if config.FACE_CROP_FALLBACK else None
        self._last_ts = -1

    @staticmethod
    def _mp_image(bgr):
        # OpenCV pictures are BGR, MediaPipe wants RGB in one continuous memory block
        rgb = np.ascontiguousarray(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB))
        return mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)

```

```python
    def detect_full(self, frame):
        """Faces on the whole frame, as (478, 2) arrays of pixel positions."""
        # VIDEO mode needs a timestamp (ms) that always increases
        ts = max(int(time.monotonic() * 1000), self._last_ts + 1)
        self._last_ts = ts
        res = self.full.detect_for_video(self._mp_image(frame), ts)
        h, w = frame.shape[:2]
        # landmarks come as 0..1 fractions of the picture: multiply by its size
        return [np.array([(p.x * w, p.y * h) for p in face]) for face in res.face_landmarks]

    def detect_crop(self, frame, box):
        """Face inside one head crop (fallback for faces too small on the full frame)."""
        if self.crop is None:
            return None
        x1, y1, x2, y2 = box
        crop = frame[y1:y2, x1:x2]
        ch, cw = crop.shape[:2]
        scale = max(1.0, config.UPSCALE_TO / max(ch, cw))
        if scale > 1.0:                          # enlarge tiny crops so the model sees detail
            crop = cv2.resize(crop, None, fx=scale, fy=scale, interpolation=cv2.INTER_LINEAR)
        res = self.crop.detect(self._mp_image(crop))
        if not res.face_landmarks:
            return None
        # crop fractions -> crop pixels -> full-frame pixels (add the crop's corner)
        return np.array([(p.x * cw + x1, p.y * ch + y1) for p in res.face_landmarks[0]])

    def close(self):
        self.full.close()
        if self.crop:
            self.crop.close()


def match_faces(persons, faces):
    """Give each face to the person whose head keypoints are closest (greedy)."""
    # 1) every (person, face) pair that is close enough, with its pixel distance
    pairs = []
    for i, p in enumerate(persons):
        head = p.head_center()
        if head is None:
            continue
        limit = max(config.FACE_MATCH_MIN_PX, config.FACE_MATCH_FACTOR * p.shoulder_width())
        for j, f in enumerate(faces):
            d = float(np.linalg.norm(f.mean(axis=0) - head))   # face centre <-> head centre
            if d < limit:
                pairs.append((d, i, j))
    # 2) closest pairs first; each person and each face is used at most once
    used_p, used_f = set(), set()
    for _, i, j in sorted(pairs):
        if i not in used_p and j not in used_f:
            persons[i].face = faces[j]
            used_p.add(i)
            used_f.add(j)


def assign_players(persons, frame_w, n):
    """Pick the n biggest people and number them left -> right on screen."""
    chosen = sorted(persons, key=lambda p: -p.area)[:n]   # biggest = closest = the players
    chosen.sort(key=lambda p: p.center_x)                  # then order them left to right
    if len(chosen) == n:
        slots = list(range(n))
    else:   # fewer people than players: use the screen region they stand in
        slots, taken = [], set()
        for p in chosen:
            # split the picture into n equal strips; the strip you stand in = your number
            s = min(n - 1, max(0, int(p.center_x / frame_w * n)))
            while s in taken:
                s = (s + 1) % n
            taken.add(s)
            slots.append(s)
    for p, s in zip(chosen, slots):
```

```
        p.player = s


def mouth_ratio(pts):
    """Lip gap / mouth width. Face-mesh points 13/14 = middle of the upper/lower
    lip, 61/291 = the mouth corners. Dividing by the width makes the number the
    same whether the face is near (big) or far (small) from the camera."""
    width = np.linalg.norm(pts[61] - pts[291])
    return float(np.linalg.norm(pts[13] - pts[14]) / width) if width else 0.0


class MouthTracker:
    """Smoothed mouth opening with hysteresis, per player slot."""

    def __init__(self):
        self.score, self.open, self.last_seen = 0.0, False, 0.0

    def update(self, pts, now):
        if pts is None:                     # face not found this frame
            if now - self.last_seen > config.MOUTH_TIMEOUT:
                self.open, self.score = False, 0.0
            return                          # short drop-outs keep the last state
        self.last_seen = now
        # smoothed score (EMA): half new measurement, half previous value
        self.score = config.MOUTH_EMA * mouth_ratio(pts) + (1 - config.MOUTH_EMA) * self.score
        # Hysteresis: open above 0.35, closed below 0.25, in between keep the state.
        # One single threshold would flicker open/closed when the score hovers near it.
        if self.open and self.score < config.MOUTH_CLOSE_T:
            self.open = False
        elif not self.open and self.score > config.MOUTH_OPEN_T:
            self.open = True


class HeadTilt:
    """Which side sticks, from the sideways tilt (roll) of the head.

    roll = angle of the line from the screen-left ear to the screen-right ear
    (the eyes are used if an ear is not seen). On the mirrored image the
    screen-right ear is the player's right ear, so roll > 0 means the right
    ear is lower = head tilted to the right. Tilt right -> right side, tilt
    left -> left side (HEAD_TILT_INVERT swaps this). Inside +-HEAD_TILT_ANGLE
    the last side is kept.
    """
    PAIRS = ((3, 4), (1, 2))   # ears, then eyes

    def __init__(self):
        self.side = "right"
        self.roll = 0.0

    def update(self, person):
        ok = person.conf > config.KEYPOINT_THR
        for a, b in self.PAIRS:
            if ok[a] and ok[b]:
                left, right = sorted((person.xy[a], person.xy[b]), key=lambda p: p[0])
                # slope angle of the line left ear -> right ear (0 = level head)
                self.roll = float(np.degrees(np.arctan2(right[1] - left[1], right[0] - left[0])))
                break
        else:                    # for-else: runs when the loop found no usable pair
            return self.side
        tilt_right = self.roll > config.HEAD_TILT_ANGLE
        tilt_left = self.roll < -config.HEAD_TILT_ANGLE
        if tilt_right or tilt_left:
            self.side = "right" if tilt_right != config.HEAD_TILT_INVERT else "left"
        return self.side


class VisionInput(InputSource):
    """Camera input. A background thread does all detection; the game thread only
    picks up the newest results. A lock makes sure the game never reads a result
    while the thread is half-way through replacing it."""
```

```
281
282     def __init__(self, source=config.CAMERA_SOURCE, pose_model=config.POSE_MODEL):
283         self.pose = PoseDetector(pose_model)
284         self.faces = FaceDetector(config.MAX_PLAYERS + 2)
285         self.renderer = PreviewRenderer()
286         PreviewRenderer.backend = self.pose.backend
287         self.cap = cv2.VideoCapture(int(source) if str(source).isdigit() else source)
288         if not self.cap.isOpened():
289             raise SystemExit(f"Could not open camera/video source {source!r}")
290
291         self._lock = threading.Lock()
292         self._preview = None
293         self._new_preview = False
294         self._reset_players(self.num_players)
295         self._wanted_n = self.num_players
296         self._running = True
297         # daemon thread: does not keep Python alive when the game window closes
298         self._thread = threading.Thread(target=self._loop, daemon=True)
299         self._thread.start()
300
301     def set_num_players(self, n):
302         self._wanted_n = n          # picked up by the worker thread
303
304     def _reset_players(self, n):
305         """Fresh per-player trackers for a new player count (and its control scheme)."""
306         self.n = n
307         self.scheme = config.CONTROL_SCHEMES[n]
308         self.smoothers = [{limb: AngleSmoother() for limb in LIMBS} for _ in range(n)]
309         self.mouths = [MouthTracker() for _ in range(n)]
310         self.tilts = [HeadTilt() for _ in range(n)] if n in config.HEAD_TILT_PLAYERS else None
311         with self._lock:
312             self._states = [PlayerState() for _ in range(n)]
313
314     # ------------------------------------------------------- worker thread --
315     def _loop(self):
316         fps, t_prev = 0.0, time.time()
317         while self._running:
318             if self._wanted_n != self.n:
319                 self._reset_players(self._wanted_n)
320             ok, frame = self.cap.read()
321             if not ok:
322                 time.sleep(0.01)
323                 continue
324             if config.MIRROR_CAMERA:
325                 frame = cv2.flip(frame, 1)       # 1 = flip left/right, like a mirror
326             states, persons = self._process(frame)
327             now = time.time()
328             fps = 0.9 * fps + 0.1 / max(now - t_prev, 1e-6)   # smoothed frames per second
329             t_prev = now
330             preview = cv2.cvtColor(self.renderer.draw(frame, persons, states, self.scheme, fps),
331                                    cv2.COLOR_BGR2RGB)
332             with self._lock:
333                 self._states, self._preview, self._new_preview = states, preview, True
334
335     def _process(self, frame):
336         """One camera frame -> one PlayerState per player slot."""
337         now = time.time()
338         persons = self.pose.detect(frame)                       # bodies
339         assign_players(persons, frame.shape[1], self.n)         # who is P1, P2, ...
340         match_faces(persons, self.faces.detect_full(frame))     # faces -> bodies
341
342         states = [PlayerState() for _ in range(self.n)]
343         by_slot = {p.player: p for p in persons if p.player is not None}
344         for slot in range(self.n):
345             p = by_slot.get(slot)
346             st = states[slot]
347             if p is not None:
348                 if p.face is None:                    # retry on a head crop
349                     box = p.head_box(frame.shape)
350                     if box is not None:
```

```
                        p.face = self.faces.detect_crop(frame, box)
                poses = limb_poses(p.xy, p.conf)
                st.visible = True
                for limb, smoother in self.smoothers[slot].items():
                    value = smoother.update(poses.get(limb))
                    if value is not None:
                        st.limbs[limb] = value
                st.face_found = p.face is not None
            if self.tilts is not None:
                tilt = self.tilts[slot]
                st.sticky_side = tilt.update(p) if p is not None else tilt.side
                st.head_tilt = tilt.roll
            mouth = self.mouths[slot]
            mouth.update(p.face if p is not None else None, now)
            st.mouth_open, st.mouth_score = mouth.open, mouth.score
        return states, persons

    # ------------------------------------------------------------ main thread --
    def get_players(self):
        with self._lock:                    # wait if the thread is writing right now
            return list(self._states)

    def get_preview(self):
        """Newest annotated camera frame (RGB numpy array) and whether it is new."""
        with self._lock:
            frame, fresh = self._preview, self._new_preview
            self._new_preview = False
        return frame, fresh

    def close(self):
        self._running = False
        self._thread.join(timeout=2)
        self.cap.release()
        self.faces.close()
```

## player_state.py (111 lines)

Data passed from input to game, and the arm-angle maths (see chapter 9).

| Lines | What happens there |
|---|---|
| 1-9 | Docstring with the 17 COCO keypoints. |
| 10-25 | Imports and keypoint index constants. |
| 26-32 | Two angles from one limb. |
| 33-41 | Everything known about one player this frame. |
| 44-53 | Angle helpers. |
| 56-62 | Which keypoints form which limb, by screen side. |
| 65-92 | Torso direction and the two angles for every visible limb. |
| 93-111 | Wrap-safe exponential moving average. |

### Full source

```python
"""What the game needs to know about each human player, plus the limb-angle math.

The pose model gives 17 "keypoints" per person in COCO order:
    0 nose, 1 left eye, 2 right eye, 3 left ear, 4 right ear,
    5 left shoulder, 6 right shoulder, 7 left elbow, 8 right elbow,
    9 left wrist, 10 right wrist, 11 left hip, 12 right hip,
    13 left knee, 14 right knee, 15 left ankle, 16 right ankle
Here they are turned into two angles per limb, which drive one character leg.
"""
import math
from dataclasses import dataclass, field

import numpy as np

import config

# COCO keypoints used here
L_SH, R_SH, L_HIP, R_HIP = 5, 6, 11, 12
ARM_CHAINS = [(5, 7, 9), (6, 8, 10)]        # shoulder, elbow, wrist
LEG_CHAINS = [(11, 13, 15), (12, 14, 16)]   # hip, knee, ankle
LIMBS = ("left_arm", "right_arm", "left_leg", "right_leg")


# @dataclass writes __init__ and printing for us: LimbPose(30.0, -10.0)
@dataclass
class LimbPose:
    """Leg command from one body limb. Degrees, 'outward' convention (see config.LEGS)."""
    thigh: float   # shoulder/hip angle vs torso: 0 = hanging down, + = away from body
    bend: float    # elbow/knee angle: lower segment direction - upper segment direction


@dataclass
class PlayerState:
    """Everything the game reads about one player, refreshed every camera frame."""
    visible: bool = False
    limbs: dict = field(default_factory=dict)   # limb name -> LimbPose
    mouth_open: bool = False
    mouth_score: float = 0.0
    face_found: bool = False
    sticky_side: str | None = None   # "left"/"right": only that side's limb sticks, None = all
    head_tilt: float = 0.0           # deg, sideways head roll; > 0 = tilted to the right (1 player)


def _heading(v):
    """Angle of a screen vector measured from 'down', positive towards +x."""
    # atan2(x, y) instead of the usual atan2(y, x): measures from straight down
    # (screen y grows downward), so an arm hanging down gives 0 degrees.
    return math.degrees(math.atan2(v[0], v[1]))


def _wrap(a):
    """Bring an angle into -180..180."""
    return (a + 180.0) % 360.0 - 180.0


def limb_chains(xy):
    """limb name -> keypoint chain, by screen side (left = smaller x)."""
    # Sides come from where the joints are on screen, not from the model's
    # "left/right" labels: on a mirrored picture those labels can be swapped.
    arms = sorted(ARM_CHAINS, key=lambda c: xy[c[0]][0])
    legs = sorted(LEG_CHAINS, key=lambda c: xy[c[0]][0])
    return dict(zip(LIMBS, arms + legs))


def limb_poses(xy, conf, thr=None):
    """limb name -> LimbPose for every limb whose three keypoints are visible."""
    if thr is None:                 # read now (not at import) so the Options screen can change it
        thr = config.KEYPOINT_THR
    ok = conf > thr                 # True for every keypoint the model is sure enough about
    if not (ok[L_SH] and ok[R_SH]):
```

```
71          return {}
72      # torso axis: shoulders -> hips, or straight down if hips are out of frame
73      if ok[L_HIP] and ok[R_HIP]:
74          down = (xy[L_HIP] + xy[R_HIP]) / 2 - (xy[L_SH] + xy[R_SH]) / 2
75      else:
76          down = np.array([0.0, 1.0])
77      torso = _heading(down)          # leaning the whole body should not swing the legs
78
79      out = {}
80      for name, (a, b, c) in limb_chains(xy).items():     # a = shoulder, b = elbow, c = wrist
81          if not (ok[a] and ok[b] and ok[c]):
82              continue
83          # side = -1 for left limbs: mirror the angles so "away from the body" is
84          # positive on both sides (the same convention as the character's legs)
85          side = -1 if name.startswith("left") else 1
86          upper = side * _wrap(_heading(xy[b] - xy[a]) - torso)   # upper arm direction vs torso
87          lower = side * _wrap(_heading(xy[c] - xy[b]) - torso)   # forearm direction vs torso
88          out[name] = LimbPose(upper, _wrap(lower - upper))       # bend = forearm relative to upper arm
89      return out
90
91
92  @dataclass
93  class AngleSmoother:
94      """EMA on angles (wrap-safe) so jittery keypoints don't shake the legs."""
95      alpha: float | None = None       # None = use config.ANGLE_EMA (read live, the Options screen changes it)
96      value: LimbPose | None = None
97
98      def update(self, new):
99          """Exponential moving average: move a fraction `alpha` of the way to the new
100         measurement. Wrapping the difference keeps 179 -> -179 a 2 degree step,
101         not a 358 degree swing. A missing measurement keeps the last value."""
102         if new is None:
103             return self.value
104         a = config.ANGLE_EMA if self.alpha is None else self.alpha
105         if self.value is None:
106             self.value = LimbPose(new.thigh, new.bend)
107         else:
108             self.value = LimbPose(
109                 _wrap(self.value.thigh + a * _wrap(new.thigh - self.value.thigh)),
110                 _wrap(self.value.bend + a * _wrap(new.bend - self.value.bend)))
111         return self.value
```

## preview.py (85 lines)

Draws the annotated camera picture.

| Lines | What happens there |
|---|---|
| 1-7 | Docstring and imports. |
| 9-15 | Which points to connect: body skeleton, lips, leg labels. |
| 18-60 | Grey skeleton for everybody, controlling limbs thick and coloured, player label, lips. |
| 61-68 | Lip outline, yellow and thicker when open. |
| 69-85 | FPS line and one status line per player. |

### Full source

```python
"""Draws the camera preview: grey bodies, controlling limbs in the player's colour, mouths only."""
import cv2
import numpy as np
from mediapipe.tasks.python.vision import FaceLandmarksConnections

import config
from player_state import limb_chains

# pairs of COCO keypoints joined by a line (the stick figure)
BODY_EDGES = [(0, 1), (0, 2), (1, 3), (2, 4), (5, 6), (5, 11), (6, 12), (11, 12),
              (5, 7), (7, 9), (6, 8), (8, 10), (11, 13), (13, 15), (12, 14), (14, 16)]
# pairs of face-mesh points that outline the lips (MediaPipe provides this list)
LIPS = [(c.start, c.end) for c in FaceLandmarksConnections.FACE_LANDMARKS_LIPS]
LEG_LABELS = {"upper_left": "top-left", "upper_right": "top-right",
              "lower_left": "bottom-left", "lower_right": "bottom-right"}


class PreviewRenderer:
    backend = ""   # shown next to the FPS, e.g. "TensorRT"

    def draw(self, frame, persons, states, scheme, fps):
        img = frame.copy()                  # never draw on the frame the detectors use
        ok_thr = config.KEYPOINT_THR
        for p in persons:
            ok = p.conf > ok_thr
            pts = p.xy.astype(int)          # OpenCV draws at whole pixels
            grey = config.BODY_COLOR_BGR
            # 1) the whole body in grey
            for i, j in BODY_EDGES:
                if ok[i] and ok[j]:
                    cv2.line(img, tuple(pts[i]), tuple(pts[j]), grey, 2, cv2.LINE_AA)
            for k in np.nonzero(ok)[0]:
                cv2.circle(img, tuple(pts[k]), 3, grey, -1)

            # 2) on top: only the limbs this player controls, thick and coloured
            if p.player is not None:
                col = config.PLAYER_COLORS_BGR[p.player]
                chains = limb_chains(p.xy)
                for player, limb, _ in scheme:
                    if player != p.player:
                        continue
                    a, b, c = chains[limb]
                    for i, j in ((a, b), (b, c)):
                        if ok[i] and ok[j]:
                            cv2.line(img, tuple(pts[i]), tuple(pts[j]), col, 5, cv2.LINE_AA)
                    for k in (a, b, c):
                        if ok[k]:
                            cv2.circle(img, tuple(pts[k]), 5, col, -1)
                head = p.head_center()
                if head is not None:
                    cv2.putText(img, f"P{p.player + 1}", (int(head[0]) - 15, int(head[1]) - 50),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.8, col, 2)
            # 3) the mouth outline (the only part of the face mesh that is shown)
            if p.face is not None:
                self._draw_mouth(img, p, states)

        self._draw_hud(img, states, scheme, fps)
        return img

    @staticmethod
    def _draw_mouth(img, p, states):
        face = p.face.astype(int)
        is_open = p.player is not None and states[p.player].mouth_open
        col = (0, 255, 255) if is_open else (255, 255, 255)        # yellow when open (BGR)
        for i, j in LIPS:
            cv2.line(img, tuple(face[i]), tuple(face[j]), col, 2 if is_open else 1, cv2.LINE_AA)

    @staticmethod
    def _draw_hud(img, states, scheme, fps):
        """Text in the top-left corner: FPS and one status line per player."""
```

```
        y = 25
        cv2.putText(img, f"{fps:.0f} FPS {PreviewRenderer.backend}", (10, y),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
        for i, st in enumerate(states):
            y += 25
            legs = ", ".join(LEG_LABELS[leg] for pl, _, leg in scheme if pl == i)
            if not st.visible:
                status = "not found"
            else:
                mouth = "OPEN (sticky)" if st.mouth_open else "closed"
                status = f"mouth {mouth}" if st.face_found else "no face"
                if st.sticky_side is not None:
                    status += f" | sticks {st.sticky_side.upper()} (roll {st.head_tilt:+.0f} deg)"
            cv2.putText(img, f"P{i + 1} [{legs}]: {status}", (10, y),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.55, config.PLAYER_COLORS_BGR[i], 2)
```

# export_engine.py (17 lines)

Builds the TensorRT engine for the pose model (see Part V).

| Lines | What happens there |
|---|---|
| 1-8 | Docstring: when and why to run it. |
| 9-11 | Imports. |
| 13-17 | Export with half precision at 640 px on GPU 0. |

**Full source**

```
"""Build the TensorRT engine for the pose model (takes a few minutes, needs an NVIDIA GPU).

    python export_engine.py

The .engine only works on the GPU / driver / TensorRT version it was built with,
so run this again on another computer or after updating them.
Needs: pip install tensorrt-cu13   (and the CUDA build of torch)
"""
from ultralytics import YOLO

import config

if __name__ == "__main__":
    # format="engine": TensorRT. quantize=16: half-precision numbers (faster, tiny
    # accuracy loss). imgsz=640: the picture size the network works on. device=0: first GPU.
    path = YOLO(config.POSE_MODEL).export(format="engine", quantize=16, imgsz=640, device=0)
    print("saved", path)
```

# download_models.py (45 lines)

Run by the setup scripts: downloads both models and tells you whether the GPU will be used.

| Lines | What happens there |
|---|---|
| 1-11 | Docstring and imports. |
| 14-15 | Work in the project folder (where the game looks for the models). |
| 17-21 | Download the face model if missing. |
| 23-25 | Loading YOLO once downloads the pose model. |
| 27-41 | Report: GPU (and TensorRT engine), NVIDIA GPU that PyTorch cannot use (driver too old), or CPU. |
| 44-45 | Run main() when started directly. |

**Full source**

```python
"""Download the two AI models and report what hardware will be used.

Run by the setup scripts (setup.ps1 / setup.sh) at the end; safe to run again.
    python download_models.py
"""
import importlib.util
import os
import shutil
import urllib.request

import config


def main():
    os.chdir(config.ROOT)            # the game loads the models from the project folder

    if os.path.exists(config.FACE_MODEL):
        print("face model: already there")
    else:
        print("face model: downloading face_landmarker.task ...")
        urllib.request.urlretrieve(config.FACE_URL, config.FACE_MODEL)

    from ultralytics import YOLO      # downloads yolo26n-pose.pt on first use
    YOLO(config.POSE_MODEL)
    print(f"pose model: {config.POSE_MODEL} ready")

    import torch
    if torch.cuda.is_available():
        print(f"GPU: {torch.cuda.get_device_name(0)} - pose detection runs on the GPU")
        engine = os.path.exists(os.path.splitext(config.POSE_MODEL)[0] + ".engine")
        trt = importlib.util.find_spec("tensorrt") is not None
        if engine and trt:
            print("TensorRT engine found - fastest mode")
        elif engine:
            print("(a TensorRT engine file exists, but TensorRT is not installed here - it is not used)")
    elif shutil.which("nvidia-smi"):
        print("NVIDIA GPU found but PyTorch cannot use it - the game will run on the CPU.\n"
              "  Usually the NVIDIA driver is too old for CUDA 13: update it to version 580 or newer,\n"
              "  then run the setup again.")
    else:
        print("no NVIDIA GPU - pose detection runs on the CPU (works, about 9 detections/s)")


if __name__ == "__main__":
    main()
```

## setup.ps1 (99 lines)

Windows setup (double-click `setup.bat`, which runs this with permission to execute scripts).

| Lines | What happens there |
|---|---|
| 1-6 | What it does and how to start it. |
| 7-9 | Options -TensorRT / -NoTensorRT (skip the question); stop on errors; work in the script's folder. |
| 11-14 | Helpers: print a step title; stop with a message if the last command failed. |
| 16 | The pinned PyTorch versions. |
| 18-50 | Find Python 3.12 (`py -3.12`, `python`, or the default install folder); if missing, offer `winget install Python.Python.3.12`. |
| 52-64 | Rebuild .venv if it was made with another Python version; create it (or reuse it); update pip. |
| 66-76 | PyTorch: CUDA build if `nvidia-smi` exists, else the CPU build. |
| 78-79 | The game's packages from requirements.txt. |
| 81-93 | Optional TensorRT: ask (or use the option), install, build the engine. |
| 95-96 | Download the models and report GPU/CPU. |
| 98-99 | Tell the user how to start the game. |

### Full source

```powershell
# All For One - one-time setup for Windows.
# Easiest: double-click setup.bat. Or in PowerShell, from the project folder:
#     powershell -ExecutionPolicy Bypass -File setup.ps1 [-TensorRT | -NoTensorRT]
# Creates .venv (a private Python 3.12 environment for the game), installs the pinned
# packages, picks the CUDA (NVIDIA) or CPU build of PyTorch and downloads the models.
# If Python 3.12 is missing it offers to install it with winget.
param([switch]$TensorRT, [switch]$NoTensorRT)
$ErrorActionPreference = "Stop"
Set-Location $PSScriptRoot

function Step($text) { Write-Host "`n== $text" -ForegroundColor Cyan }
function Check($what) {
    if ($LASTEXITCODE -ne 0) { Write-Host "FAILED: $what" -ForegroundColor Red; exit 1 }
}

$TorchVersion = "torch==2.14.1", "torchvision==0.29.1"

Step "1/6 Looking for Python 3.12"
# The game needs exactly Python 3.12: newer versions do not have packages for
# mediapipe / PyTorch yet. Other Python versions on the computer are left alone.
$Is312 = "import sys; sys.exit(0 if sys.version_info[:2] == (3, 12) else 1)"
function Find-Python312 {
    $default = Join-Path $env:LOCALAPPDATA "Programs\Python\Python312\python.exe"   # python.org / winget
    foreach ($cand in @(@("py", "-3.12"), @("python"), @("python3"), @($default))) {
        try {
            $exe, $rest = $cand
            & $exe @rest -c $Is312 2>$null
            if ($LASTEXITCODE -eq 0) { return , $cand }
        } catch { }
    }
    return $null
}
$py = Find-Python312
if (-not $py -and (Get-Command winget -ErrorAction SilentlyContinue)) {
    Write-Host "Python 3.12 is not installed. It can be added next to any other Python versions."
    $answer = Read-Host "Install Python 3.12 now with winget? [y/N]"
    if ($answer -match '^[yY]') {
        winget install -e --id Python.Python.3.12 --scope user --accept-package-agreements
        $py = Find-Python312
    }
}
if (-not $py) {
    Write-Host "Python 3.12 not found." -ForegroundColor Red
    Write-Host "Install it from https://www.python.org/downloads/ (3.12.x), or run:" -ForegroundColor Red
    Write-Host "    winget install -e --id Python.Python.3.12" -ForegroundColor Red
    Write-Host "then run the setup again." -ForegroundColor Red
    exit 1
}
$exe, $rest = $py
& $exe @rest --version

Step "2/6 Creating the environment in .venv"
if (Test-Path ".venv\Scripts\python.exe") {
    & ".venv\Scripts\python.exe" -c $Is312 2>$null
    if ($LASTEXITCODE -ne 0) {
        Write-Host ".venv was made with another Python version - rebuilding it with Python 3.12"
        Remove-Item -Recurse -Force ".venv"
    }
}
if (-not (Test-Path ".venv\Scripts\python.exe")) {
    & $exe @rest -m venv .venv; Check "creating .venv"
} else { Write-Host ".venv already exists - updating it" }
$vpy = Join-Path $PSScriptRoot ".venv\Scripts\python.exe"
& $vpy -m pip install --upgrade pip --quiet; Check "upgrading pip"

Step "3/6 Installing PyTorch"
$gpu = $null -ne (Get-Command nvidia-smi -ErrorAction SilentlyContinue)
if ($gpu) {
    Write-Host "NVIDIA GPU found - installing the CUDA build (large download, a few minutes)"
    & $vpy -m pip install @TorchVersion --index-url https://download.pytorch.org/whl/cu130
```

```
    Check "installing PyTorch (CUDA)"
} else {
    Write-Host "No NVIDIA GPU - installing the small CPU build"
    & $vpy -m pip install @TorchVersion --index-url https://download.pytorch.org/whl/cpu
    Check "installing PyTorch (CPU)"
}

Step "4/6 Installing the game's packages"
& $vpy -m pip install -r requirements.txt; Check "installing requirements.txt"

Step "5/6 Optional: TensorRT (about 2x faster pose detection)"
$wantTrt = $false
if ($gpu -and -not $NoTensorRT) {
    if ($TensorRT) { $wantTrt = $true }
    else {
        $answer = Read-Host "Install TensorRT and build the engine? Takes ~10 minutes [y/N]"
        $wantTrt = $answer -match '^[yY]'
    }
}
if ($wantTrt) {
    & $vpy -m pip install -r requirements-gpu.txt; Check "installing TensorRT"
    & $vpy export_engine.py; Check "building the TensorRT engine"
} else { Write-Host "skipped (the game works without it)" }

Step "6/6 Downloading the AI models"
& $vpy download_models.py; Check "downloading the models"

Write-Host "`nDone! Start the game with:  run.bat" -ForegroundColor Green
Write-Host "(or: .venv\Scripts\python.exe main.py)" -ForegroundColor Green
```

## setup.sh (119 lines)

Linux setup. Same steps as setup.ps1 plus Linux checks (venv package, OpenCV system library, webcam permission).

| Lines | What happens there |
|---|---|
| 1-6 | Run with bash; what it does and how to start it. |
| 7-8 | Stop on any error, unset variable or failed pipe; work in the script's folder. |
| 10-17 | Read the --tensorrt / --no-tensorrt options. |
| 18-20 | Pinned PyTorch versions and print helpers. |
| 22-58 | Find Python 3.12: `python3.12`/`python3`/`python`, else with `uv` if installed, else offer to install uv (or show the deadsnakes alternative) and stop. |
| 60-72 | Rebuild .venv if it was made with another Python version; create it; if that fails, explain the missing python3.12-venv package. |
| 74-83 | PyTorch: CUDA build if an NVIDIA GPU answers `nvidia-smi -L`, else the CPU build. |
| 85-90 | Game packages; check that OpenCV can load (libGL) and explain the fix if not. |
| 92-106 | Optional TensorRT: ask only in an interactive terminal. |
| 108-109 | Download the models and report GPU/CPU. |
| 111-117 | Warn if there is no /dev/video* device or no permission to read it. |
| 119 | Tell the user how to start the game. |

### Full source

```
#!/usr/bin/env bash
# All For One - one-time setup for Linux (x86-64 PC). From the project folder:
#     ./setup.sh [--tensorrt | --no-tensorrt]
# Creates .venv (a private Python 3.12 environment for the game), installs the pinned
# packages, picks the CUDA (NVIDIA) or CPU build of PyTorch and downloads the models.
# If Python 3.12 is missing (e.g. Ubuntu with 3.14) it can get it with uv, without sudo.
set -euo pipefail
cd "$(dirname "$0")"

TRT=ask
for arg in "$@"; do
    case "$arg" in
        --tensorrt) TRT=yes ;;
        --no-tensorrt) TRT=no ;;
        *) echo "unknown option: $arg"; exit 1 ;;
    esac
done
TORCH=(torch==2.14.1 torchvision==0.29.1)
step() { printf '\n\033[36m== %s\033[0m\n' "$1"; }
fail() { printf '\033[31m%s\033[0m\n' "$1"; exit 1; }

step "1/6 Looking for Python 3.12"
# The game needs exactly Python 3.12: newer versions (e.g. Ubuntu's 3.14) do not have
# packages for mediapipe / PyTorch yet. The system Python is never changed.
is312() { "$1" -c 'import sys; sys.exit(0 if sys.version_info[:2] == (3, 12) else 1)' 2>/dev/null; }
uv_python() {   # uv downloads its own Python 3.12 into the home folder (no sudo)
    uv python install 3.12 && PY="$(uv python find 3.12)"
}
PY=""
for cand in python3.12 python3 python; do
    if command -v "$cand" >/dev/null 2>&1 && is312 "$cand"; then PY="$(command -v "$cand")"; break; fi
done
if [ -z "$PY" ] && command -v uv >/dev/null 2>&1; then
    echo "Python 3.12 is not installed - getting it with uv"
    uv_python
fi
if [ -z "$PY" ]; then
    echo "Python 3.12 is not installed (this computer has: $(python3 --version 2>&1))."
    echo "It can be installed for this user only with 'uv' (https://docs.astral.sh/uv/),"
    echo "without sudo and without changing the system Python."
    answer=n
    if [ -t 0 ]; then read -r -p "Install uv and Python 3.12 now? [y/N] " answer; fi
    case "$answer" in
        [yY]*)
            if command -v curl >/dev/null 2>&1; then curl -LsSf https://astral.sh/uv/install.sh | sh
            else wget -qO- https://astral.sh/uv/install.sh | sh; fi
            export PATH="$HOME/.local/bin:$PATH"
            uv_python ;;
        *)
            fail "Install Python 3.12 one of these ways, then run ./setup.sh again:
  a) uv (any Linux, no sudo):  curl -LsSf https://astral.sh/uv/install.sh | sh
                               then: ~/.local/bin/uv python install 3.12
  b) Ubuntu (deadsnakes):      sudo add-apt-repository ppa:deadsnakes/ppa
                               sudo apt install python3.12 python3.12-venv" ;;
    esac
fi
[ -n "$PY" ] && is312 "$PY" || fail "Could not get Python 3.12."
"$PY" --version

step "2/6 Creating the environment in .venv"
if [ -x .venv/bin/python ] && ! is312 .venv/bin/python; then
    echo ".venv was made with $(.venv/bin/python --version 2>&1) - rebuilding it with Python 3.12"
    rm -rf .venv
fi
if [ ! -x .venv/bin/python ]; then
    "$PY" -m venv .venv || fail "Could not create .venv. On Ubuntu/Debian: sudo apt install python3.12-venv
then run ./setup.sh again."
else
    echo ".venv already exists - updating it"
fi
```

```
VPY=.venv/bin/python
"$VPY" -m pip install --upgrade pip --quiet

step "3/6 Installing PyTorch"
GPU=no
if command -v nvidia-smi >/dev/null 2>&1 && nvidia-smi -L >/dev/null 2>&1; then GPU=yes; fi
if [ "$GPU" = yes ]; then
    echo "NVIDIA GPU found - installing the CUDA build (large download, a few minutes)"
    "$VPY" -m pip install "${TORCH[@]}" --index-url https://download.pytorch.org/whl/cu130
else
    echo "No NVIDIA GPU - installing the small CPU build"
    "$VPY" -m pip install "${TORCH[@]}" --index-url https://download.pytorch.org/whl/cpu
fi

step "4/6 Installing the game's packages"
"$VPY" -m pip install -r requirements.txt
if ! "$VPY" -c "import cv2" 2>/dev/null; then
    fail "OpenCV cannot load a system library. On Ubuntu/Debian: sudo apt install libgl1 libglib2.0-0
then run ./setup.sh again."
fi

step "5/6 Optional: TensorRT (about 2x faster pose detection)"
if [ "$GPU" = yes ] && [ "$TRT" = ask ]; then
    if [ -t 0 ]; then
        read -r -p "Install TensorRT and build the engine? Takes ~10 minutes [y/N] " answer
        case "$answer" in [yY]*) TRT=yes ;; *) TRT=no ;; esac
    else
        TRT=no
    fi
fi
if [ "$GPU" = yes ] && [ "$TRT" = yes ]; then
    "$VPY" -m pip install -r requirements-gpu.txt
    "$VPY" export_engine.py
else
    echo "skipped (the game works without it)"
fi

step "6/6 Downloading the AI models"
"$VPY" download_models.py

# webcam check: the camera is a /dev/video* device the user must be allowed to read
if ! ls /dev/video* >/dev/null 2>&1; then
    printf '\033[33mNo webcam found (/dev/video*). Plug one in, or play with: ./run.sh --keyboard\033[0m\n'
elif [ ! -r /dev/video0 ]; then
    printf '\033[33mYou may not have permission to use the webcam. Run:\n'
    printf '  sudo usermod -aG video %s\nthen log out and back in.\033[0m\n' "$USER"
fi

printf '\n\033[32mDone! Start the game with:  ./run.sh\033[0m\n'
```

# requirements.txt (13 lines)

The pinned versions of the game's packages (PyTorch is installed separately by the setup scripts).

| Lines | What happens there |
|---|---|
| 1-5 | Why the versions are pinned and why PyTorch is not in this list. |
| 6-7 | pygame and numpy with their exact tested versions. |
| 8-11 | Both OpenCV packages at one version (they share the `cv2` module). |
| 12-13 | The two AI libraries. |

**Full source**

```
# Packages the game needs, pinned to versions the game was tested with.
# Use the setup script instead of installing this by hand:
#     Windows:  .\setup.ps1          Linux:  ./setup.sh
# The setup script installs PyTorch FIRST (CUDA build on NVIDIA computers, small
# CPU build otherwise), because ultralytics would otherwise pull a default build.
pygame==2.6.1
numpy==2.5.3
# Both OpenCV packages install the same cv2 module (mediapipe wants "contrib", ultralytics
# the plain one); they MUST have the same version or they overwrite each other.
opencv-python==4.11.0.86
opencv-contrib-python==4.11.0.86
mediapipe==1.0.1
ultralytics==8.4.170
```

# Part IV - Reference

## 14. Every setting in config.py

Read straight from the file when this guide was built. Grey rows are the section headers of the file.

| Setting | Value | Meaning |
|---|---|---|
| ROOT | os.path.dirname(os.path.abspath(__file__)) | |
| ASSET_DIR | os.path.join(ROOT, "assets") | |
| [maps] | | |
| MAP_HEIGHT | 2164 | |
| MAPS | (several lines - see the listing) | |
| SPAWN_LIFT | 1 / 8 | start this far (MAP_HEIGHT fraction) above the floor, or mid-gap if lower |
| [map / view] | | |
| VIEW_FRACTION | 1 / 3 | game view height = 1/3 of the map height |
| VIEW_ASPECT | 4096 / 2164 | view width / height (the mountain map's shape) |
| WINDOW_SCALE | 1.0 | max scale of the window (it also shrinks to fit the screen) |
| LAYOUT_MARGIN | 0.025 | gap between window edge / game / camera |
| LAYOUT_CAMERA_WIDTH | 0.40 | camera panel width |
| LAYOUT_SCREEN_FILL | 0.92 | window may use at most this fraction of the desktop |
| LAYOUT_BG | (40, 42, 48) | dark grey around the two panels |
| START_FULLSCREEN | False | F11 toggles fullscreen at any time |
| FPS | 60 | frames drawn per second (the loop waits to keep this rate) |
| PHYSICS_SUBSTEPS | 3 | physics steps per rendered frame (stability) |
| CAMERA_LERP | 6.0 | how fast the game view follows the character (1/s) |
| SKY_COLOR | (135, 206, 235) | fills transparent gaps behind the map images |
| [character size] | | |
| CHARACTER_FRACTION | 1 / 8 | |
| SPRITE_CANVAS | 500 | size of the character png canvases |
| HEAD_CENTER | (250, 250) | |
| HEAD_RADIUS | 95 | collision circle of the head |
| LEG_PIVOT | (249.5, 190.5) | joint at the top of leg-part.png / leg-part-with-shoe.png |
| SEGMENT_LENGTH | 117 | pivot -> pivot of leg-part.png |
| FOOT_OFFSET | (20, 160) | shoe centre from LEG_PIVOT while the sprite hangs down (toe = +x) |
| FOOT_RADIUS | 30 | collision circle of the shoe |
| LEGS | (several lines - see the listing) | |
| MIRRORED_SHOES | {"upper_left", "upper_right"} | |

| Setting | Value | Meaning |
|---|---|---|
| CONTROL_SCHEMES | (several lines - see the listing) | |
| MAX_PLAYERS | 4 | highest player count offered in the menu |
| LEG_STRETCH | {1: 1.5} | |
| SHOE_TOP | 300 | canvas y where the shoe starts on leg-part-with-shoe.png |
| [flag] | | |
| FLAG_SEARCH | 0.1 | fraction of the map width searched for the peak |
| FLAG_FRACTION | 1 / 9 | flag.png canvas height relative to MAP_HEIGHT |
| FLAG_BASE | (160, 492) | canvas px of flag.png that sits on the ground (pole foot) |
| [physics] | | |
| GRAVITY | 2000.0 | px/s^2 (map pixels) |
| MAX_SPEED | 1500.0 | px/s, speed limit while touching; falling speed limit in the air |
| AIR_DRAG | 0.3 | 1/s |
| LEG_MAX_SPEED | 420.0 | deg/s, how fast a leg joint can follow the arm |
| SOLVER_ITERATIONS | 4 | passes over all contacts per physics step (more = stiffer, slower) |
| WALKABLE_SLOPE | 55.0 | deg; feet only get friction on surfaces flatter than this |
| ROTOR_WALKABLE_SLOPE | 10.0 | deg; same for the windmill rotor: slippery (sticky feet still glue) |
| ROTOR_PUSH_SPEED | 400.0 | px/s; a blade can push a free foot/head away at most this much faster than it moves |
| CONTACT_EPS | 2.0 | px; a foot this close to the ground still counts as touching |
| STICKY_GRAB_DIST | 8.0 | px; a sticky foot grabs ground within this distance |
| STICKY_BREAK_DIST | 40.0 | px; a sticky foot tears off if pulled this far from its anchor |
| STICKY_STIFFNESS | 0.6 | share of a glued foot's error fixed per solver pass (1 = all) |
| LEG_BLOCK_TOLERANCE | 3.0 | px a free foot may sink into rock while another foot is glued |
| STICKY_SINK | 6.0 | px a sticky/glued foot may sink into rock while another foot is glued |
| HEAD_FRICTION | 6.0 | 1/s, damping while the head scrapes the ground |
| [vision] | | |
| CAMERA_SOURCE | "0" | webcam index or video path |
| POSE_MODEL | "yolo26n-pose.pt" | YOLO "nano" pose model, downloaded automatically |
| USE_TENSORRT | True | |
| FACE_MODEL | os.path.join(ROOT, "face_landmarker.task") | MediaPipe face model file |
| FACE_URL | ("https://storage.googleapis.com/mediapipe-mod... | |
| MIRROR_CAMERA | True | flip the webcam so it behaves like a mirror |

| Setting | Value | Meaning |
|---|---|---|
| KEYPOINT_THR | 0.5 | keypoints with lower confidence (0..1) count as "not seen" |
| FACE_MATCH_FACTOR | 1.0 | max face-to-head distance = factor * shoulder width |
| FACE_MATCH_MIN_PX | 40 | |
| FACE_CROP_FALLBACK | True | if the full-frame face search misses a player, retry on a head crop |
| CROP_SCALE | 2.4 | head crop size = spread of nose/eyes/ears * this |
| CROP_DOWN | 0.25 | move the crop down (in spreads) so chin and mouth are inside |
| MIN_CROP | 24 | px; smaller heads are skipped (too few pixels for a face mesh) |
| UPSCALE_TO | 256 | small crops are enlarged to this before the face model |
| MOUTH_OPEN_T, MOUTH_CLOSE_T | 0.35, 0.25 | hysteresis on lip gap / lip width |
| MOUTH_EMA | 0.5 | smoothing of the mouth score (1 = no smoothing) |
| MOUTH_TIMEOUT | 0.5 | s; forget the mouth state if the face is lost for this long |
| HEAD_TILT_PLAYERS | {1} | player counts that use this |
| HEAD_TILT_ANGLE | 10.0 | deg the ear line must tilt to switch side (in between: keep last) |
| HEAD_TILT_INVERT | False | True: swap (tilt right -> left-arm leg) |
| ANGLE_EMA | 0.6 | smoothing of arm angles (1 = no smoothing) |
| PLAYER_COLORS_BGR | [(0, 0, 255), (0, 220, 0), (255, 120, 0), (0, ... | |
| PLAYER_COLORS_RGB | [tuple(reversed(c)) for c in PLAYER_COLORS_BGR... | |
| BODY_COLOR_BGR | (150, 150, 150) | grey for non-controlling body parts in the preview |
| [fonts] | | |
| FONT_HEAVY | "arialblack,arial,dejavusans,liberationsans,fr... | titles, buttons |
| FONT_TEXT | "arial,dejavusans,liberationsans,freesans" | normal text |
| FONT_MONO | "consolas,couriernew,dejavusansmono,liberation... | numbers, timer |
| FONT_EMOJI | "segoeuiemoji,notocoloremoji,notoemoji,symbola... | the sheep in the menu |
| [menu] | | |
| GAME_TITLE | "All For One" | |
| GAME_CREDIT | "From The Group MeEeEe" | |
| GAME_CREDIT_EMOJI | "[sheep emoji]" | sheep |

# 15. How do I...?

Most player-facing values can be changed in the **OPTIONS** screen without touching code; the table lists where everything lives in the code.

| I want to... | Change this |
|---|---|
| set the game up on a new computer | Windows: `setup.bat`. Linux: `./setup.sh`. Then `run.bat` / `./run.sh`. |
| tune mouth, legs, physics while playing | The OPTIONS screen (saved in `settings.json`). |
| offer a new value in the Options screen | Add a `Setting` to `SETTINGS` in settings.py with a safe range. The code must read that config value when it uses it (not copy it at start-up). |
| rebuild this PDF after code changes | `pip install -r requirements-dev.txt` once, then `python tools/docs/make_docs.py`. Text is in `tools/docs/content.py`. |
| update a package version | Change the pin in requirements.txt, run the setup again, test. |
| add a map | Put `background-x.png` and `foreground-x.png` (transparent = air) in `assets/` and add an entry to `MAPS` in config.py with a spawn point (fractions of the map) and optionally a `scale`. The flag and the menu picture are automatic. |
| add a windmill to a map | Add a `"windmill"` entry (tower and rotor pictures, hub in image pixels, `rps`) to the map in `MAPS`, like the Windmill map. |
| change the rotor speed / slipperiness | `"rps"` of the map in `MAPS` / `ROTOR_WALKABLE_SLOPE`. |
| clear the score board | Delete `scores.json` (or remove lines from it). |
| make jumps stronger / weaker | `LEG_MAX_SPEED` (how fast legs straighten) or `GRAVITY`. |
| change who controls which leg | `CONTROL_SCHEMES` - each row is (player, limb, leg). |
| make the mouth easier to trigger | Lower `MOUTH_OPEN_T` (and keep `MOUTH_CLOSE_T` about 0.1 lower). |
| make legs smoother / more responsive | `ANGLE_EMA` (lower = smoother, slower). |
| change the character size | `CHARACTER_FRACTION` (relative to the map height). |
| make a map bigger relative to the character | That map's `"scale"` in `MAPS`. |
| change the player colours | `PLAYER_COLORS_BGR` (blue, green, red order!). |
| switch head-tilt direction | `HEAD_TILT_INVERT`. |
| start in fullscreen | `START_FULLSCREEN = True`. |
| use the camera without mirroring | `MIRROR_CAMERA = False`. |

# 16. Glossary

| Word | Meaning here |
|---|---|
| anchor | The point where a sticky foot is glued to the rock. |
| bilinear interpolation | Blending the 4 nearest table values by distance, for smooth results between pixels. |
| blit | Copy (paste) one picture onto another (pygame). |
| canvas | The fixed-size picture every scene draws on. |
| delta time (dt) | Seconds since the last frame/step; movement = speed x dt. |
| EMA | Exponential moving average: smooth = smooth + a x (new - smooth). |
| Gauss-Seidel | Fixing constraints one after another, repeated a few times. |
| hysteresis | Two thresholds (on/off) with a gap so a value near the edge does not flicker. |

| Word | Meaning here |
|---|---|
| inference | Running a trained AI model on new input. |
| keypoint / landmark | A named point found by a model (body joint / face point). |
| kinematics | Computing joint positions from angles and lengths. |
| lock | Makes one thread wait while another changes shared data. |
| normal | Unit vector pointing straight out of a surface. |
| pinned version | An exact package version in requirements.txt (==), so every install is the same. |
| PBD | Position-based dynamics: move, correct positions, derive velocity. |
| SDF | Signed distance field: distance to the nearest surface, negative inside. |
| substep | One of several smaller physics steps inside one frame. |
| tangent | Direction along a surface (normal turned 90 degrees). |
| thread | A part of the program that runs at the same time as others. |
| TensorRT / FP16 | NVIDIA's optimiser for running networks on its GPUs / 16-bit numbers. |
| virtual environment (.venv) | A private Python installation for one project, with its own packages. |

# Part V - How the AI models work

This part explains the two models at the level needed to use them well, without the mathematics of training.

## 17. Pose detection: YOLO

**What it does.** Given a picture, `yolo26n-pose` returns for every person a box, a person-confidence and 17 keypoints (x, y and a confidence 0..1 for each). It is a *neural network*: millions of numbers (weights) learned by training on the COCO dataset, where people were labelled by hand with these 17 points.

**How it works, roughly.** YOLO stands for *You Only Look Once*: the whole picture goes through the network a single time and all people come out together, which is why it is fast enough for live video.

1. The frame is resized to fit 640 x 640 (padding the rest) and turned into numbers.
2. The *backbone* (stacked convolution layers) turns the picture into feature maps: coarse grids where each cell describes what is in its region - edges first, then shapes, then body parts.
3. The *neck* combines feature maps of different sizes, so both near (big) and far (small) people are found.
4. The *head* predicts, for each grid cell, whether a person is centred there, the box, and the 17 keypoint positions with their confidences.
5. Overlapping duplicates are removed, leaving one result per person. (Ultralytics designs the YOLO26 family to do this inside the network, without a separate clean-up step.)

The **n** means *nano*, the smallest and fastest size. Bigger sizes (s, m, l, x) are more accurate on small or far people but slower. **Keypoint confidence** drops for hidden or blurry joints; the game ignores keypoints below 0.5, and a limb with a missing joint simply keeps its last angle.

### Speed: CPU, GPU and TensorRT

A network is mostly multiplication of large tables of numbers, which a graphics card does in parallel. With the CUDA build of PyTorch the model runs on the RTX GPU. **TensorRT** goes further: it compiles the network for *this specific* GPU - merging layers, choosing the fastest routines and using 16-bit numbers (FP16) instead of 32-bit. That is why the `.engine` file only works on the computer it was built on.

| Backend | Pose model per frame | Whole vision step per frame |
|---|---|---|
| CPU | about 51 ms | about 110 ms (9 FPS) |
| GPU (CUDA PyTorch) | about 12 ms | about 33 ms |
| TensorRT FP16 | about 5 ms | about 23 ms (40+ FPS) |

Measured on the development laptop (RTX 4070 Laptop) on a sample photo; the rest of the vision step is mostly the face model, which runs on the CPU.

## 18. Face landmarks: MediaPipe Face Landmarker

**What it does.** For every face it returns **478 landmarks**: 468 points of a face mesh (outline, eyebrows, eyes, nose, lips) plus 10 iris points, each as a position relative to the picture. It can also return 52 *blendshape* scores such as `jawOpen` (your original test script printed this); the game uses the simpler lip-gap ratio instead, because it also works on the head-crop fallback.

**How it works.** It is two networks in a row:

1. A small, very fast **face detector** (BlazeFace) finds where faces are and how they are rotated.
2. Each face is cut out, straightened and resized, and a **landmark network** predicts the positions of all 478 points at once (it *regresses* the coordinates: the output layer simply is the list of x, y, z values).

In **VIDEO mode** (used on the full frame) it *tracks*: the landmarks of the previous frame tell it where the face is now, so the detector can often be skipped - faster and steadier. That is why it needs increasing timestamps. **IMAGE mode** (used for head crops) treats every picture independently, which is right there because consecutive crops can come from different people.

## 19. Limits to keep in mind

- **Distance**: the face model is meant for faces within about two metres; far away, faces get too small. The head-crop fallback enlarges small faces, which helps.
- **Light and blur**: dark rooms and fast arm movement lower keypoint confidence; limbs then hold their last angle.
- **Overlap**: players standing behind each other can swap numbers or hide limbs; standing side by side works best.
- **Side view**: a head turned far sideways hides one ear and much of the mouth; head tilt falls back to the eyes, the mouth may be lost briefly (kept for 0.5 s).
- **Mirroring**: the model's own left/right labels can be wrong on a mirrored picture - the code uses screen positions instead (chapter 9).

> Everything in this guide refers to the code as it is in the repository at the time of writing. The tables in Part III find their line numbers automatically from the real files, so they match the listings.